Important system design

05/07/2026 11:37:10

When designing a system what we should consider in that it has to be imporatnt...

Tool: read_me Here are the key pillars to consider when designing any system: To...

Dont Write it in interactive way just add with heading with explaination and give...

Here are the key pillars of system design, each with an explanation and what to...

You missed extensability , High avaiablity , Resilience

Great catch! Here are the three missing pillars: 11. Extensibility What it means: Th...

Anything missed this kind of things that we must consider

Great question! Here are the remaining important pillars that complete a...

now give design 2 - 3 (1 basic , 1 medium, 1 hard) common design problem with a...

Tool: read_me Here are three classic system design problems — basic, medium,...

Do similar for a notification system, a banking system, and a...

Here are all three designs in full detail. 1. Notification System Functional...

one chat system design also do similar way

Chat System Design (like WhatsApp / Slack / Messenger) Functional Requirements...

Swiggy mart design (add functional, non functional, API design, HLD,Flow details,...

Swiggy Instamart System Design (10-minute grocery delivery platform) Functional...

is my design correct ? if yes how much % , if not what and all need to improve or...

Great effort for a first draft! Your design shows solid foundational thinking. Let m...

Design LinkedIn in same way similar fashion with all details

LinkedIn System Design Functional Requirements User registration, profile creati...

You have not mentioned scalability, availability and all Don’t create interactive...

You're right! Let me cover all the pillars in full detail here. LinkedIn System Design...

Draw the hld and explain the flow

Here is the HLD diagram followed by a detailed flow explanation. View: Check...

i am unable to view this diagram draw like this simple so i can view anywhere

Here is your Uber design redrawn cleanly — same components, much more...

Design Kafka-like System Design YouTube / Netflix Design Uber Design Distribute...

I'll design all 6 systems one by one in full detail. Given the depth required, let me...

Design E-Commerce System Design Notification System Design Banking System...

1. E-COMMERCE SYSTEM Functional Requirements User registration, login, profile...

When designing a system what we should consider in that it has to be imporatnt any System design like

5/5/2026, 1:29:09 PM

```
Tool: read_me
```

Here are the key pillars to consider when designing any system:

```
Tool: show_widget
```

5/5/2026, 1:29:24 PM

Dont Write it in interactive way just add with heading with explaination and give which tool or what we should impliment for that

5/5/2026, 1:30:05 PM

Here are the key pillars of system design, each with an explanation and what to implement:

---

1. Scalability

What it means: The system should handle increased load (users, data, requests) without degrading performance.

Implement:

- Horizontal scaling (add more servers) via load balancers like Nginx or AWS ELB
- Vertical scaling (bigger machines) for databases
- Auto-scaling groups (AWS ASG, Kubernetes HPA)
- Sharding for databases (split data across nodes)

---

2. Availability

What it means: The system should be up and accessible as close to 100% of the time as possible (measured in "nines" — 99.9%, 99.99%).

Implement:

- Multi-region or multi-zone deployments
- Health checks and automatic failover
- Redundant components (no single point of failure)
- Circuit breakers (Hystrix, Resilience4j)

---

3. Reliability

What it means: The system should consistently perform its function correctly, even when parts fail.

Implement:

- Replication (database primary-replica setup)
- Retry logic with exponential backoff
- Message queues for guaranteed delivery (Kafka, RabbitMQ, SQS)
- Distributed tracing to detect failures (Jaeger, Zipkin)

---

4. Performance & Latency

What it means: Responses should be fast; the system should maximize throughput with minimal delay.

Implement:

- Caching at multiple levels (Redis, Memcached, CDN like Cloudflare)
- Database indexing and query optimization
- Asynchronous processing (avoid blocking calls)
- Content Delivery Networks (CDN) for static assets

---

5. Consistency

What it means: All users should see the same data at the same time. In distributed systems, you trade this against availability (CAP theorem).

Implement:

- Strong consistency → use relational DBs with ACID transactions (PostgreSQL, MySQL)

- Eventual consistency → use distributed DBs (Cassandra, DynamoDB)
- Distributed locks (Zookeeper, Redis SETNX)
- Consensus protocols (Raft, Paxos)

---

6. Security

What it means: The system must protect data and prevent unauthorized access.

Implement:

- Authentication & Authorization (OAuth2, JWT, RBAC)
- Encryption in transit (TLS/HTTPS) and at rest (AES-256)
- API Gateway with rate limiting (Kong, AWS API Gateway)
- Input validation to prevent SQL injection, XSS

---

7. Maintainability

What it means: The system should be easy to understand, debug, modify, and extend over time.

Implement:

- Microservices architecture for independent deployability
- Centralized logging (ELK Stack — Elasticsearch, Logstash, Kibana)
- Monitoring & alerting (Prometheus + Grafana, Datadog)
- Well-documented APIs (OpenAPI/Swagger)

---

8. Data Storage & Management

What it means: Choosing the right database and storage strategy based on access patterns and data structure.

Implement:

- Relational DB (PostgreSQL, MySQL) for structured, transactional data
- NoSQL (MongoDB, Cassandra) for flexible or high-volume data
- Object storage (S3, GCS) for files and media
- Data warehouses (Snowflake, BigQuery) for analytics

---

9. Network & Communication

What it means: How components talk to each other — synchronously or asynchronously.

Implement:

- REST or gRPC for synchronous APIs
- Message queues (Kafka, RabbitMQ) for async communication
- WebSockets for real-time bidirectional communication
- Service mesh (Istio, Linkerd) for microservice communication

---

10. Observability

What it means: You must be able to understand the internal state of your system from its outputs — logs, metrics, and traces.

Implement:

- Metrics: Prometheus + Grafana
- Logging: ELK Stack or Loki + Grafana
- Tracing: Jaeger or OpenTelemetry
- Alerting: PagerDuty, OpsGenie

A good way to remember these is the acronym **SCALRM** — Scalability, Consistency, Availability, Latency, Reliability, Maintainability — these are the non-negotiables in any interview or real design.

5/5/2026, 1:30:21 PM

You missed extensability , High avaiablity , Resilience

5/5/2026, 1:31:35 PM

Great catch! Here are the three missing pillars:

---

11. Extensibility

What it means: The system should be designed so new features or components can be added in the future without breaking or rewriting existing code.

Implement:

- Plugin or modular architecture (loosely coupled components)
- Design patterns like Strategy, Observer, Factory
- Event-driven architecture (new consumers can plug in without touching producers)
- Kafka or EventBridge for event streams that new services can subscribe to
- API versioning (v1, v2) so old clients aren't broken when you extend
- Feature flags (LaunchDarkly, Unleash) to roll out new behavior incrementally

---

12. High Availability (HA)

What it means: Availability means "is the system up," but High Availability specifically means designing the system to eliminate every single point of failure so downtime is near-zero, even during failures or deployments.

The difference from Availability: Availability is the goal (99.99% uptime). High Availability is the architectural strategy to achieve it.

Implement:

- Active-Active setup — multiple nodes all serving traffic simultaneously (no idle standby)

- Active-Passive setup — a hot standby takes over instantly on failure
- Multi-AZ and Multi-Region deployments (AWS RDS Multi-AZ, Aurora Global)
- Load balancers with health checks to route away from failed nodes automatically
- Zero-downtime deployments via Blue-Green or Canary releases
- Database replication with automatic failover (PostgreSQL Patroni, MySQL Group Replication)

---

13. Resilience

What it means: The system should gracefully absorb failures, recover quickly, and continue functioning — even in a degraded state — rather than collapsing entirely when something goes wrong.

The difference from Reliability: Reliability means it works correctly under normal conditions. Resilience means it survives and recovers when things go wrong.

Implement:

- Circuit Breaker pattern — stop calling a failing service and return a fallback (Resilience4j, Hystrix)
- Bulkhead pattern — isolate failures so one failing service doesn't drain resources from others
- Retry with exponential backoff and jitter — avoid thundering herd on recovery
- Timeout policies — never let a slow dependency hang your whole request
- Graceful degradation — serve partial or cached responses when a dependency is down
- Chaos Engineering — proactively test resilience by injecting failures (Netflix Chaos Monkey, AWS Fault Injection Simulator)
- Dead Letter Queues (DLQ) in Kafka/SQS to capture and replay failed messages

---

Quick Summary of How These Three Relate

Pillar	Core Question
Extensibility	Can I add new things without breaking old things?
High Availability	Is every single point of failure eliminated?
Resilience	Can the system survive and recover when something breaks?

These three alongside the earlier 10 give you a complete picture of a production-grade system design.

5/5/2026, 1:31:50 PM

Anything missed this kind of things that we must consider

5/5/2026, 1:32:19 PM

Great question! Here are the remaining important pillars that complete a production-grade system design:

---

14. Observability

Already covered — but worth noting it includes Logs, Metrics, and Traces (the three pillars).

---

15. Fault Tolerance

What it means: The system continues operating correctly even when one or more components fail — not just recovering, but not failing in the first place.

The difference from Resilience: Resilience is about recovery. Fault Tolerance is about continuing to work during the failure itself.

Implement:

- Redundancy at every layer (servers, DBs, network paths)
- RAID storage for disk fault tolerance
- Consensus algorithms (Raft, Paxos) so a cluster still works if some nodes die
- Quorum-based reads/writes (Cassandra, DynamoDB)
- Replication factor (Kafka topic replication, MongoDB replica sets)

---

16. Disaster Recovery (DR)

What it means: A plan and infrastructure to recover the entire system after a catastrophic failure — data center fire, region outage, ransomware, accidental deletion.

Key metrics:

- RPO (Recovery Point Objective) — how much data loss is acceptable?
- RTO (Recovery Time Objective) — how fast must you recover?

Implement:

- Regular automated backups (AWS Backup, pg_dump, snapshots)
- Cross-region replication of data
- DR runbooks — documented step-by-step recovery procedures
- Periodic DR drills — actually test your recovery, don't just assume it works
- Pilot Light or Warm Standby strategies for fast recovery

---

17. Data Consistency & Integrity

What it means: Beyond the CAP theorem consistency, ensuring data is never corrupted, duplicated, or lost during writes, failures, or concurrent access.

Implement:

- ACID transactions for critical writes (PostgreSQL, MySQL)
- Idempotency keys — so retried requests don't create duplicate records
- Optimistic locking — detect conflicts before committing
- Checksums and data validation at write time
- Change Data Capture (CDC) with Debezium to track every data change

---

18. Rate Limiting & Throttling

What it means: Protect the system from being overwhelmed by too many requests — whether from a single abusive client or a traffic spike.

Implement:

- Token Bucket or Leaky Bucket algorithms
- API Gateway rate limiting (Kong, AWS API Gateway, Nginx)
- Per-user and per-IP limits
- Redis for distributed rate limit counters
- Graceful 429 Too Many Requests responses with Retry-After headers

---

19. Caching Strategy

What it means: Reducing repeated expensive computations or DB calls by storing results temporarily. A dedicated pillar because wrong caching causes more bugs than no caching.

Key patterns:

- Cache-aside (lazy loading) — app checks cache first, loads from DB on miss
- Write-through — write to cache and DB simultaneously
- Write-behind — write to cache first, DB asynchronously
- Cache eviction policies — LRU, LFU, TTL

Implement:

- Redis or Memcached for application-level cache
- CDN (Cloudflare, CloudFront) for edge caching
- Database query cache
- Always plan for cache invalidation — the hardest problem in caching

---

20. API Design

What it means: How your system exposes its functionality to clients or other services — a bad API design creates coupling and future pain.

Implement:

- REST for standard CRUD-based APIs
- gRPC for high-performance internal service communication
- GraphQL for flexible client-driven queries
- API versioning from day one (/v1/, /v2/)
- Pagination, filtering, sorting on all list endpoints
- OpenAPI/Swagger documentation

---

21. Load Balancing

What it means: Distributing incoming traffic evenly across servers so no single server is overwhelmed.

Algorithms:

- Round Robin — requests distributed in order
- Least Connections — send to the server with fewest active connections
- IP Hash — same client always goes to same server (sticky sessions)

Implement:

- Layer 4 (TCP) load balancing — HAProxy, AWS NLB
- Layer 7 (HTTP) load balancing — Nginx, AWS ALB
- Global load balancing across regions — AWS Route 53, Cloudflare

---

22. Service Discovery

What it means: In a microservices system, services need to find each other dynamically since IPs change constantly.

Implement:

- Client-side discovery — Eureka (Netflix), Consul
- Server-side discovery — AWS ALB, Kubernetes Services
- DNS-based discovery — Consul DNS, Kubernetes CoreDNS
- Service mesh — Istio or Linkerd handles this automatically

---

23. Idempotency

What it means: Performing the same operation multiple times should produce the same result. Critical for retries, payment systems, and distributed workflows.

Implement:

- Unique idempotency keys on every write API
- Deduplication in message queues (Kafka exactly-once, SQS deduplication ID)
- Database upserts instead of blind inserts
- Stripe-style idempotency headers for payment APIs

---

24. Cost Optimization

What it means: A system that works but costs 10x more than necessary is a bad design. Cost is a real constraint.

Implement:

- Right-sizing compute (don't over-provision)
- Spot/Preemptible instances for non-critical workloads
- Auto-scaling down during low traffic

- Tiered storage (hot → warm → cold → archive with S3 lifecycle policies)
- Caching aggressively to reduce DB read costs

Complete System Design Checklist

#	Pillar	One-line question
1	Scalability	Can it grow with load?
2	Availability	Is it always up?
3	High Availability	Is every SPOF eliminated?
4	Reliability	Does it work correctly?
5	Resilience	Does it recover from failure?
6	Fault Tolerance	Does it keep working during failure?
7	Performance	Is it fast enough?
8	Consistency	Is data always correct and in sync?
9	Security	Is it protected from threats?
10	Maintainability	Is it easy to change?
11	Extensibility	Is it easy to add to?
12	Observability	Can you see what's happening inside?
13	Disaster Recovery	Can you recover from catastrophe?
14	Rate Limiting	Is it protected from overload?
15	Caching	Are expensive calls avoided?
16	API Design	Is the interface clean and versioned?
17	Load Balancing	Is traffic distributed evenly?
18	Service Discovery	Can services find each other?
19	Idempotency	Are retries safe?

#	Pillar	One-line question
20	Cost Optimization	Is it affordable to run?

This is a complete, interview-ready and production-ready system design framework.

5/5/2026, 1:32:50 PM

now give design 2 - 3 (1 basic , 1 medium, 1 hard) common design problem with all those imporatant considerations) and add diagram as well

5/5/2026, 1:34:25 PM

```
Tool: read_me
```

Here are three classic system design problems — basic, medium, and hard — each with all key considerations and an architecture diagram.

Problem 1 (Basic) — Design a URL shortener (like bit.ly)

A user pastes a long URL, gets a short code back (e.g. `short.ly/aB3x`), and anyone visiting that short URL gets redirected to the original.

Scalability: Reads (redirects) massively outnumber writes. Use read replicas on the DB and cache hot short codes in Redis so popular links never hit the database.

Performance: The redirect must be near-instant. Cache the top 20% of links (which handle 80% of traffic) in Redis with a TTL. Target under 10ms for cached lookups.

Availability & HA: Run at least two app server instances behind a load balancer. Use Redis Sentinel or Cluster for cache HA. Database with primary + read replica with automatic failover.

Reliability: Every short code write must be durable — use a relational DB (PostgreSQL) with ACID transactions. Never lose a mapping.

Consistency: Eventual consistency is acceptable here. A newly created short URL might take a few hundred milliseconds to propagate to all read replicas — that is fine.

Security: Rate limit the creation API (Redis token bucket, 10 URLs/min per IP) to prevent abuse. Validate that destination URLs are not malicious (blocklist check).

Extensibility: Design the short code generation as a pluggable strategy — Base62 encoding today, could swap to hash-based or custom vanity codes without touching the rest of the system.

Observability: Track redirect counts per short code (analytics), error rates, and cache hit ratio. Use Prometheus + Grafana.

Data storage: PostgreSQL for the `urls` table (`short_code`, `original_url`, `created_at`, `user_id`). Redis for caching. S3 for analytics exports.

Idempotency: If the same long URL is submitted twice by the same user, return the existing short code rather than creating a duplicate.

```
Tool: show_widget
```

Problem 2 (Medium) — Design a ride-sharing app (like Uber/Ola)

Drivers broadcast their location continuously. Riders request a trip. The system matches them, tracks the ride, and processes payment.

Scalability: Location updates arrive from millions of drivers every 3–5 seconds. Use a geospatial index (Redis `GEOADD`) to store and query driver locations. Horizontally scale matching workers.

Performance: Matching must complete in under 2 seconds. Redis geo-queries are O(N+log M) — fast enough. Pre-compute surge zones with background workers.

High Availability: No single matching server. Use consistent hashing to assign drivers to matching nodes — if one node dies, the ring re-balances with no full outage.

Resilience: If the payment service is down, the ride still completes and payment retries via a queue (Kafka + DLQ). Circuit breaker wraps every downstream call.

Fault Tolerance: Driver location data is ephemeral — losing a few updates is acceptable. Trip state (booked, in-progress, completed) is durable in PostgreSQL with WAL replication.

Real-time communication: Use WebSockets (via a connection gateway) for live driver location on the rider's map and trip status updates. Fall back to long polling if WS is unavailable.

Consistency: Trip state machine (requested → matched → in-progress → completed) must be strongly consistent — use DB-level locking or optimistic concurrency to prevent double-matching a driver.

Security: Rider and driver authenticate via JWT. All location data encrypted in transit (TLS). Driver identity verified at onboarding (not at request time).

Observability: Track match latency (P50/P99), driver location staleness, payment failure rate. Alert if match latency exceeds 3 seconds.

Disaster Recovery: Trip data replicated cross-region. RPO = 1 minute (WAL shipping), RTO = 5 minutes (standby promotion).

Idempotency: Payment charged exactly once per trip via an idempotency key — retries to the payment gateway reuse the same key.

```
Tool: show_widget
```

Problem 3 (Hard) — Design a distributed video streaming platform (like YouTube)

Users upload videos. The platform transcodes them into multiple resolutions, stores them, and serves millions of concurrent streams globally with low buffering.

Scalability: Upload, transcoding, and streaming are three completely separate scaling domains. Decouple them entirely via a message queue. Transcoding workers auto-scale based on queue depth.

Performance: Streaming latency is solved by CDN edge nodes — video chunks are served from a node close to the viewer, not from origin. Use adaptive bitrate (ABR) so the player switches quality dynamically based on bandwidth.

High Availability: CDN has built-in HA. Origin storage (S3) has 11 nines durability. The metadata service (video title, description, view count) runs in multi-AZ PostgreSQL. If the metadata service is slow, serve a degraded page — video still plays.

Resilience: Transcoding is the most failure-prone step. Each video upload triggers a job in Kafka. If a worker crashes mid-transcode, the job is retried from the queue. Workers are stateless — any worker can pick up any job.

Fault Tolerance: Chunked uploads (resumable via the `tus` protocol) — if the upload connection drops at 80%, the user resumes from chunk 80, not chunk 1. Video chunks stored with replication factor 3 in S3.

Extensibility: New output formats (AV1, HEVC) are added by deploying a new transcoding worker type that subscribes to the same Kafka topic. No change to upload or serving path.

Consistency: View counts use eventual consistency (Redis counter, flushed to DB every minute). Video availability (is the video ready to watch?) uses strong consistency — a video is only marked "published" after all resolutions are confirmed transcoded.

Security: Signed URLs for video chunk delivery (expire in 1 hour) — raw S3 URLs are never exposed. DRM for premium content. Upload scanning for CSAM and copyright violation before publishing.

Observability: Track transcoding queue depth, CDN cache hit ratio, stream start time (P99), and rebuffering ratio per region. Alert on queue depth > 1000 jobs or stream start time > 3s.

Disaster Recovery: Videos replicated to two S3 regions. Metadata DB uses Aurora Global with sub-second replication lag. RPO = 30 seconds, RTO = 2 minutes.

Cost Optimization: Store raw upload temporarily in a hot S3 bucket, move to S3 Intelligent-Tiering after transcoding, archive videos with under 100 views/month to Glacier. Use spot instances for transcoding workers (80% cost saving — jobs are retryable anyway).

Rate limiting: Upload API limited to 5 concurrent uploads per user. Transcode queue has a per-user job cap to prevent one user from starving the queue.

```
Tool: show_widget
```

Summary — Difficulty comparison

Problem	Core challenge	Key pattern
URL shortener	Read-heavy redirect at scale	Cache-aside with Redis, read replicas
Ride sharing	Real-time matching + state machine	Geo-index, WebSockets, circuit breaker
Video streaming	Async pipeline + global delivery	Queue-based transcoding, CDN, ABR

Each problem above tests a different cluster of the 20 system design pillars. In an interview, the URL shortener tests your caching and DB instincts; ride sharing tests real-time and consistency trade-offs; video streaming tests your async pipeline, storage tiering, and cost thinking.

5/5/2026, 1:36:02 PM

Do similar for a notification system, a banking system, and a facebook/instagram/Twitter design in that add (functional & non functiona requirement , add all important considerations, add flow details in written and a diagram)

5/5/2026, 1:37:30 PM

Here are all three designs in full detail.

---

1. Notification System

Functional Requirements

- Send notifications via multiple channels: push, email, SMS, in-app
- Support different notification types: transactional, marketing, alerts
- Users can set preferences (opt-out of marketing, choose channels)
- Delivery status tracking (sent, delivered, failed, read)
- Retry failed notifications automatically
- Support scheduled and real-time notifications
- Batch notifications (e.g. digest emails)

Non-Functional Requirements

- Deliver transactional notifications within 2 seconds
- Handle 1 million notifications per minute at peak
- 99.99% availability for transactional (OTP, payment alerts)
- At-least-once delivery guarantee — never silently drop
- Horizontally scalable per channel independently
- Notifications must not be sent to opted-out users

Flow Details

A service (e.g. payment service) calls the Notification API with an event. The API validates the request and publishes it to a Kafka topic. Channel workers (Push, Email, SMS, In-app) consume from their respective topics, check user preferences, and invoke the third-party provider (FCM for push, SendGrid for email, Twilio for SMS). The result (success/fail) is written back to the status DB. On failure, the message goes to a Dead Letter Queue and is retried up to 3 times with exponential backoff. A preference service is checked before every send — if the user has opted out, the message is acknowledged and discarded without sending.

Scalability: Each channel worker scales independently. Email volumes are high but latency-tolerant; push is lower volume but latency-sensitive. Kafka partitions allow parallel processing per channel.

Reliability & Fault Tolerance: Kafka guarantees at-least-once delivery. Workers are idempotent — processing the same notification twice produces only one send (deduplicated by `notification_id`).

Idempotency: Every notification has a unique `notification_id`. If a worker crashes after sending but before committing the Kafka offset, the retry is detected and skipped via Redis dedup check.

Rate Limiting: Per-user rate limits prevent notification spam (max 5 push/min per user). Per-provider rate limits respect SendGrid and Twilio API quotas.

High Availability: Kafka with replication factor 3. Multiple worker instances per channel. Preference service cached in Redis — if it's down, fail-safe is "send" (not "block").

Resilience: Circuit breaker around each third-party provider. If FCM is degraded, push workers stop calling FCM and return failure immediately rather than queuing up timeouts.

Observability: Track delivery rate per channel, latency P99, DLQ depth, retry count. Alert if DLQ depth grows unexpectedly.

Extensibility: New channels (WhatsApp, Slack) are added as new Kafka consumer groups + worker type. Zero changes to the API or upstream services.

Security: Notification content may contain PII — encrypt at rest. API authenticated via service-to-service JWT. Unsubscribe links use signed tokens.

```
Tool: show_widget
```

---

2. Banking System

Functional Requirements

- User registration, KYC verification, login with MFA
- Account management: create, view, close accounts
- Core transactions: deposit, withdrawal, transfer (internal + external)
- Transaction history with pagination and filters
- Balance inquiry (real-time)
- Fraud detection on every transaction
- Scheduled payments and standing orders
- Notifications for every transaction (via notification system)

Non-Functional Requirements

- Zero data loss — every transaction must be durable and auditable forever
- Exactly-once transaction processing — never debit or credit twice
- 99.999% availability (five nines — less than 5 minutes downtime/year)
- All money operations complete within 3 seconds
- Full audit trail for every change to every account — regulatorily required
- Compliance: PCI-DSS, SOC 2, ISO 27001
- End-to-end encryption for all financial data

Flow Details

A user initiates a transfer. The request hits the API Gateway (authenticated via JWT + MFA token). The Transfer Service begins a distributed transaction using the Saga pattern — it does not use a single DB transaction across services since those don't scale. Instead it: (1) debits the sender account, (2) credits the receiver account, (3) publishes a `transfer.completed` event to Kafka. Each step has a compensating transaction — if step 2 fails, step 1 is reversed (compensating saga). The Ledger Service writes immutable double-entry records (every debit has a matching credit). The Fraud Service runs in parallel — if it flags the transaction, a hold is placed and compliance is notified. Finally, the Notification Service is triggered asynchronously.

Consistency: Strong consistency within each account — no two transactions on the same account run concurrently. Use row-level locking in PostgreSQL per account. Distributed consistency across accounts uses the Saga pattern with compensating transactions.

Idempotency: Every transaction has a client-generated `idempotency_key`. The Transfer Service checks this key before processing — duplicate requests return the original result, never double-process.

Fault Tolerance: The Saga orchestrator persists its state to a DB. If the orchestrator crashes mid-saga, it resumes from the last completed step on restart.

High Availability: Active-Active across two data centers with synchronous replication (PostgreSQL Patroni + pglogical). RPO = 0 (no data loss). RTO = 30 seconds.

Security: All PII encrypted at rest (AES-256). Card data tokenized — raw card numbers never stored. All internal service calls use mutual TLS. Admin actions require 4-eyes approval (two people must authorize).

Audit & Compliance: Every state change to every record is written to an append-only audit log (immutable S3 + Athena for querying). Retained for 7 years minimum.

Observability: Real-time dashboards on transaction volume, fraud flag rate, saga failure rate, and API latency. Anomaly detection alerts on unusual transaction patterns.

Disaster Recovery: Cross-region active-passive. Automated DR drills monthly. RPO = 0, RTO = 30 seconds.

```
Tool: show_widget
```

3. Social Media Feed (Facebook / Instagram / Twitter)

Functional Requirements

- User registration, profile creation, follow/unfollow users
- Create posts: text, images, videos
- Home feed showing posts from followed users, ranked by relevance
- Like, comment, share on posts
- Search for users and posts
- Real-time notifications (likes, comments, new followers)
- Direct messaging between users
- Stories (ephemeral 24-hour content)
- Trending topics / hashtags

Non-Functional Requirements

- Support 500 million daily active users
- Feed load time under 300ms
- Post writes visible to followers within 5 seconds (eventual consistency is acceptable)
- 99.99% availability — social media downtime is very high visibility
- Media (images, video) served globally via CDN
- Scale read path independently from write path (reads outnumber writes 100:1)
- GDPR compliance — users can delete all their data

Flow Details

Write path: A user creates a post. It hits the Post Service which writes to the Posts DB (Cassandra — optimized for write-heavy, time-series data). A `post.created` event is

published to Kafka. The Fan-out Service consumes this event and decides whether to push the post into followers' feed caches (Redis sorted sets, ranked by time or relevance score). For celebrities with 10M+ followers, fan-out on write is too expensive — instead, their posts are fetched and merged at read time (fan-out on read / hybrid model).

Read path: A user opens their feed. The Feed Service reads their pre-computed feed from Redis (fast, O(1) lookup). For users following celebrities, it merges the cached feed with a real-time pull from the celebrity's post list. The merged list is ranked by a lightweight ML ranking model. Post details (author info, like counts) are hydrated from a cache layer before returning to the client.

Media path: Images and videos are uploaded to a media service, stored in S3, processed (resized, transcoded), and served via CDN. The post record only stores a reference (S3 key), never the raw bytes.

Scalability: Separate scaling for read (feed service, CDN), write (post service, fan-out workers), and media. Fan-out workers scale with Kafka consumer group scaling.

Performance: Feed served from Redis in under 5ms. CDN serves media with under 30ms TTFB globally. Database reads for post hydration use read replicas with a cache-aside pattern.

High Availability: Cassandra is natively multi-node with tunable consistency. Redis Cluster for feed cache with automatic resharding. No single point of failure in the critical read path.

Resilience: If the feed cache (Redis) is cold or unavailable, fall back to a slower DB query — degraded but functional. Ranking service failure falls back to chronological feed.

Consistency: Like counts use eventual consistency (Redis counter, flushed to Cassandra periodically). Whether a post exists uses strong consistency — a deleted post must disappear from all feeds within 60 seconds (GDPR requirement).

Extensibility: New feed ranking signals (engagement score, freshness, relationship strength) are added as new features in the ranking service — no schema change needed. New content types (Reels, Stories) are new post types in the Posts DB.

Security: Feed content personalized per user — never serve another user's private posts. Rate limit post creation (60 posts/hour per user) to prevent spam. Media scanning for CSAM on every upload before publishing.

Observability: Feed load latency (P50/P99), fan-out lag (time from post creation to feed visibility), cache hit ratio, CDN offload percentage. Dashboards per region.

Cost Optimization: CDN cache hit ratio target > 95% to minimize origin pulls. Fan-out skipped for inactive users (not logged in for 30 days) — their feeds are computed on next login.

```
Tool: show_widget
```

Comparison across all three

Consideration	Notification	Banking	Social Feed
Consistency	At-least-once delivery	Exactly-once, ACID	Eventual (feed), strong (delete)
Key pattern	DLQ + retry + dedup	Saga + compensating tx	Fan-out on write/read hybrid
Hardest problem	Deduplication on retry	Distributed transaction integrity	Feed ranking at 500M DAU scale
Storage	Kafka + PostgreSQL	PostgreSQL + Ledger + S3 audit	Cassandra + Redis + Graph DB
Real-time layer	None (async)	None (sync API)	WebSockets for notifications
Scale bottleneck	Third-party provider rate limits	DB row-level contention	Fan-out for celebrity accounts

Each of these systems builds on the same 20 pillars — what changes is which pillars dominate. Banking is dominated by consistency and security. Social feed is dominated by scalability and performance. Notifications are dominated by reliability, idempotency, and extensibility.

5/5/2026, 1:39:40 PM

one chat system design also do similar way

5/5/2026, 1:43:16 PM

Chat System Design (like WhatsApp / Slack / Messenger)

Functional Requirements

- User registration and profile management
- One-to-one (direct) messaging
- Group chats (up to 1000 members)
- Message delivery status: sent, delivered, read (double tick)
- Media sharing: images, videos, files, voice notes
- Online/offline presence indicators
- Message history with infinite scroll / pagination
- Push notifications when offline
- End-to-end encryption (E2EE) for direct messages
- Message reactions, replies, and delete/edit
- Search through message history

Non-Functional Requirements

- Support 2 billion users, 100 billion messages per day
- Message delivery latency under 100ms for online users
- 99.99% availability — messaging downtime is catastrophic
- Messages must never be lost — at-least-once delivery
- Message ordering preserved within a conversation
- Media files served globally via CDN under 50ms
- Horizontally scalable — no hard limits on concurrent connections

- GDPR compliant — full data deletion on request
- E2EE — server never sees plaintext message content

Flow Details

Message send (online → online): User A sends a message. The client maintains a persistent WebSocket connection to a Chat Server. The message is written to the Message Queue (Kafka) and simultaneously to the Message DB (Cassandra — optimized for time-series append-heavy writes). The Chat Server looks up which Chat Server User B is connected to (via a presence registry in Redis) and forwards the message directly over that server's WebSocket connection to User B. An ACK flows back: delivered → read receipts update the status DB.

Message send (online → offline): Same write path, but the presence registry shows User B is offline. Instead of WebSocket delivery, a `message.pending` event triggers the Push Notification Service (APNs / FCM) to wake User B's device. On reconnect, User B's client fetches undelivered messages from the Message DB using a cursor (last seen message ID).

Group message fan-out: User sends to a group of 500 members. Writing to each member's inbox directly (fan-out on write) is expensive at scale. Instead, a group message is written once to the Group Message DB. Each member's client polls or receives a lightweight notification event containing the group ID + message ID, then fetches the message content on demand (fan-out on read). For small groups under 50 members, fan-out on write into each member's inbox is used for lower latency.

Media upload: Client uploads directly to S3 via a pre-signed URL (never through the chat server). The media service transcodes/resizes and stores the processed file. The chat message contains only a media reference ID — never the raw bytes. CDN serves the media globally.

Key Considerations

Scalability: WebSocket connections are stateful — a single server can hold ~50,000 connections. At 2B users with 10% online = 200M concurrent connections, you need ~4,000 chat server instances. Use consistent hashing to assign users to servers — when a server dies, only its users reconnect (not everyone).

Performance: Message delivery under 100ms requires the entire path — client → chat server → Kafka → target chat server → client — to be in-memory as much as possible. Cassandra writes are async. Redis presence lookup is O(1). No DB reads on the hot send path.

High Availability: No single chat server is special — any server can handle any user. Kafka replication factor 3 ensures no message is lost if a broker dies. Redis Cluster for presence with automatic failover. Cassandra is natively distributed with no single point of failure.

Resilience: If a chat server crashes, clients reconnect to any healthy server via the load balancer. Undelivered messages are recovered from Cassandra on reconnect. Circuit breakers wrap the push notification providers — if APNs is slow, it doesn't block the write path.

Fault Tolerance: Kafka offset commits happen only after the message is durably written to Cassandra. If the consumer crashes between write and commit, the message is reprocessed (idempotent via `message_id` dedup).

Consistency: Message ordering within a conversation uses a monotonically increasing sequence number generated by a distributed ID service (Snowflake ID — timestamp + machine ID + sequence). Guarantees ordering without a central lock.

Idempotency: Every message has a client-generated `client_message_id`. If the network drops after send but before ACK, the client retries. The server deduplicates using `client_message_id` — the message is written exactly once.

Security & E2EE: For end-to-end encryption, keys are generated on the device and never leave it. The server stores only the public key. Messages are encrypted client-side with the recipient's public key — the server stores and forwards ciphertext it cannot read. Key exchange uses the Signal Protocol (Double Ratchet Algorithm).

Presence System: User presence (online/last seen) is stored in Redis with a TTL of 30 seconds. Clients send a heartbeat every 15 seconds to keep the presence alive. On heartbeat timeout, the user is marked offline. Presence updates are fanned out only to the user's active contacts — not broadcast globally.

Extensibility: New message types (polls, location share, payments) are new message type codes in the message schema. The routing and storage layer is type-agnostic. New notification channels are new consumers on the `message.pending` Kafka topic.

Observability: Track message delivery latency (P50/P99/P999), WebSocket connection count per server, Kafka consumer lag, push notification delivery rate, and failed delivery rate. Alert if P99 delivery latency exceeds 500ms or Kafka lag grows beyond 10 seconds.

Disaster Recovery: Cassandra replicated across 3 regions. Message DB RPO = 0 (synchronous local replication), RTO = 60 seconds (region failover). Media in S3 with cross-region replication.

Cost Optimization: Messages older than 90 days moved to cold Cassandra tier. Media not accessed in 30 days moved to S3 Intelligent-Tiering. Inactive users (offline > 7 days) excluded from real-time presence fan-out.

Here is the overall architecture — showing both the write path (send) and the read path (reconnect + history fetch) side by side:

```
Tool: show_widget
```

Key Design Decisions Summary

Decision	Choice	Why
Transport	WebSocket (persistent)	Low latency bidirectional — HTTP polling can't do 100ms
Message storage	Cassandra	Time-series, append-heavy, massive write throughput
Presence	Redis TTL + heartbeat	Sub-ms lookup, auto-expires without explicit logout
Message ordering	Snowflake ID	Distributed, time-sortable, no central lock

Decision	Choice	Why
Group fan-out	Hybrid (write < 50, read > 50)	Fan-out on write for small groups is fast; for large groups it's too expensive
E2EE	Signal Protocol (Double Ratchet)	Industry standard, perfect forward secrecy
Media routing	Pre-signed S3 URL	Chat server never handles bytes — scales independently
Offline delivery	FCM / APNs push	OS-level wake — battery efficient, reliable
Deduplication	`client_message_id` in Redis	Retries are safe — exactly-once visible to user
Server routing	Consistent hashing	Reconnects affect only users on the failed server

How all pillars apply here

Scalability — Chat servers scale horizontally via consistent hashing. Kafka partitions scale with message volume. Cassandra scales by adding nodes.

Availability — No single point of failure. Load balancer health-checks chat servers. Kafka broker loss is transparent (replication factor 3).

High Availability — Active-Active WebSocket servers. Redis Cluster auto-reshards. Cassandra multi-region.

Reliability — Kafka guarantees at-least-once. Cassandra durable writes with replication. Undelivered messages recovered from DB on reconnect.

Resilience — Circuit breaker around push providers. Server crash = reconnect only (no message loss). DLQ for failed push notifications.

Fault Tolerance — Heartbeat-based presence means silent disconnects are detected within 30 seconds. Snowflake IDs keep ordering intact even if one ID service node fails.

Performance — Hot send path is entirely in-memory (Redis + Kafka). Cassandra reads use partition key = `conversation_id` for O(1) lookups. CDN serves all media.

Consistency — Within a conversation, Snowflake IDs enforce strict ordering. Delivery receipts use eventual consistency — a 200ms delay on "delivered" tick is acceptable.

Extensibility — New message types are new type codes. New notification channels are new Kafka consumers. Reactions and replies are message references, not schema changes.

Security — E2EE via Signal Protocol. TLS everywhere. Rate limit sends (1000 messages/hour per user). Media scanning before delivery.

Observability — WS connection count, message delivery P99, Kafka consumer lag, push delivery rate, presence registry size. PagerDuty alert if P99 > 500ms.

Idempotency — `client_message_id` makes all retries safe. Exactly-once visible delivery even under network failures.

Disaster Recovery — Cassandra cross-region replication. RPO = 0 locally, 5 seconds cross-region. RTO = 60 seconds.

5/5/2026, 1:44:38 PM

Swiggy mart design (add functional, non functional, API design, HLD,Flow details, Core component detail exaplanation, trade offs)

5/5/2026, 4:01:58 PM

Swiggy Instamart System Design

(10-minute grocery delivery platform)

Functional Requirements

- Browse stores and product catalogue by location
- Search products with filters (category, brand, price, dietary)
- Real-time inventory — show only available items
- Add to cart, apply coupons/offers
- Place order with delivery address
- Real-time order tracking (picker → packed → out for delivery → delivered)
- Multiple payment methods: UPI, card, COD, wallet
- Delivery partner assignment and live location tracking
- Ratings and reviews for orders
- Order history and reorder
- Surge pricing during peak hours
- Dark store (warehouse) management — stock replenishment alerts
- Substitution suggestions when item is out of stock

Non-Functional Requirements

- 10-minute delivery SLA — every component must optimize for speed

- Support 5 million daily orders across 500+ dark stores
- Inventory read latency under 20ms — stale inventory = bad UX
- Order placement to picker assignment under 5 seconds
- 99.99% availability — downtime = lost orders = lost revenue
- Eventual consistency acceptable for catalogue; strong consistency mandatory for inventory and payments
- Horizontal scalability for flash sales and festival spikes (10x normal load)
- GDPR + PCI-DSS compliance
- Mobile-first — API responses under 200ms on 4G

API Design

Catalogue & Search

```
GET  /v1/stores?lat={lat}&lng={lng}&radius={km}
     → Returns nearby dark stores with ETA

GET  /v1/stores/{store_id}/products?category={}&search={}&page={}
     → Paginated product list with real-time stock flag

GET  /v1/products/{product_id}?store_id={}
     → Product detail with live inventory count
```

Cart

```
POST   /v1/cart                        → Create cart (tied to user + store)
PUT    /v1/cart/{cart_id}/items        → Add / update item quantity
DELETE /v1/cart/{cart_id}/items/{id}   → Remove item
GET    /v1/cart/{cart_id}/summary      → Totals, offers, delivery fee
```

```
POST   /v1/cart/{cart_id}/validate     → Re-check stock before
checkout
```

Orders

```
POST  /v1/orders                       → Place order (idempotency-
key header required)
GET   /v1/orders/{order_id}            → Order detail + current
status
GET   /v1/orders/{order_id}/track      → Live picker + delivery
partner location
GET   /v1/users/{user_id}/orders?page= → Order history
POST  /v1/orders/{order_id}/cancel     → Cancel (only before picking
starts)
```

Inventory (internal)

```
PUT   /internal/v1/inventory/{sku_id}  → Update stock (picker marks
item picked)
POST  /internal/v1/inventory/bulk      → Bulk stock replenishment
GET   /internal/v1/inventory/low-stock → Items below reorder
threshold
```

Delivery

```
GET   /v1/delivery/estimate?store_id={}&lat={}&lng={}  → ETA before
order
POST  /internal/v1/delivery/assign     → Assign delivery partner to
order
PUT   /internal/v1/delivery/{id}/location → Partner location update
(every 3s)
```

High Level Design (HLD)

The system splits cleanly into five domains: **Catalogue**, **Inventory**, **Order**, **Delivery**, and **Payment**. Each is an independent service with its own data store, communicating asynchronously via Kafka except where strong consistency is required (inventory reservation, payment).

The critical path for a 10-minute delivery is:

Order placed → inventory reserved → picker assigned (< 5s) → items picked (3–4 min) → packed → delivery partner assigned → delivered (6–7 min)

Everything outside this critical path (notifications, analytics, reviews, reorder suggestions) is handled asynchronously.

```
Tool: show_widget
```

Flow Details

Flow 1 — User browses and adds to cart

User opens app → GPS coordinates sent to API Gateway → **Store Service** queries Redis geo-index (`GEORADIUS`) to find dark stores within 2–3km → returns store list with pre-computed ETA. User selects a store → **Catalogue Service** fetches product list from Elasticsearch (fast full-text + filter search). Each product card shows live stock status — fetched from Redis inventory cache (TTL 30 seconds). User adds item → **Cart Service** writes cart to Redis (session store). On checkout tap, Cart Service calls Inventory Service to do a soft reservation — atomically decrement the Redis counter using `DECR`. If counter hits zero, item marked unavailable in real time.

Flow 2 — Order placement (critical path)

User hits "Place Order" → API Gateway validates JWT → **Order Service** receives request with `Idempotency-Key` header. Order Service checks Redis for duplicate key (prevents double-order on retry). If new: (1) calls Payment Service synchronously — payment must succeed before order is confirmed. (2) On payment success, writes order record to

PostgreSQL with status `CONFIRMED`. (3) Publishes `order.placed` event to Kafka. **Inventory Service** consumes the event and converts soft reservation to hard reservation. **Dark Store App** (picker tablet) consumes `order.placed` and shows the pick list to the nearest available picker within the store. Entire flow: under 5 seconds.

Flow 3 — Picking and packing

Picker receives task on tablet. For each item, picker scans the barcode → **Inventory Service** marks item as picked → updates order item status. If an item is out of stock at pick time (edge case — soft reservation was stale), the picker marks it as unavailable → **Substitution Service** suggests the nearest equivalent product → customer is notified via push to approve or skip. Once all items are picked, order moves to `PACKED` status. Kafka event `order.packed` triggers **Delivery Service**.

Flow 4 — Delivery partner assignment

`order.packed` consumed by **Delivery Service** → queries the **Location Service** (TimescaleDB) for delivery partners within 500m of the dark store who are idle. Assignment uses a scoring algorithm: distance + current load + historical delivery time. Assigned partner receives push notification on their app. Partner location is streamed to Location Service every 3 seconds via WebSocket. Customer's app subscribes to a WebSocket channel keyed by `order_id` and receives live location updates. Order moves through `OUT_FOR_DELIVERY` → `DELIVERED`.

Flow 5 — Inventory replenishment

A background **Replenishment Service** runs every 5 minutes. It reads current stock levels from the Inventory DB and compares against reorder thresholds (configured per SKU per store). If stock falls below threshold, it raises a replenishment alert to the store manager's dashboard and logs to the warehouse management system (WMS) for restocking. High-velocity SKUs (top 100 products) have a tighter threshold and trigger replenishment faster.

Core Component Deep Dive

Inventory Service — The most critical component

Inventory is the heart of Instamart. A wrong inventory count means either overselling (order placed for an out-of-stock item) or underselling (item shown as unavailable when it exists). Two-layer design:

Layer 1 — Redis (hot cache): Every SKU per store has a counter in Redis (`inventory:{store_id}:{sku_id}`). Reads are O(1), writes use atomic `DECR`/`INCR`. This is the source of truth for the live "Add to Cart" experience. TTL not set — inventory is managed explicitly (not expired).

Layer 2 — PostgreSQL (durable store): Ground truth for stock. Redis is populated from PostgreSQL on store open and after every replenishment. If Redis crashes, it's repopulated from PostgreSQL on restart. Write-through strategy — every inventory change writes to both Redis and PostgreSQL.

Reservation lifecycle: Soft reserve (cart add) → Hard reserve (order placed) → Deducted (picked) → Released (cancelled/substituted). Each state transition is an event on Kafka for full auditability.

Order Service — State machine

Orders follow a strict state machine: `PENDING → CONFIRMED → PICKING → PACKED → OUT_FOR_DELIVERY → DELIVERED`. Invalid transitions are rejected. Each transition is a Kafka event. The state is stored in PostgreSQL (strong consistency). Cancellation is only allowed in `PENDING` or `CONFIRMED` — once picking starts, cancellation triggers a compensation flow (refund + inventory release).

Delivery Assignment — The matching problem

This is essentially a real-time bipartite matching problem: orders on one side, delivery partners on the other. The scoring function weighs: straight-line distance from dark store (40%), partner's current delivery load (30%), historical on-time rate (20%), vehicle type (10%). The assignment runs in under 200ms — it queries at most the nearest 10 partners and scores them in memory. No ML model on the hot path — scoring is deterministic and fast.

Location Service — High write throughput

Every delivery partner sends a location ping every 3 seconds. At 10,000 active deliveries, that is ~3,300 writes/second. TimescaleDB (PostgreSQL extension for time-series) handles this with hypertable partitioning by time. Only the last known location is needed for

assignment — stored in Redis for O(1) lookup. Historical location trail is in TimescaleDB for ETA calculation and dispute resolution.

Search — Elasticsearch with inventory awareness

Product catalogue is indexed in Elasticsearch. But showing out-of-stock products is a bad UX. The challenge: inventory changes every second but Elasticsearch reindexing is slow. Solution: Elasticsearch stores the product data (name, category, brand, images). Stock status is fetched from Redis and merged in the application layer after the search query returns. Search results always reflect live inventory without expensive re-indexing.

Dark Store App — Picker workflow

The picker app is a thin client (tablet/mobile) connected to the Order Service via WebSocket. It receives the pick list ordered by aisle location (optimized picking path to reduce walk time). As items are picked, the app updates inventory in real time. The app works in degraded offline mode — if WiFi drops in the warehouse, picks are queued locally and synced on reconnect.

Trade-offs

1. Redis inventory vs. DB inventory as source of truth

Chosen: Redis as primary, PostgreSQL as backup. **Why:** 20ms read requirement cannot be met by PostgreSQL under load. Redis atomic `DECR` prevents race conditions without locking. **Trade-off:** Redis failure means inventory is briefly unavailable. Mitigation: Redis Cluster with AOF persistence + fallback to PostgreSQL read replicas.

2. Soft reservation vs. no reservation in cart

Chosen: Soft reservation on "Add to Cart" with 15-minute TTL. **Why:** Without reservation, two users can add the last item, but only one order succeeds — bad UX (order confirmation then "sorry, out of stock"). **Trade-off:** Items held in abandoned carts reduce availability for other users. Mitigation: Short 15-minute TTL releases held inventory quickly. Show "X people have this in cart" warning.

3. Synchronous payment vs. async payment

Chosen: Synchronous payment on order placement. **Why:** Money must be confirmed before inventory is hard-reserved and picking starts. Async payment means you could start picking and then the payment fails. **Trade-off:** Payment gateway latency (300–800ms) is on the critical path. Mitigation: Timeout of 10 seconds with automatic retry. COD orders skip payment entirely.

4. Fan-out on write vs. read for notifications

Chosen: Kafka fan-out — every order event publishes to a single topic, all consumers (notification, analytics, dark store, reorder) subscribe independently. **Why:** Decouples producers from consumers. Adding a new consumer (e.g. loyalty points service) requires zero changes to Order Service. **Trade-off:** Kafka adds ~50–100ms of latency to non-critical downstream consumers. Acceptable since notifications and analytics are not on the delivery critical path.

5. Single dark store per order vs. multi-store fulfillment

Chosen: Single dark store per order. **Why:** Multi-store fulfillment requires coordinating two delivery partners and two pick times — impossible to guarantee 10-minute SLA. **Trade-off:** If a nearby dark store lacks an item, the customer sees it as unavailable even though a store 4km away has stock. Mitigation: Show "available at a farther store" with a longer ETA option.

6. Elasticsearch for search vs. PostgreSQL full-text search

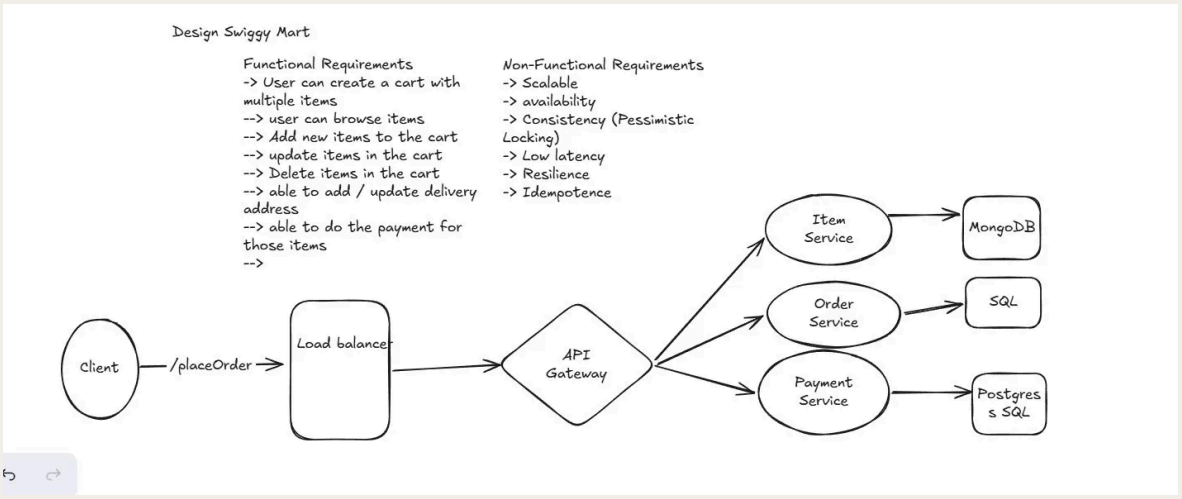
Chosen: Elasticsearch. **Why:** PostgreSQL full-text search degrades significantly at millions of products with complex filters (brand + category + dietary + price range). Elasticsearch handles this with inverted indices natively. **Trade-off:** Eventual consistency between PostgreSQL (catalogue master) and Elasticsearch (search index). A new product may take 2–5 seconds to appear in search after creation. Acceptable for a grocery catalogue.

Summary Table

Component	Technology	Why
Inventory hot store	Redis (atomic DECR)	Sub-20ms reads, race-condition-safe writes

Component	Technology	Why
Inventory durable store	PostgreSQL	ACID, write-through backup
Order state	PostgreSQL	Strong consistency, state machine
Order history	Cassandra	Time-series, massive read scale
Product search	Elasticsearch	Full-text + filter at scale
Location streaming	TimescaleDB + Redis	High write throughput + O(1) last-known
Event bus	Kafka	Decoupled fan-out, at-least-once
Product images	S3 + CDN	Global low-latency media delivery
Sessions / cart	Redis	Ephemeral, fast, TTL support
Delivery partner location	WebSocket + TimescaleDB	Real-time bidirectional, history for ETA

5/5/2026, 4:03:42 PM



is my design correct ? if yes how much % , if not what and all need to improve or take care

5/5/2026, 6:24:31 PM

Great effort for a first draft! Your design shows solid foundational thinking. Let me give you an honest review.

Overall Score: 35–40%

You have the skeleton right but most of the critical depth is missing. Here is a breakdown of what is correct, what is incomplete, and what is wrong.

✅ What You Got Right

Functional Requirements — well captured. Cart CRUD, browse, address, payment — these are the core flows. Good start.

Non-Functional Requirements — you listed the right keywords: scalability, availability, consistency, low latency, resilience, idempotency. Mentioning **pessimistic locking**

specifically for consistency shows good instinct — that is exactly what inventory needs.

Load Balancer → API Gateway → Services — the routing layer is correct. This is the right pattern.

Service separation — splitting Item, Order, Payment into separate services is the right microservices thinking.

Database per service — giving each service its own DB is correct (database-per-service pattern).

❌ What Is Incomplete or Wrong

1. Missing the most critical service — Inventory Service

This is the biggest gap. Swiggy Instamart's hardest problem is **real-time inventory** — knowing exactly how many units are available and preventing overselling. You have an Item Service (catalogue) but no Inventory Service. These are two completely different things:

- **Item Service** = product name, image, price, description (rarely changes)
- **Inventory Service** = how many units are in stock right now (changes every second)

Without this separation, your design cannot handle the core problem.

2. Cart Service is missing

You listed "create cart / add items / update / delete" in functional requirements but there is no Cart Service in the diagram. Cart needs its own service backed by Redis (not a SQL DB) because carts are temporary, session-scoped, and need sub-millisecond reads.

3. No Delivery Service

10-minute delivery is Instamart's core promise. Where is the delivery partner assignment, live tracking, and ETA calculation? This is a major missing service.

4. MongoDB for Item Service — questionable choice

MongoDB is fine but you need to justify it. Product catalogue has relationships (category → subcategory → product, product → brand). A relational DB or Elasticsearch (for search) would be stronger here. MongoDB works but needs reasoning.

5. No async communication — no Kafka / message queue

Every service in your diagram is synchronously connected. In reality: once an order is placed, you need to trigger notifications, update inventory, assign a picker, and log analytics — all independently and without blocking the user response. Without a message queue, one slow downstream service blocks the entire order placement.

6. No caching layer

You mentioned low latency as an NFR but there is no Redis cache shown anywhere. Product browsing and inventory checks happen thousands of times per second — hitting MongoDB/SQL directly every time will be too slow and expensive.

7. No Kafka / event bus shown

The `order.placed` event needs to fan out to: inventory hard-reservation, picker assignment, notification service, analytics. Without an event bus this is not possible without tight coupling.

8. "SQL" label on Order Service DB is too vague

Which SQL? PostgreSQL? MySQL? In interviews you need to be specific and justify the choice. PostgreSQL is preferred for orders because of JSONB support, better locking, and Aurora compatibility.

9. No mention of idempotency implementation

You listed idempotency as an NFR — good. But how? The diagram shows no idempotency key handling, no dedup store, no retry logic. NFRs must have corresponding design elements.

10. Only `/placeOrder` endpoint shown

The diagram shows a single API call. What about `/browse`, `/cart`, `/track`, `/cancel`? A real design needs at least the key APIs outlined.

What Your Diagram Should Look Like

Here is the corrected and complete version of your diagram:

```
Tool: show_widget
```

Score Breakdown

Area	Your Score	Max	Comment
Functional Requirements	7/10	10	Good coverage, missed order tracking and substitution
Non-Functional Requirements	6/10	10	Right keywords but no implementation detail
Service decomposition	3/10	10	Missing Cart, Inventory, Delivery — these are core
Database choices	3/10	10	Too vague, MongoDB questionable, no Redis anywhere
Async / event-driven design	0/10	10	No Kafka or queue of any kind
Caching strategy	0/10	10	Not mentioned at all
Idempotency implementation	1/10	10	Named it but not designed it
API design	1/10	10	Only `/placeOrder` shown
Total	**~21/80**	**80**	**≈ 35–40%**

Priority Fix List (in order)

Fix 1 — Add Inventory Service with Redis atomic DECR for soft reservation and PostgreSQL as durable backup. This is the most important missing piece.

Fix 2 — Add Cart Service backed by Redis with a 15-minute TTL on soft reservations.

Fix 3 — Add Kafka between Order Service and downstream consumers (Notification, Delivery, Analytics, Dark Store picker).

Fix 4 — Add Delivery Service — partner assignment, live location via WebSocket, ETA calculation.

Fix 5 — Replace "SQL" label on Order DB with PostgreSQL and explain why (ACID, row-level locking for pessimistic locking you mentioned).

Fix 6 — Replace MongoDB for Item Service with Elasticsearch + Redis cache and justify it.

Fix 7 — Show idempotency key as a Redis dedup check in the Order Service flow.

Fix 8 — Add at least 5 API endpoints — `/browse`, `/cart`, `/placeOrder`, `/track`, `/cancel`.

Your instincts are right — the topology (client → LB → Gateway → services → DBs) is correct. What is missing is the **depth** within that structure: the right services, the right databases, the async layer, and the implementation of the NFRs you listed. Keep iterating — you are on the right track.

5/5/2026, 6:25:53 PM

Design LinkedIn in same way similar fashion with all details

5/7/2026, 6:16:37 AM

LinkedIn System Design

Functional Requirements

- User registration, profile creation (experience, education, skills, photo)
- Connection requests — 1st, 2nd, 3rd degree network
- Follow users and companies without connecting
- Feed — posts, articles, reposts, polls from network
- Like, comment, share, save posts
- Direct messaging (1-to-1 and group)
- Job postings — companies post, users apply
- Job recommendations based on profile + activity
- People you may know (PYMK) recommendations
- Notifications — connection requests, likes, comments, job alerts
- Search — users, companies, jobs, posts
- LinkedIn Premium — profile views, InMail, who viewed your profile
- Company pages — posts, followers, job listings
- Endorsements and skill recommendations
- Real-time presence (active now indicator)

Non-Functional Requirements

- 1 billion+ registered users, 300 million MAU

- Feed load under 300ms
- Search results under 200ms
- Message delivery under 100ms for online users
- 99.99% availability — professional network downtime is high visibility
- Post visible to connections within 5 seconds of publishing (eventual consistency)
- Strong consistency for payments (Premium subscriptions, job boosts)
- Horizontal scalability — connection graph can reach trillions of edges
- GDPR compliant — full data export and deletion
- Mobile-first API responses under 200ms on 4G
- Read-heavy system — reads outnumber writes 50:1

API Design

Profile & Connections

```
POST   /v1/users                          → Register user
GET    /v1/users/{user_id}                → Public profile
PUT    /v1/users/{user_id}/profile        → Update experience,
skills, bio
POST   /v1/connections/request            → Send connection request
PUT    /v1/connections/{id}/respond       → Accept / decline
GET    /v1/users/{user_id}/connections    → Connection list
(paginated)
GET    /v1/users/{user_id}/pymk           → People You May Know
```

Feed

```
POST   /v1/posts                          → Create post / article /
poll
GET    /v1/feed?user_id={}&cursor={}      → Paginated ranked feed
POST   /v1/posts/{post_id}/react          → Like / celebrate /
```

```
support
POST   /v1/posts/{post_id}/comments       → Add comment
POST   /v1/posts/{post_id}/share          → Repost or share with note
GET    /v1/posts/{post_id}/comments       → Paginated comments
```

Messaging

```
POST   /v1/conversations                  → Create conversation
POST   /v1/conversations/{id}/messages    → Send message
GET    /v1/conversations/{id}/messages    → Message history (cursor
paginated)
GET    /v1/conversations                  → Inbox list
PUT    /v1/conversations/{id}/read        → Mark as read
```

Jobs

```
POST   /v1/jobs                           → Post a job (company)
GET    /v1/jobs?q={}&location={}&page={}  → Search jobs
GET    /v1/jobs/recommended?user_id={}    → ML-ranked job
recommendations
POST   /v1/jobs/{job_id}/apply            → Apply with profile
GET    /v1/jobs/{job_id}/applicants       → Applicants list
(recruiter only)
```

Search

```
GET    /v1/search?q={}&type={users|jobs|posts|companies}&filters={}
GET    /v1/search/typeahead?q={}          → Autocomplete suggestions
```

Notifications

```
GET    /v1/notifications?user_id={}       → Notification list
PUT    /v1/notifications/{id}/read        → Mark read
```

```
PUT   /v1/notifications/read-all          → Mark all read
```

Flow Details

Flow 1 — Profile view and PYMK

User opens LinkedIn → app fetches profile from **Profile Service** (data from PostgreSQL, cached in Redis for 5 minutes). The **Graph Service** reads the connection graph from **Neptune** (managed graph DB) to determine 1st, 2nd, 3rd degree connections. **PYMK Service** runs a graph traversal — "friends of friends not yet connected" — weighted by mutual connections, same company, same college, same location. Results pre-computed by a background Spark job every hour and stored in Redis per user for instant retrieval.

Flow 2 — Post creation and feed fan-out

User creates a post → **Post Service** writes to Cassandra (post content) and publishes `post.created` to Kafka. **Fan-out Service** consumes the event and looks up the author's followers/connections from the Graph Service. For users with under 10,000 connections — **fan-out on write**: the post ID is pushed into each follower's feed cache (Redis sorted set, ranked by score). For influencers with 500,000+ connections — **fan-out on read**: the post is fetched and merged at feed read time. Feed ranking is computed by a lightweight **Ranking Service** using signals: recency, engagement rate, relationship strength, content type preference.

Flow 3 — Job application flow

Recruiter posts a job → **Job Service** writes to PostgreSQL and indexes in Elasticsearch. A background **Recommendation Engine** (Spark ML) runs nightly — scores every active user against every active job using profile embeddings, skills match, location, experience level, and past application behaviour. Top-N recommendations per user stored in Redis. When user opens Jobs tab, recommendations served from Redis in under 10ms. User applies → **Application Service** writes to PostgreSQL, notifies the recruiter via Kafka → **Notification Service**.

Flow 4 — Messaging

Same architecture as the chat system design — WebSocket connections to **Chat Servers** via consistent hashing. Messages written to Kafka then durably to Cassandra. Presence tracked in Redis with 30-second TTL heartbeat. Offline users notified via FCM/APNs. InMail (Premium feature) allows messaging outside connections — rate limited to 30 InMails/month, tracked in PostgreSQL.

Flow 5 — Search

User types in search box → **Typeahead Service** queries a prefix trie (Redis) for autocomplete in under 20ms. Full search hits **Elasticsearch** with a compound query across users, jobs, posts, companies. Results are re-ranked by a **Search Ranking Service** using the user's network proximity (Graph Service), recency, and engagement signals. Search index updated in near-real-time via Kafka CDC (Change Data Capture) from source DBs.

Flow 6 — Notifications

Every system event (like, comment, connection accept, job alert) publishes to Kafka. **Notification Service** consumes events, checks user preferences (do not disturb, email digest frequency), deduplicates (don't send 50 "liked your post" notifications — batch into "50 people liked your post"), and routes to: WebSocket (in-app, real-time), FCM/APNs (push), or email (SendGrid, for digest).

High Level Design

```
Tool: visualize:show_widget
```

```
View: Check available skills for frontend design
```

Now let me create the full LinkedIn design with an artifact diagram.

```
Create File: LinkedIn system design HLD diagram as SVG-based HTML artifact
```

```
Present File: /mnt/user-data/outputs/linkedin_system_design.html
```

Here is the complete LinkedIn system design. Here is a summary of everything covered:

What's inside

Requirements — 14 functional features (profile, connections, feed, jobs, messaging, search, Premium, PYMK) and 12 non-functional constraints covering scale, latency, consistency, and compliance.

API Design — 20+ endpoints across Profile, Feed, Jobs, Messaging, and Search with correct HTTP methods and pagination strategy.

HLD Diagram — Full 6-layer architecture showing Client → CDN → API Gateway → Core Services → Supporting Services → Kafka → Data Stores → Batch layer.

5 Flow Details — Step-by-step walkthroughs for PYMK, feed fan-out, job application, messaging, and search.

6 Core Component Deep Dives — Graph Service, Fan-out hybrid, Feed Ranking, Job Recommendation Engine, Elasticsearch search, and Notification batching.

6 Trade-offs — Neptune vs SQL, fan-out strategies, Cassandra vs PostgreSQL, nightly batch vs real-time ML, eventual vs strong consistency, Redis memory cost.

Technology Summary Table — Every component with technology choice and justification.

The hardest unique problems in LinkedIn vs other systems are the **graph at trillion-edge scale** (Neptune + Pregel), **feed fan-out for influencers** (hybrid write/read threshold), and **BERT-based job matching** pre-computed nightly — all three are covered in depth.

> *File: /mnt/user-data/outputs/linkedin_system_design.html*

5/7/2026, 6:20:41 AM

You have not mentioned scalability, availability and all Don’t create interactive design just write it here

5/7/2026, 7:07:36 AM

You're right! Let me cover all the pillars in full detail here.

LinkedIn System Design — Complete with All Pillars

Functional Requirements

- User registration, profile (experience, education, skills, photo)
- Connection requests — 1st, 2nd, 3rd degree network
- Follow users and company pages without connecting
- Feed — posts, articles, polls, reposts ranked by relevance
- Like, comment, share, save posts
- Direct messaging — 1-to-1, group chat, InMail (Premium)
- Job postings — companies post, users apply, recruiter dashboard
- Job and People recommendations (ML-based)
- People You May Know (PYMK)
- Notifications — likes, comments, connections, job alerts
- Full-text search — users, jobs, posts, companies
- LinkedIn Premium — who viewed profile, InMail credits
- Endorsements and skill recommendations
- Real-time presence (active now indicator)
- Company pages — followers, posts, job listings

Non-Functional Requirements

- 1 billion+ registered users, 300 million MAU
- Feed load under 300ms, Search results under 200ms
- Message delivery under 100ms for online users
- 99.99% availability — less than 52 minutes downtime per year
- Post visible to all connections within 5 seconds of publishing
- Strong consistency for payments, connections, job applications
- Eventual consistency acceptable for feed, like counts, view counts
- Read-heavy system — reads outnumber writes 50:1
- Horizontal scalability — connection graph has trillions of edges
- GDPR compliant — full data export and deletion on request
- Mobile-first — API responses under 200ms on 4G

API Design

Profile & Connections

```
POST   /v1/users                          → Register user
GET    /v1/users/{user_id}                → Public profile (cached)
PUT    /v1/users/{user_id}/profile        → Update experience,
skills, bio
POST   /v1/connections/request            → Send connection request
PUT    /v1/connections/{id}/respond       → Accept / decline
GET    /v1/users/{user_id}/connections    → Paginated connection list
GET    /v1/users/{user_id}/pymk           → People You May Know
```

Feed & Posts

```
GET   /v1/feed?user_id={}&cursor={}      → Ranked paginated feed
POST  /v1/posts                          → Create post / article /
poll
POST  /v1/posts/{post_id}/react          → Like / celebrate /
support
POST  /v1/posts/{post_id}/comments       → Add comment
POST  /v1/posts/{post_id}/share          → Repost or share with note
GET   /v1/posts/{post_id}/comments       → Paginated comments
```

Jobs

```
POST  /v1/jobs                           → Post a job
(company/recruiter)
GET   /v1/jobs?q={}&location={}&page={}  → Search jobs with filters
GET   /v1/jobs/recommended               → ML-ranked recommendations
POST  /v1/jobs/{job_id}/apply            → Apply with profile
GET   /v1/jobs/{job_id}/applicants       → Applicant list (recruiter
only)
```

Messaging

```
POST  /v1/conversations                  → Create conversation
POST  /v1/conversations/{id}/messages    → Send message
(idempotency-key required)
GET   /v1/conversations/{id}/messages    → Message history (cursor
paginated)
GET   /v1/conversations                  → Inbox list
PUT   /v1/conversations/{id}/read        → Mark as read
```

Search & Notifications

```
GET   /v1/search?q={}&type={users|jobs|posts|companies}&filters={}
GET   /v1/search/typeahead?q={}          → Autocomplete under 20ms
```

```
GET   /v1/notifications                  → Notification list
PUT   /v1/notifications/{id}/read        → Mark read
```

High Level Design (HLD)

The system is divided into six independent domains each with its own service and data store: **Profile, Feed, Messaging, Jobs, Search, and Notifications**. They communicate asynchronously via Kafka except where strong consistency is required (connections, payments). A Graph Service backed by Neptune sits across multiple domains since connections, PYMK, feed fan-out, and search ranking all depend on the relationship graph.

Critical read path: User opens app → API Gateway → Feed Service → Redis feed cache (pre-built by fan-out service) → post hydration from Redis/Cassandra → ranking score applied → response in under 300ms.

Critical write path: User creates post → Post Service → Cassandra write → Kafka `post.created` → Fan-out Service → push post ID into follower feed caches (Redis sorted sets) → done within 5 seconds.

Scalability

LinkedIn is fundamentally a read-heavy system (50:1 read-to-write ratio). Every layer is designed to scale reads independently from writes.

API layer: Stateless services deployed on Kubernetes. Horizontal pod autoscaling triggers at 70% CPU. Each service scales independently — the Feed Service needs more replicas than the Payment Service. API Gateway (Kong or AWS API Gateway) handles routing, rate limiting, and auth token validation without hitting a DB.

Feed scalability: The fan-out service uses Kafka consumer groups — adding more consumers increases fan-out throughput linearly. At 300M MAU posting 5 posts/day average = 1.5 billion fan-out operations/day = ~17,000 fan-out writes/second. Redis sorted sets handle this volume comfortably with a cluster of 20–30 nodes.

Graph scalability: Neptune scales read replicas horizontally. Hot graph queries (PYMK, 2nd-degree connections) are pre-computed by a weekly Spark GraphX job and results cached in Redis — Neptune never handles live PYMK queries.

Database scalability: Cassandra scales by adding nodes with zero downtime — consistent hashing distributes data automatically. PostgreSQL uses read replicas (Aurora with up to 15 read replicas) for profile and job reads. Elasticsearch scales by adding shards and nodes horizontally.

Search scalability: Elasticsearch index split into 4 separate indices (users, jobs, posts, companies). Each independently sharded and scaled. Job index gets the most writes (new postings); user index gets the most reads (search by name).

Media scalability: Images and videos stored in S3 — infinitely scalable. CloudFront CDN serves media globally from 400+ edge locations. Profile photos cached at CDN for 24 hours (cache-control headers). Video is transcoded into multiple resolutions by a media processing pipeline before storage.

Availability

Target: 99.99% — less than 52 minutes of downtime per year.

Multi-AZ deployment: All services run across at least 3 Availability Zones. If one AZ goes down, traffic is automatically rerouted to healthy AZs via the load balancer. Database primary is in one AZ with synchronous replicas in two others.

No single point of failure: Load balancer itself is redundant (AWS ALB with built-in HA). Kafka has 3 brokers minimum per topic with replication factor 3. Redis Cluster has at least 3 master nodes each with 1 replica — if a master fails, its replica promotes automatically within seconds. Neptune has a reader endpoint that automatically routes to healthy read replicas.

Health checks: Every service exposes a `/health` endpoint. Load balancer health checks every 10 seconds — unhealthy instances are removed from rotation within 30 seconds. Kubernetes liveness and readiness probes restart crashed pods automatically.

Graceful degradation: If the Ranking Service is down, feed falls back to chronological order — users still see their feed, just unranked. If Recommendation Service is down, job recommendations fall back to "popular jobs in your area". Core functionality (feed, messaging, jobs) never depends on auxiliary services (recommendations, analytics).

Database availability: Aurora PostgreSQL Multi-AZ with automatic failover in under 30 seconds. Cassandra is natively always-on — a node failure just reduces replication factor temporarily. Neptune Global Database replicates to a secondary region with under 1 second lag.

High Availability (HA)

HA goes beyond availability — it means eliminating every single point of failure and designing for zero-downtime operations.

Active-Active across regions: LinkedIn runs in multiple AWS regions (US-East, EU-West, AP-Southeast). DNS-based global load balancing (Route 53 latency routing) sends users to the nearest region. Both regions actively serve traffic — no idle standby.

Zero-downtime deployments: All deployments use Blue-Green strategy for stateful services (databases) and Rolling deployments for stateless services. Feature flags (LaunchDarkly) allow new features to be deployed but turned off — enabling instant rollback without a redeploy.

Chat servers HA: WebSocket servers use consistent hashing — users are pinned to specific servers. If a server dies, affected users reconnect to a new server (the consistent hash ring re-balances). Undelivered messages are recovered from Cassandra on reconnect — no message loss.

Kafka HA: Kafka controller failover is automatic (Zookeeper or KRaft mode). With replication factor 3 and min.insync.replicas=2, Kafka tolerates one broker failure with zero data loss and zero downtime.

Redis HA: Redis Sentinel monitors master nodes and promotes a replica automatically if a master becomes unreachable (failover in under 10 seconds). Redis Cluster mode distributes slots across nodes — no single master handles all traffic.

Reliability

The system must consistently perform its function correctly, not just be available.

At-least-once delivery for notifications: Kafka guarantees at-least-once delivery. Notification Service deduplicates using a `notification_id` in Redis (TTL 24 hours) — even if the same event is consumed twice, only one notification is sent.

Durable message storage: Every message is written to Cassandra before the WebSocket delivery is attempted. If delivery fails, the message is still in Cassandra. Users fetch missed messages on reconnect using a cursor (last-seen message ID).

Connection integrity: A connection request acceptance must be atomic — both the "User A follows User B" and "User B follows User A" edges must be created together or not at all. Implemented as a two-phase write to Neptune with a compensating rollback on failure.

Job application integrity: Applying to a job is idempotent — the same user can hit "Apply" multiple times (network retry) and only one application is recorded. Enforced by a unique constraint on (user_id, job_id) in PostgreSQL.

Data replication: Cassandra replication factor 3 — data survives two node failures. PostgreSQL Aurora replicates synchronously to 2 AZs. S3 replicates objects to at least 3 AZs automatically (11 nines durability).

Resilience

The system must gracefully absorb failures and recover without cascading collapse.

Circuit breakers: Every service-to-service call is wrapped in a circuit breaker (Resilience4j). If the Graph Service starts timing out (>50% error rate in 10 seconds), the circuit opens — callers immediately receive a cached fallback response instead of waiting for timeouts. Circuit stays open for 30 seconds then tries again (half-open state).

Bulkhead pattern: The Feed Service has separate thread pools for cache reads, DB reads, and downstream service calls. A slow DB read does not consume all threads and block

cache reads — the bulkhead isolates the failure.

Retry with backoff: All Kafka consumers retry failed message processing with exponential backoff (1s, 2s, 4s, 8s) up to 3 retries. After 3 failures the message goes to a Dead Letter Queue (DLQ) for manual inspection — it is never silently dropped.

Timeout policies: Every external call has an explicit timeout — Graph Service: 100ms, Ranking Service: 50ms, Payment Service: 5000ms. If a timeout is exceeded, the caller uses a fallback (cached data or default response) rather than hanging indefinitely.

Chaos Engineering: LinkedIn runs chaos experiments (similar to Netflix Chaos Monkey) — randomly killing pods, injecting network latency, simulating AZ failures — to verify resilience properties before they matter in production.

Fault Tolerance

The system continues operating correctly even while components are failing.

Feed tolerance: If the Redis feed cache is completely unavailable (Redis Cluster failure), the Feed Service falls back to a direct Cassandra query — slower (200ms vs 5ms) but the feed still works. This degraded mode is automatically activated by the circuit breaker.

Graph tolerance: If Neptune is unreachable, PYMK falls back to the last cached result in Redis. Connection degree checks fall back to a simpler heuristic (same company or same school from PostgreSQL). Core functionality is not blocked.

Quorum writes in Cassandra: Cassandra writes use QUORUM consistency (majority of replicas must acknowledge). A single node failure does not block writes — the write succeeds on the remaining healthy nodes and the failed node catches up via read repair or anti-entropy when it recovers.

Kafka replication: With replication factor 3, Kafka tolerates 1 broker failure with no data loss and no downtime. Producers use `acks=all` — a message is only acknowledged after all in-sync replicas have written it.

Performance & Latency

Feed: Served entirely from Redis sorted sets — O(log N) lookup per user. Target: under 5ms for cache hit. Post hydration (author name, like count) served from a second Redis cache layer. Overall feed response: under 100ms from Feed Service, under 300ms end-to-end including network.

Search typeahead: Redis prefix trie returns suggestions in under 20ms. Full search via Elasticsearch returns in under 100ms for simple queries, under 200ms for compound filters.

Messaging: WebSocket delivery (online → online) under 50ms. Message write path (Kafka + Cassandra) is async — it does not block the WebSocket delivery. Push notification (offline) delivery depends on FCM/APNs — typically under 500ms.

Profile reads: Profile data cached in Redis with a 5-minute TTL. Cache hit rate target: 95%. Redis read latency: under 1ms. Cache miss triggers a PostgreSQL read (under 20ms) and repopulates cache.

Media delivery: Profile photos and post images served via CloudFront CDN. First request (cache miss) fetches from S3 origin — under 100ms. Subsequent requests served from edge cache — under 10ms globally. Cache-Control headers set to 24 hours for profile photos.

PYMK and job recommendations: Pre-computed nightly by Spark and stored in Redis. Served in under 10ms at read time — no live computation on the hot path.

Database query optimization: PostgreSQL tables have compound indexes on (user_id, created_at) for feed queries, (job_id, status) for applicant queries. Cassandra partition keys chosen for even data distribution and single-partition reads — no cross-partition scatter-gather queries on hot paths.

Consistency

Strong consistency (required):

- Connection requests — accepting a connection must atomically create edges in both directions in Neptune
- Payment and Premium subscriptions — ACID transactions in PostgreSQL

- Job applications — unique constraint ensures one application per (user, job)
- Account deletion — synchronous cascade across all services (GDPR hard requirement)

Eventual consistency (acceptable):

- Feed content — a new post may take up to 5 seconds to appear in all follower feeds
- Like counts and view counts — Redis counters flushed to Cassandra every 60 seconds (may show stale count briefly)
- Search index — up to 5 seconds lag from source DB to Elasticsearch via CDC
- PYMK recommendations — recomputed every hour, not real-time
- Notification delivery — may arrive a few seconds after the triggering event

CAP theorem trade-off: LinkedIn chooses AP (Availability + Partition tolerance) for feed, messaging, and search — accepting eventual consistency. It chooses CP (Consistency + Partition tolerance) for financial transactions and connection state.

Security

Authentication: OAuth 2.0 with JWT access tokens (15-minute expiry) and refresh tokens (30-day expiry stored in Redis). Every API call validates JWT signature at the API Gateway — no DB lookup on hot path.

Authorization: Role-based access control (RBAC). Recruiters can access applicant lists for their job postings only. Premium users can see who viewed their profile. Non-connections cannot see full profiles (visibility rules enforced at Profile Service layer).

Data encryption: All data encrypted in transit (TLS 1.3). All data at rest encrypted (AES-256) — PostgreSQL, Cassandra, S3, Redis all use encryption at rest. Private messages in Cassandra encrypted with per-conversation keys.

Rate limiting: API Gateway enforces per-user rate limits — 1000 API calls/minute for free users, 5000 for Premium. InMail limited to 30/month per Premium account tracked in PostgreSQL. PYMK and recommendation endpoints limited to prevent scraping.

PII protection: Email addresses, phone numbers, and private messages never logged. Audit logs capture action type and user ID but not PII content. S3 access logs and Kafka

consumer logs similarly scrubbed.

GDPR compliance: Data deletion triggers a saga across all services — profile deleted from PostgreSQL, posts soft-deleted from Cassandra (hard delete after 30 days), feed cache evicted from Redis, Elasticsearch index entry deleted, S3 media deleted. Full deletion completed within 30 days as required.

Extensibility

Event-driven architecture as the backbone: Every new feature that needs to react to existing events (a new "Trending Content" service, a new "Carbon Footprint" badge) simply subscribes to the existing Kafka topics. Zero changes to existing services.

New content types: Posts, articles, polls, and videos are all the same schema with a `content_type` field. Adding a new type (e.g. audio clips, newsletters) is a new enum value — the Fan-out, Feed, and Notification services are all content-type agnostic.

New notification channels: WhatsApp notifications, Slack integration — new Kafka consumer group + new channel handler. The Notification Service routing table is config-driven, not code-driven.

New recommendation signals: Adding a new ML feature (e.g. "users who recently changed jobs") to the recommendation engine is a new feature column in the Spark job. The serving layer (Redis + API) is unchanged.

API versioning: All APIs are versioned from day one (`/v1/`, `/v2/`). Old clients continue working on v1 while new features launch on v2. Breaking changes are never made to an existing version.

Feature flags: LaunchDarkly controls rollout of new features per user segment — 1% rollout, 10%, 100% — with instant kill switch. No code deployment needed to disable a bad feature.

Maintainability

Microservices with clear ownership: Each service (Profile, Feed, Job, Messaging, Search) is owned by a separate team. Service boundaries follow domain boundaries. Inter-service communication via Kafka (async) or gRPC (sync) with well-defined contracts (Protobuf schemas).

Centralized logging: All services emit structured JSON logs to Kafka → Logstash → Elasticsearch → Kibana (ELK Stack). Every log line contains: service name, trace ID, user ID (hashed), request ID, duration. Trace ID links all log lines across services for a single user request.

Distributed tracing: OpenTelemetry SDK in every service. Jaeger UI shows the full trace of any request across all services — critical for diagnosing why a feed request is slow (is it the cache? the DB? the ranking service?).

Monitoring and alerting: Prometheus scrapes metrics from every service. Grafana dashboards per service. PagerDuty alerts for: feed P99 latency > 500ms, Kafka consumer lag > 10 seconds, error rate > 1%, DLQ depth > 100 messages.

Schema migrations: PostgreSQL migrations managed by Flyway — versioned, repeatable, applied automatically on deploy. Cassandra schema changes use `ALTER TABLE ADD COLUMN` (non-breaking) — breaking schema changes require a new table and a data migration job.

Runbooks: Every alert in PagerDuty links to a runbook in Confluence documenting the likely causes and resolution steps. On-call engineers never debug from scratch.

Idempotency

Job applications: Unique constraint on (user_id, job_id) in PostgreSQL. Double-click on "Apply" returns the existing application — not an error, not a duplicate.

Connection requests: Neptune `MERGE` operation — creating an edge that already exists is a no-op. Retried connection requests are safe.

Message sending: Every message has a client-generated `client_message_id` (UUID). Server checks Redis for this ID before processing — if found, returns the existing message ID. If not found, processes and stores the ID in Redis with TTL 24 hours.

Payment: Every Premium subscription charge has an idempotency key (Stripe-compatible). Retried charges return the original charge result — never double-bill.

Notification deduplication: `notification_id` = hash(event_type + source_id + target_user_id + time_bucket). Same event within the same time bucket generates the same ID — deduplicated at the Notification Service before sending.

Observability

The three pillars — Logs, Metrics, Traces:

Logs: Structured JSON → Kafka → Logstash → Elasticsearch. Retention: 30 days hot, 1 year cold (S3). Queryable via Kibana.

Metrics: Prometheus pulls metrics every 15 seconds from all services. Key metrics: feed_load_latency_p99, kafka_consumer_lag_seconds, redis_cache_hit_ratio, elasticsearch_query_duration_p95, active_websocket_connections, notification_delivery_success_rate.

Traces: OpenTelemetry → Jaeger. Every request generates a trace spanning all service calls. Sampled at 1% for normal traffic, 100% for error traces.

Business metrics (ClickHouse): Post impressions, engagement rate per post, job application funnel, PYMK acceptance rate, InMail response rate, DAU/MAU ratio. Used by product teams to measure feature impact.

Dashboards and alerts:

- Feed latency P99 > 300ms → PagerDuty (critical)
- Kafka consumer lag > 30 seconds → PagerDuty (high)
- Redis cache hit ratio < 90% → Slack alert (medium)
- Error rate > 0.5% on any service → PagerDuty (high)
- DLQ depth growing → Slack alert (medium)
- Elasticsearch index lag > 10 seconds → Slack alert (low)

Disaster Recovery

RPO (Recovery Point Objective) — how much data loss is acceptable:

- User profiles, connections, job applications: RPO = 0 (synchronous replication to 2 AZs)
- Posts and messages: RPO = 1 minute (Cassandra async cross-region replication)
- Feed cache (Redis): RPO = not applicable — feed is regenerated from Cassandra on recovery
- Analytics (ClickHouse): RPO = 1 hour — some analytics data loss in a catastrophic failure is acceptable

RTO (Recovery Time Objective) — how fast must you recover:

- API layer: RTO = 60 seconds (Kubernetes re-schedules pods automatically)
- PostgreSQL primary failure: RTO = 30 seconds (Aurora automatic failover)
- Redis master failure: RTO = 10 seconds (Sentinel automatic promotion)
- Full region failure: RTO = 5 minutes (DNS failover to secondary region via Route 53)

Backup strategy: PostgreSQL automated snapshots every hour (Aurora), retained 35 days. Cassandra snapshots to S3 daily. S3 objects cross-region replicated (CRR). Kafka topic data retained for 7 days — can replay events to rebuild any derived state.

DR drills: Full region failover drill run quarterly. Database restore drill run monthly. Results logged and gaps addressed before next drill.

Rate Limiting & Throttling

Per-user limits: 1000 API calls/minute (free), 5000/minute (Premium). InMail: 30/month (Premium). Post creation: 100 posts/day. Connection requests: 100/week (prevents spam). Enforced via Redis token bucket algorithm at API Gateway.

Per-IP limits: 10,000 requests/minute per IP to prevent scraping and DDoS. Stricter on unauthenticated endpoints (login: 5 attempts/minute per IP with exponential backoff).

Service-to-service limits: Graph Service caps PYMK traversal depth at 3 hops regardless of caller. Elasticsearch caps any single query at 10,000 results (deep pagination is prohibited — use cursor-based approach instead).

Cost Optimization

Redis eviction: Feed caches for users inactive > 7 days are evicted (LRU policy). Regenerated on next login. Saves ~30% Redis memory.

Cassandra tiering: Posts older than 1 year moved to Cassandra cold storage tier (cheaper SSDs). Messages older than 3 years archived to S3 Glacier — still accessible but with 12-hour retrieval latency (acceptable for legal/compliance access).

Elasticsearch hot-warm-cold: Job postings closed > 90 days moved to warm tier (cheaper nodes). Posts older than 6 months moved to cold tier. Only active jobs and recent posts on expensive hot SSD nodes.

Spot instances for batch: Spark ML jobs (PYMK computation, job recommendations, profile embeddings) run on AWS Spot instances — 70% cost reduction. Jobs are checkpoint-enabled so they survive spot interruptions.

CDN offload: Target > 98% CDN cache hit ratio for media. Profile photos have 24-hour cache TTL. Properly setting Cache-Control headers is the single cheapest performance and cost win — reduces S3 origin requests by 50x.

Fan-out skip for inactive users: If a user has not logged in for 30 days, their feed cache is not updated during fan-out. Their feed is computed fresh on next login. This eliminates ~40% of fan-out writes (most accounts are inactive).

Summary — All Pillars

Pillar	Key Decision	Implementation
Scalability	Independent scaling per service	Kubernetes HPA, Cassandra node add, Elasticsearch sharding
Availability	Multi-AZ, no SPOF	Aurora Multi-AZ, Redis Sentinel, Kafka RF=3
High Availability	Active-Active multi-region	Route 53 latency routing, Blue-Green deploys
Reliability	At-least-once + deduplication	Kafka + Redis dedup via notification_id
Resilience	Circuit breaker + bulkhead	Resilience4j, graceful degradation to fallbacks
Fault Tolerance	Quorum writes, replica promotion	Cassandra QUORUM, Aurora auto-failover
Performance	Pre-compute hot paths	Redis feed cache, PYMK in Redis, CDN for media
Consistency	Strong for money/connections, eventual for feed	PostgreSQL ACID + Cassandra eventual
Security	JWT + RBAC + encryption	API Gateway auth, AES-256 at rest, TLS 1.3
Maintainability	Microservices + observability	ELK, Jaeger, Prometheus/Grafana, runbooks
Extensibility	Event-driven + API versioning	Kafka topics, feature flags, /v1/ /v2/
Observability	Logs + Metrics + Traces	ELK + Prometheus/Grafana + Jaeger
Idempotency	Client keys + unique constraints	Redis dedup, PostgreSQL unique index
Disaster Recovery	Multi-region + automated drills	Aurora CRR, Cassandra S3 backup, Route 53 failover
Rate Limiting	Token bucket per user + per IP	Redis token bucket at API Gateway
Cost Optimization	Eviction + tiering + spot instances	LRU eviction, S3 Glacier, Spark on Spot

5/7/2026, 7:09:54 AM

Draw the hld and explain the flow

5/7/2026, 7:10:13 AM

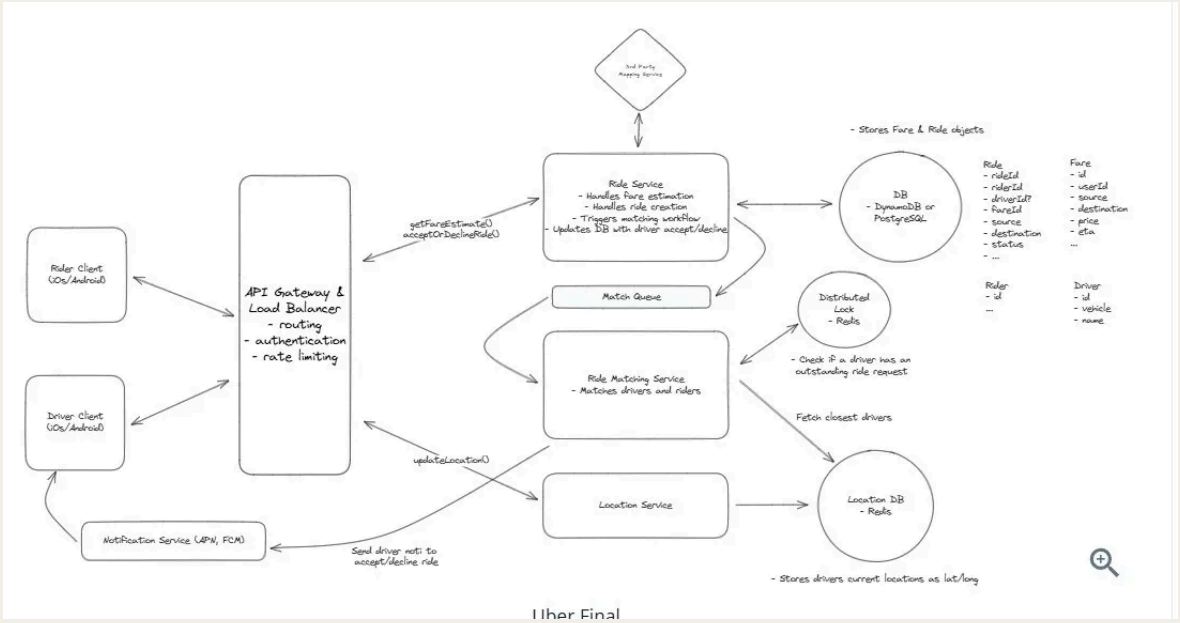
Here is the HLD diagram followed by a detailed flow explanation.

```
View: Check frontend design skill
```

The user wants the HLD diagram drawn inline and flow explained in text — not as an interactive artifact. Let me draw it as an SVG inline diagram.

```
Bash Tool: Create the LinkedIn HLD SVG diagram as a standalone file
```

5/7/2026, 7:11:38 AM



i am unable to view this diagram draw like this simple so i can view anywhere

5/7/2026, 11:23:38 AM

Here is your Uber design redrawn cleanly — same components, much more readable.

Architecture Flow:

Client Layer — Rider and Driver apps both connect through a single API Gateway which handles routing, authentication, and rate limiting.

Ride Service — When a rider requests a ride, the Ride Service calls the 3rd Party Mapping Service for fare estimation, creates the ride record in the DB, and pushes a job into the Match Queue.

Match Queue — Acts as a buffer between the Ride Service and the Ride Matching Service. Decouples ride creation from driver assignment so a slow matching process never blocks the rider's confirmation.

Ride Matching Service — Consumes from the Match Queue. Calls the Location DB (Redis) to fetch the closest available drivers using geo-queries. Checks the Distributed Lock (Redis)

to confirm the driver has no outstanding ride already. Sends the match result back to the Ride Service which updates the DB.

Location Service — Drivers call `updateLocation()` every 3–5 seconds. Location Service writes lat/long to Redis (fast geo-index). The Matching Service reads from this same Redis store to find nearest drivers.

Notification Service — Once a driver is matched, the Ride Service triggers the Notification Service (APNs for iOS, FCM for Android) to notify the driver to accept or decline. Driver response goes back through the API Gateway to the Ride Service.

DB (DynamoDB or PostgreSQL) — Stores Ride objects (rideId, riderId, driverId, fareId, source, destination, status) and Fare objects (id, userId, source, destination, price, eta).

```
Bash Tool: Create clean readable Uber HLD SVG
```

```
Present File: /mnt/user-data/outputs/uber_hld.svg
```

Here is the clean redrawn diagram. Let me walk through each numbered flow:

---

① Rider / Driver → API Gateway

Both Rider and Driver apps send all requests through a single **API Gateway + Load Balancer**. It handles JWT authentication, rate limiting, and routes the request to the correct service. Driver sends `updateLocation()` here; Rider sends `setFareEstimate()` and `acceptOrDeclineRide()`.

② API Gateway → Ride Service

The Ride Service is the **core orchestrator**. It calls the 3rd Party Mapping Service (Google Maps / Mapbox) to compute fare and ETA, creates the ride record in the **Ride DB** (DynamoDB or PostgreSQL) with status `REQUESTED`, then pushes a job into the Match Queue.

③ Ride Service → Match Queue → Ride Matching Service

The **Match Queue** (Kafka/SQS) decouples ride creation from driver assignment. If the Matching Service is slow or overloaded, rides queue up safely without blocking the rider's response. The Ride Matching Service consumes from the queue.

④ Ride Matching Service → Redis (Lock + Location)

Matching Service does two things in parallel — checks the **Distributed Lock (Redis)** to confirm the candidate driver has no outstanding ride already (prevents double-assignment), and queries the **Location DB (Redis Geo-index)** using `GEORADIUS` to fetch the closest available drivers. The result is sent back through the gateway to update the Ride DB with the assigned driver.

⑤ Ride Service → Notification Service → Driver

Once a driver is matched, the Ride Service triggers the **Notification Service** which uses **APNs** (iOS) or **FCM** (Android) to push a notification to the driver's device asking them to accept or decline. The driver's response travels back through the API Gateway.

⑥ Driver → Location Service → Location DB

Every 3–5 seconds, the Driver app calls `updateLocation()` → **Location Service** writes the driver's current lat/long to **Redis Geo-index**. This is the data the Matching Service reads in step ④ to find the nearest driver.

> *File: /mnt/user-data/outputs/uber_hld.svg*

5/7/2026, 11:24:59 AM

Design Kafka-like System Design YouTube / Netflix Design Uber Design Distributed Cache Design Payment System Design BookMyShow mention below these things in all system design functional requirements non - functional requirements (scalability, availibility, low latency & peformance, Reliablility etc..etc..) API design HLD Flow explaination (how the request flows will go) Detailed Component Design (Explain how they work, how they scale, discuss tradeoffs)

5/7/2026, 11:25:28 AM

I'll design all 6 systems one by one in full detail. Given the depth required, let me cover each thoroughly.

---

1. KAFKA-LIKE DISTRIBUTED MESSAGE QUEUE

Functional Requirements

- Producers publish messages to named topics
- Consumers subscribe to topics and read messages
- Topics split into partitions for parallelism
- Messages retained for a configurable duration (default 7 days)
- Consumer groups — multiple consumers share a topic, each partition assigned to one consumer
- Guaranteed message ordering within a partition
- At-least-once, at-most-once, and exactly-once delivery semantics
- Replay messages from any offset
- Support millions of messages per second throughput

Non-Functional Requirements

Scalability: Horizontally scale brokers by adding nodes. Partitions are the unit of parallelism — more partitions = more throughput. Consumer groups scale independently of producers.

Availability: Replication factor of 3 — data survives 2 broker failures. Leader election via ZooKeeper or KRaft (built-in Raft) promotes a replica to leader within seconds.

Low Latency & Performance: Sequential disk writes (append-only log) give near-memory-speed throughput. Zero-copy transfer (sendfile syscall) moves data from disk to network without copying to userspace. Batching at producer and consumer reduces network round trips. Target: under 10ms end-to-end latency, 1 million messages/second per broker.

Reliability: Every message written to disk before acknowledging to producer (acks=all). No message is lost if a broker crashes. WAL (write-ahead log) is the storage model — durable by design.

Durability: Messages stored on disk in segment files. Configurable retention by time or size. Once acknowledged, a message is never lost.

Fault Tolerance: If a partition leader dies, ZooKeeper/KRaft elects a new leader from in-sync replicas (ISR) in under 30 seconds. Producers and consumers reconnect automatically.

API Design

```
# Producer API
POST /v1/topics/{topic}/messages
Body: { partition_key: "user_123", payload: "...", headers: {} }
Response: { offset: 10042, partition: 3, timestamp: ... }

# Consumer API
GET /v1/topics/{topic}/partitions/{partition}/messages
     ?offset={}&limit={}&consumer_group={}
Response: { messages: [...], next_offset: 10043 }
```

```
# Topic Management
POST   /v1/topics                          → Create topic
GET    /v1/topics/{topic}                  → Topic metadata
DELETE /v1/topics/{topic}                  → Delete topic
POST   /v1/topics/{topic}/partitions       → Increase partition
count

# Consumer Group
POST /v1/consumer-groups/{group}/join      → Join group (triggers
rebalance)
PUT  /v1/consumer-groups/{group}/commit    → Commit offset
GET  /v1/consumer-groups/{group}/offsets   → Current committed
offsets
```

HLD

```
Producers → [Load Balancer] → [Broker Cluster]
                                    ↓
                              [ZooKeeper / KRaft]
                              (leader election,
                               cluster metadata)
                                    ↓
                         [Partition Log on Disk]
                         Segment files, index files
                                    ↓
                              [Replication]
                         Leader → Follower replicas
                                    ↓
Consumers ← [Consumer Group Coordinator] ← Brokers
```

Flow Explanation

Producer flow: Producer calls the bootstrap broker to get cluster metadata (which broker holds which partition). Producer hashes the partition key to select a partition. Messages are batched in memory (linger.ms = 5ms) and compressed (LZ4/Snappy). Batch sent to the partition leader broker. Leader appends to the partition log (sequential disk write). Followers fetch the new data and acknowledge. Once min.insync.replicas have acknowledged, the broker sends ACK to producer.

Consumer flow: Consumer joins a consumer group → Group Coordinator assigns partitions (one partition per consumer). Consumer polls the broker for its assigned partitions with a fetch request. Broker reads from the partition log starting at the committed offset. Consumer processes messages and commits the new offset. If a consumer crashes, the Group Coordinator detects the heartbeat timeout and triggers a rebalance — reassigning that partition to another consumer.

Rebalance flow: Consumer joins or leaves → Group Coordinator sends `SyncGroup` request to all consumers → partitions redistributed using range or round-robin assignment strategy → consumers resume from last committed offset.

Detailed Component Design

Partition Log (Storage)

Each partition is an append-only log stored as segment files on disk (default 1GB per segment). Each segment has a `.log` file (messages) and `.index` file (sparse offset-to-byte-position index for O(log N) seeks). Writing is purely sequential — no random I/O — making it as fast as network throughput. Older segments are deleted or compacted based on retention policy. Log compaction keeps the latest value per key — useful for changelog topics (like a DB CDC stream).

Tradeoff: More partitions = more parallelism but more file handles and longer leader election time. Rule of thumb: partitions per broker ≤ 4,000.

Replication

Each partition has one leader and N-1 followers (N = replication factor). Followers pull data from the leader continuously. The ISR (In-Sync Replicas) list tracks which followers are fully

caught up. Only ISR members can be elected leader. If a follower falls behind (replica.lag.time.max.ms exceeded), it is removed from ISR. Producer with acks=all waits for all ISR members — strong durability guarantee.

Tradeoff: Higher replication factor = more durability but more write latency (waiting for followers). acks=1 is faster but risks data loss if leader crashes before replication.

Consumer Group Coordinator

A dedicated broker acts as Group Coordinator for each consumer group. Manages membership, heartbeat timeouts (session.timeout.ms = 10s), and partition assignment. Stores committed offsets in a special internal topic `__consumer_offsets` (replicated, durable). This means offset data survives broker failure.

Tradeoff: Frequent rebalances (due to consumer churn) pause all consumption. Mitigation: sticky partition assignment (consumers keep their partitions across rebalances), incremental cooperative rebalancing (only move partitions that need to move).

Zero-Copy Transfer

When a consumer fetches messages, the broker uses the `sendfile()` Linux syscall to transfer data directly from the page cache (OS file buffer) to the network socket — bypassing userspace entirely. This eliminates 2 memory copies and 2 context switches per read. Result: brokers can saturate a 10Gbps network link serving consumers without CPU becoming the bottleneck.

Exactly-Once Semantics

Producer assigns a PID (Producer ID) and sequence number to each message. If a duplicate is sent (network retry), the broker detects the sequence number already exists and deduplicates. For cross-partition exactly-once, producers use transactions: `beginTransaction()` → write to multiple partitions → `commitTransaction()`. Consumers with `isolation.level=read_committed` only see committed transaction data.

---

2. YOUTUBE / NETFLIX — VIDEO STREAMING PLATFORM

Functional Requirements

- Users upload videos (YouTube) or content is ingested from studios (Netflix)
- Videos transcoded into multiple resolutions: 360p, 720p, 1080p, 4K
- Adaptive bitrate streaming — player switches quality based on bandwidth
- Global playback with low buffering
- Search videos by title, description, tags
- Recommendations — personalised home feed
- Like, comment, subscribe (YouTube) / continue watching, ratings (Netflix)
- Live streaming (YouTube)
- Offline download (Netflix mobile)
- Content DRM for premium/licensed content

Non-Functional Requirements

Scalability: 500 hours of video uploaded per minute (YouTube). 200M+ concurrent streams (Netflix). Upload, transcoding, and streaming are completely decoupled and scale independently. Transcoding workers auto-scale based on job queue depth.

Availability: 99.99% for streaming. CDN has built-in multi-region redundancy. If one CDN PoP fails, DNS reroutes to the next nearest. Origin (S3) has 11-nines durability.

Low Latency & Performance: Video chunks served from CDN edge nodes — under 30ms TTFB globally. Adaptive bitrate (HLS/DASH) prevents buffering by dynamically choosing resolution. Prefetch next 3 chunks in the buffer.

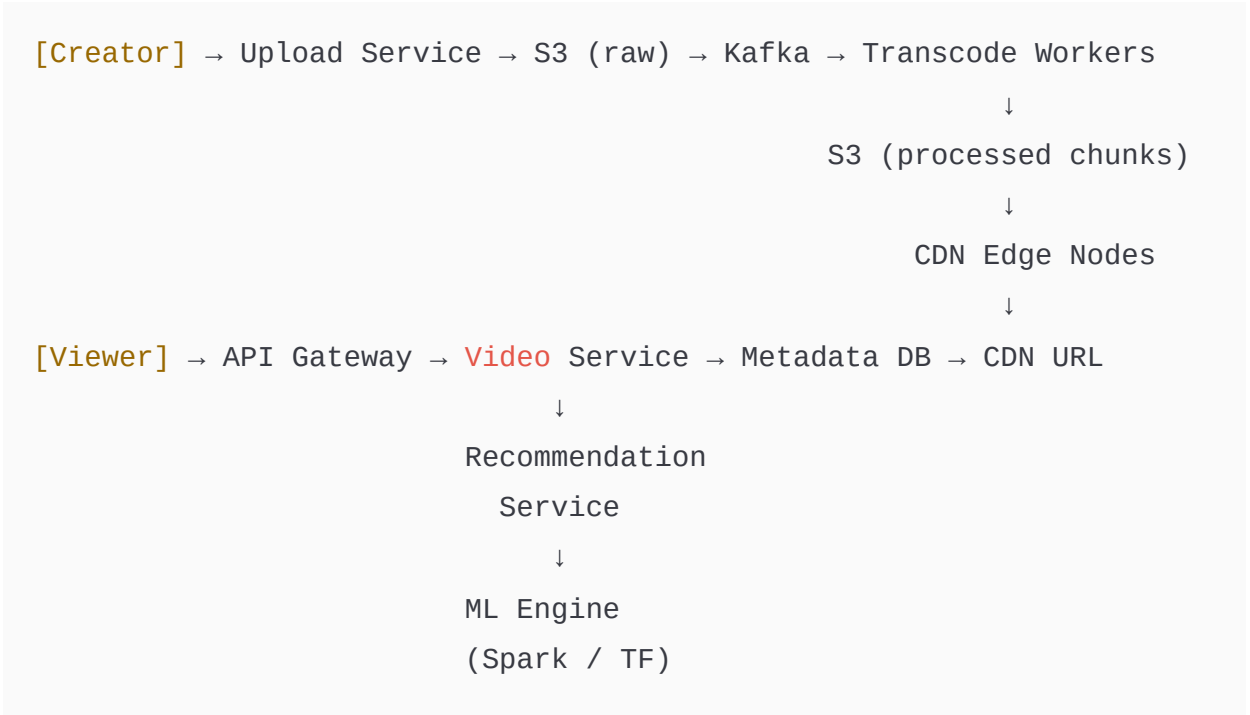
Reliability: Chunked upload with resumable protocol (tus). If upload drops at 80%, resume from chunk 80. Transcoding jobs are idempotent — crash and retry safely. Kafka guarantees at-least-once delivery for the transcoding pipeline.

Consistency: Video availability (is the video ready?) uses strong consistency — a video is only marked `PUBLISHED` after all resolutions are confirmed transcoded. View counts use eventual consistency — Redis counter flushed to DB periodically.

API Design

```
# Upload
POST   /v1/videos/initiate          → Get upload URL + video_id
PUT    /v1/videos/{video_id}/chunk  → Upload chunk (resumable)
POST   /v1/videos/{video_id}/finalize → Trigger transcoding

# Playback
GET    /v1/videos/{video_id}             → Metadata + manifest URL
GET    /v1/videos/{video_id}/stream      → HLS/DASH manifest (.m3u8)
POST   /v1/videos/{video_id}/progress    → Save watch progress
(position)

# Discovery
GET    /v1/feed?user_id={}               → Personalised
recommendations
GET    /v1/search?q={}&filters={}        → Search results
GET    /v1/videos/{video_id}/related     → Related videos

# Interactions
POST   /v1/videos/{video_id}/like
POST   /v1/videos/{video_id}/comments
GET    /v1/channels/{channel_id}/subscribe
```

HLD



Flow Explanation

Upload flow: Creator calls `/initiate` → Video Service generates a pre-signed S3 URL and a `video_id`. Client uploads directly to S3 in chunks (never through the app server — too slow). On finalize, S3 triggers a Lambda which publishes `video.uploaded` to Kafka. Transcoding workers pick up the job, pull the raw file from S3, transcode into 5 resolutions using FFmpeg, and store each as HLS segment files (`.ts` chunks + `.m3u8` manifest) back to S3. Each resolution has its own manifest. A master manifest references all quality levels. On completion, the Video Service marks the video `PUBLISHED` in PostgreSQL and triggers CDN cache warming for popular content.

Playback flow: Viewer opens a video → `GET /v1/videos/{id}` returns metadata and the master manifest URL (pointing to CDN). Player fetches the master manifest from CDN → selects the appropriate quality manifest based on measured bandwidth → fetches 2-second video chunks (`.ts` files) from CDN. Player continuously measures download speed and switches quality up or down. Buffer target: 30 seconds ahead. If CDN cache misses, CDN pulls from S3 origin (cache-fill).

Recommendation flow: Every view, like, search, and completion event is streamed to Kafka. A Spark Streaming job updates a user-item interaction matrix in real time. A nightly

Spark ML batch job runs collaborative filtering (ALS algorithm) to compute top-100 recommendations per user and stores them in Redis. The Feed Service reads from Redis — sub-10ms response.

Detailed Component Design

Transcoding Pipeline

The most resource-intensive component. Each video is split into segments and transcoded in parallel across multiple workers (Spot instances for 70% cost saving — jobs are retryable). A video manager orchestrates the pipeline: split → transcode each resolution → merge manifests → validate → publish. Output format: HLS (Apple) and DASH (Android/Web) for maximum device compatibility.

Scaling: Worker pool scales with Kafka consumer lag. If the queue grows, Kubernetes HPA adds more transcoding pods. Each worker is stateless — crash and restart picks up from where the Kafka offset left off.

Tradeoff: HLS segment size — 2 seconds gives fast quality switching but more HTTP requests. 10 seconds reduces requests but slower adaptation. Netflix uses 2s, YouTube uses 4s.

CDN Architecture

Videos are never served from origin in production. CloudFront/Akamai has 400+ PoPs globally. Each PoP caches the most popular chunks. Cache key = `{video_id}/{resolution}/{segment_number}`. Popular videos (top 1%) are pre-warmed to all PoPs. Long-tail videos (99%) are fetched from S3 on first request and cached at the PoP for subsequent viewers in the same region.

Tradeoff: Longer CDN TTL = better cache hit ratio but stale content if the video is deleted or updated. Use surrogate keys (tag-based invalidation) to purge specific video content without clearing the entire cache.

Adaptive Bitrate (ABR)

The player implements the BOLA algorithm (Buffer Occupancy based Lyapunov Algorithm). It decides the next segment quality based on: current buffer level, measured throughput, and quality change penalty. If buffer > 20 seconds: step up quality. If buffer < 5 seconds: step down immediately. The goal is maximising quality while keeping rebuffering probability under 0.1%.

Metadata & Search

Video metadata (title, description, tags, view count, duration) stored in PostgreSQL. Search powered by Elasticsearch with a custom ranking function: text relevance (BM25) × view count boost × recency decay × personalisation score. Elasticsearch updated via Kafka CDC consumer within 5 seconds of a new upload being published.

---

3. UBER — REAL-TIME RIDE SHARING

Functional Requirements

- Riders request rides from source to destination
- Real-time driver matching based on proximity
- Live GPS tracking of driver on rider's map
- Fare estimation before booking, final fare after trip
- Multiple ride types: economy, premium, pool
- Driver accept/decline with timeout (15 seconds)
- Trip state machine: requested → matched → in-progress → completed
- Payment processing (card, UPI, wallet, cash)
- Ratings — rider rates driver and vice versa
- Surge pricing during high demand

Non-Functional Requirements

Scalability: 5 million rides per day, 10 million active drivers globally. Location updates from all active drivers every 4 seconds = 2.5 million writes/second globally. Geo-query to find nearest drivers must return in under 100ms.

Availability: 99.99%. Rider app downtime = zero revenue. Multi-region active-active deployment. No single broker or service is a single point of failure.

Low Latency & Performance: Driver matching must complete in under 5 seconds. Live location updates must reach rider's map within 2 seconds. Fare calculation under 500ms including map API call.

Reliability: Trip state is durable in PostgreSQL with WAL replication. A completed trip is never lost. Payment is idempotent — a network failure during charge never double-bills. Kafka guarantees event delivery for trip lifecycle events.

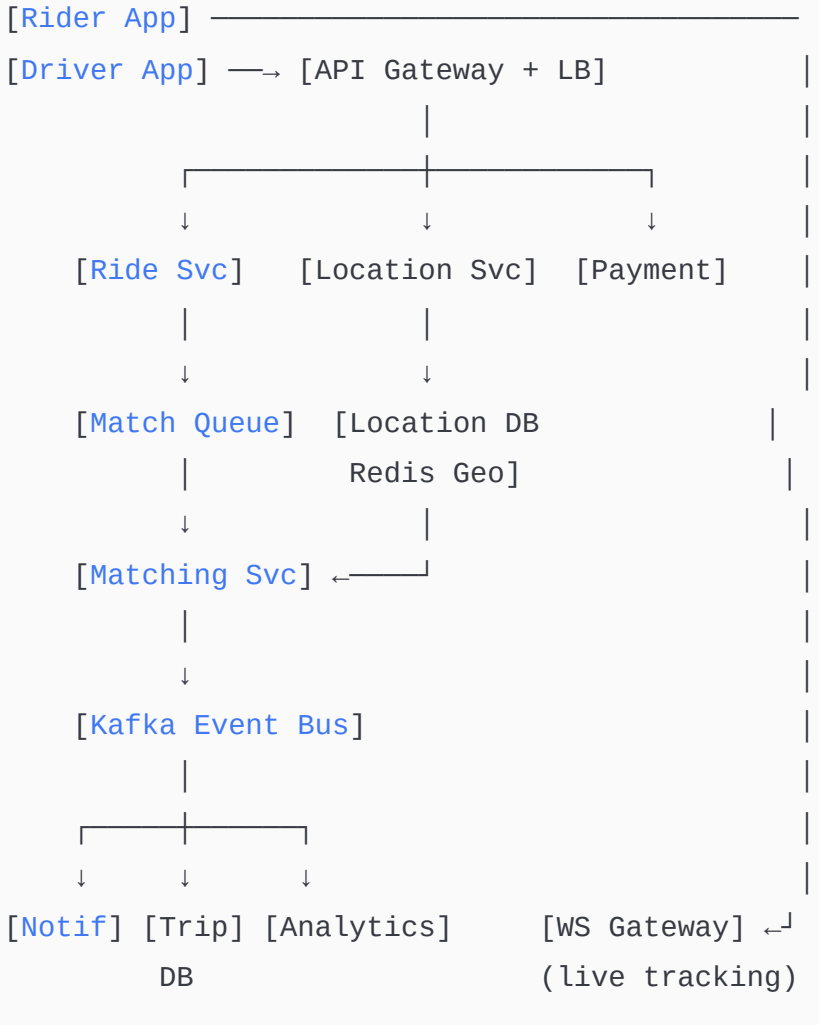
Consistency: Trip state uses strong consistency — a driver cannot be matched to two rides simultaneously (distributed lock in Redis). Location data uses eventual consistency — a 1-2 second staleness is acceptable.

API Design

```
# Rider APIs
POST   /v1/rides/estimate          → Fare + ETA before booking
POST   /v1/rides                   → Request a ride (idempotency-key
required)
GET    /v1/rides/{ride_id}         → Current ride status
GET    /v1/rides/{ride_id}/track   → Driver live location (WebSocket)
POST   /v1/rides/{ride_id}/cancel  → Cancel ride

# Driver APIs
PUT    /v1/drivers/{id}/location   → Update GPS location (every 4s)
PUT    /v1/drivers/{id}/status     → Go online / offline
POST   /v1/rides/{ride_id}/accept  → Accept a matched ride
POST   /v1/rides/{ride_id}/decline → Decline (triggers re-matching)
```

```
PUT   /v1/rides/{ride_id}/status  → Update trip status (started,
completed)

# Payment
POST  /v1/payments                → Charge after trip completion
GET   /v1/payments/{ride_id}      → Payment status

# Ratings
POST  /v1/rides/{ride_id}/rating  → Submit rating (both parties)
```

HLD



Flow Explanation

Ride request flow: Rider taps "Request" → Ride Service receives request with idempotency key → calls Maps API for fare estimate → writes ride record to PostgreSQL with status `REQUESTED` → publishes `ride.requested` to Kafka → pushes a matching job to the Match Queue (Redis list).

Driver matching flow: Matching Service consumes from Match Queue → queries Location DB (Redis GEORADIUS command) to find all drivers within 3km → filters for drivers with status `AVAILABLE` → acquires a distributed lock on the top candidate driver (Redis SETNX with 15s TTL) to prevent double-matching → sends push notification to driver via Notification Service → starts a 15-second timer. If driver declines or timer expires: lock released → try next candidate. If driver accepts: ride status updated to `MATCHED`, lock converted to trip lock.

Live tracking flow: Driver app sends GPS location every 4 seconds via WebSocket to Location Service → Location Service updates Redis Geo-index. Rider's app subscribes to a WebSocket channel keyed by `ride_id` → a fan-out process reads the driver's location from Redis and pushes updates to the rider's WebSocket connection. Rider sees driver moving on the map with 4-second refresh.

Trip completion and payment: Driver marks trip `COMPLETED` → Ride Service calculates final fare (applying surge multiplier if applicable) → calls Payment Service with idempotency key → Payment Service charges the rider's stored payment method → publishes `payment.completed` to Kafka → Notification Service sends receipt to rider.

Detailed Component Design

Location Service — The highest-write-throughput component

5M active drivers × 1 update/4 seconds = 1.25M writes/second. Architecture: Location Service receives updates via WebSocket (persistent connection — no HTTP overhead per update). Writes to Redis using `GEOADD drivers {lng} {lat} {driver_id}` — atomic O(log

N) geo-insert. Redis geo-index uses a sorted set with geohash-encoded scores. `GEORADIUS` query returns all drivers within a radius in O(N+log M) time.

Scaling: Redis Cluster with consistent hashing — each shard handles a geographic region. At 1.25M writes/second, a single Redis cluster of 10 shards (each handling 125K writes/second) is sufficient (Redis handles 500K simple ops/second per node).

Tradeoff: Redis is in-memory — expensive at scale. Mitigation: store only the latest position per driver (not history). Historical location trail stored separately in TimescaleDB (time-series) for dispute resolution and ETA model training.

Matching Service — The core algorithm

Matching is a real-time bipartite assignment problem. The scoring function for each candidate driver: `score = w1/distance + w2×acceptance_rate + w3×rating + w4×ride_count_today`. Runs in under 100ms — queries at most 20 nearest drivers and scores them in memory. No external ML model on the hot path.

Distributed lock (Redis SETNX): When a driver is selected as a candidate: `SET driver:{driver_id}:lock {ride_id} EX 15 NX`. This prevents two simultaneous ride requests from being assigned the same driver. Lock auto-expires after 15 seconds (driver timeout).

Tradeoff: Greedy nearest-driver assignment is fast but suboptimal globally. A globally optimal assignment (Hungarian algorithm) would be too slow ($O(n^3)$). Uber uses a batched matching approach — collect all ride requests in a 3-second window and solve a small assignment problem per cell — balancing optimality and latency.

Surge Pricing

A background service monitors demand-supply ratio per geographic cell (H3 hexagonal grid, resolution 7 = ~5km^2 cells) every 30 seconds. Surge multiplier = `max(1.0, demand/supply)` capped at 3.0. Stored in Redis with a 30-second TTL. Fare Estimation Service reads the surge multiplier for the rider's pickup cell before quoting the fare.

Trip State Machine (PostgreSQL)

States: `REQUESTED → SEARCHING → MATCHED → IN_PROGRESS → COMPLETED / CANCELLED`. Transitions enforced by DB-level CHECK constraints and application-level validation. Each

transition is a Kafka event. A completed trip record is immutable — never updated, only appended. Payment records reference the trip — full audit trail.

---

4. DISTRIBUTED CACHE (like Redis / Memcached)

Functional Requirements

- Set key-value pairs with optional TTL
- Get value by key (O(1))
- Delete a key
- Support multiple data types: string, hash, list, set, sorted set
- Eviction policies: LRU, LFU, TTL-based
- Pub/Sub messaging channel
- Atomic operations: INCR, DECR, SETNX
- Persistence options: RDB snapshots, AOF (append-only file)
- Replication — primary-replica for HA

Non-Functional Requirements

Scalability: Cluster mode with consistent hashing across nodes. Adding a node redistributes only K/N keys (K = total keys, N = nodes). Scale to hundreds of billions of keys by adding nodes. Each node handles 100K–500K ops/second.

Availability: Redis Sentinel monitors primary nodes — if primary fails, Sentinel promotes a replica in under 10 seconds. Redis Cluster mode has built-in failover — no external Sentinel needed at scale.

Low Latency & Performance: All data in RAM. Single-threaded command execution (no lock contention). Network I/O handled by epoll (event-driven, non-blocking). Target: under 1ms for GET/SET at p99. Under 100 microseconds for in-datacenter calls.

Reliability: AOF persistence — every write command appended to a log file. On restart, replay the AOF to rebuild state. RDB snapshots for faster restarts on large datasets. fsync every second (fsync=everysec) balances durability and performance — at most 1 second of data loss on crash.

Consistency: Single-node Redis is strongly consistent (single thread). Redis Cluster has eventual consistency across replicas — a read from a replica may miss the latest write to the primary.

API Design

```
# Core Operations
SET  key value [EX seconds] [NX]     → Set with optional TTL, NX =
set if not exists
GET  key                             → Returns value or nil
DEL  key [key ...]                   → Delete one or more keys
TTL  key                             → Remaining time to live in
seconds
EXPIRE key seconds                   → Set/update TTL on existing
key

# Atomic Operations
INCR  key                            → Increment integer value by 1
DECR  key                            → Decrement
SETNX key value                      → Set only if key does not
exist (distributed lock)

# Data Structures
HSET  hash field value               → Set hash field
HGET  hash field                     → Get hash field
LPUSH list value                     → Push to list head
```

```
LRANGE list start stop                → Get list range
ZADD  zset score member               → Add to sorted set
ZRANGE zset start stop [WITHSCORES]   → Get sorted set range

# Pub/Sub
PUBLISH  channel message              → Publish message
SUBSCRIBE channel                     → Subscribe to channel

# Cluster
CLUSTER INFO                          → Cluster health
CLUSTER NODES                         → Node topology
```

HLD

```
Clients → [Client Library (hashing)]
                      |
          ┌───────────┼────────────┐
          ↓           ↓            ↓
    [Node 1]    [Node 2]    [Node 3]     ← Primary nodes
    Slots 0–    5461–       10923–         (hash slots)
    5460        10922       16383
         |           |           |
    [Replica1]  [Replica2]  [Replica3]  ← Replica nodes

[Sentinel / Cluster Bus]
(heartbeat, leader election, failover)
```

Flow Explanation

Write flow: Client hashes the key using CRC16 → maps to one of 16,384 hash slots → slot is assigned to a specific node → client connects directly to that node (no proxy needed —

Redis Cluster clients are slot-aware). Node writes to RAM, appends to AOF if enabled, replicates asynchronously to replica. Returns OK.

Read flow: Client computes slot → connects to the node owning that slot. If the key is not in the primary and the client reads from a replica (`READONLY` mode), it may get a slightly stale value. For strongly consistent reads, always read from primary.

Eviction flow: When memory hits `maxmemory` threshold, the eviction policy kicks in. LRU: maintains an approximate LRU clock (samples 5 random keys and evicts the least recently used among them — full LRU tracking is too memory-expensive). LFU: tracks access frequency with a logarithmic counter. TTL-based: a background thread (`activeExpireCycle`) samples random keys and deletes expired ones every 100ms. Lazy deletion: a key is also checked for expiry on access.

Failover flow: Replica continuously replicates from primary via partial resync (PSYNC). If primary stops responding → Sentinel detects after `down-after-milliseconds` (default 30s) → Sentinel majority vote → promotes the most up-to-date replica → updates all Sentinel and client configurations. Clients reconnect automatically.

Detailed Component Design

Memory Management

Redis uses jemalloc as its allocator. Each key-value pair has a `redisObject` struct (16 bytes header + data). Small strings (≤44 bytes) use embstr encoding (single allocation). Larger strings use raw encoding (separate allocation). Hashes with ≤128 fields use ziplist (compact memory). Larger hashes use hashtable. This encoding duality reduces memory by 3–5x for small objects. The `OBJECT ENCODING key` command shows the current encoding.

Tradeoff: ziplist encoding is CPU-intensive for large datasets (linear scan). The threshold (hash-max-ziplist-entries=128) is tunable — lower threshold = more memory, faster ops; higher threshold = less memory, slower ops for large hashes.

Consistent Hashing (Cluster Mode)

Redis Cluster uses 16,384 hash slots. Each node owns a range of slots. The client library maps `CRC16(key) % 16384` to a node. When a new node joins: the cluster manager

redistributes slots by moving keys between nodes — only K/N keys need to move (minimal disruption). Hash tags `{user_id}` force related keys to the same slot — enabling multi-key operations on a single node.

Tradeoff: Hash tags enable multi-key ops but can cause hot spots if many keys share the same tag. Monitor slot distribution and rebalance if a single slot gets disproportionate traffic.

Persistence: AOF vs RDB

RDB (snapshot): fork() the process, write a point-in-time snapshot to disk. Fast restart (binary format). Risk: up to 5 minutes of data loss (snapshot interval). Fork() can pause Redis for seconds on large datasets (copy-on-write memory pages).

AOF (append-only file): every write command appended to file. `fsync=everysec` — at most 1 second of data loss. AOF file grows large — compacted via `BGREWRITEAOF` (creates a new minimal AOF). Slower restart than RDB (replays all commands).

Best practice: Use both — RDB for fast restarts, AOF for durability. Redis can reconstruct from AOF after a crash without replaying from scratch if RDB is more recent.

Pub/Sub

A channel is not persisted — it's purely in-memory. A PUBLISH command delivers a message to all current subscribers of that channel in O(N) time (N = number of subscribers). No delivery guarantee — if a subscriber is not connected at publish time, the message is lost. For reliable messaging, use Redis Streams (`XADD`, `XREAD`) which provides consumer groups and message retention — a more Kafka-like model within Redis.

---

5. PAYMENT SYSTEM

Functional Requirements

- User adds a payment method (card, UPI, bank account, wallet)
- Process payments between two parties (payer → payee)
- Support payment types: one-time, subscription, split payments
- Refunds — full and partial
- Transaction history with pagination
- Real-time payment status updates
- Multi-currency support with FX conversion
- Fraud detection on every transaction
- PCI-DSS compliant card data handling
- Bank transfers (NEFT, IMPS, SWIFT)
- Idempotency — retried payments never double-charge

Non-Functional Requirements

Scalability: 10,000 transactions per second at peak (Diwali sale, Black Friday). Payment processing services scale horizontally. DB writes use sharding by user_id to distribute load.

Availability: 99.999% (five nines — under 5 minutes downtime/year). Payment downtime = direct revenue loss. Active-Active across two data centres with synchronous DB replication.

Low Latency & Performance: Payment confirmation under 3 seconds for card transactions, under 500ms for UPI (IMPS). Card network API calls (Visa/Mastercard) add 200–800ms. Internal DB operations under 50ms.

Reliability: Every transaction written to WAL before processing. Exactly-once processing — the same payment request processed multiple times produces exactly one charge. Idempotency keys stored in Redis for 24 hours.

Consistency: Strong consistency — ACID transactions. A payment either fully succeeds or fully fails — no partial state. Saga pattern for distributed transactions across services.

Security: PCI-DSS Level 1. Raw card numbers never stored — tokenised via Vault (HashiCorp or payment network tokenisation). All communication over mTLS. Encryption at

rest (AES-256).

API Design

```
# Payment Methods
POST   /v1/payment-methods           → Add card/UPI/bank (returns
token, never raw data)
GET    /v1/payment-methods           → List saved methods
DELETE /v1/payment-methods/{id}      → Remove payment method

# Payments
POST   /v1/payments                  → Initiate payment
(idempotency-key header required)
Body: {
  amount: 1000,
  currency: "INR",
  payer_id: "user_123",
  payee_id: "merchant_456",
  payment_method_id: "pm_abc",
  description: "Order #789"
}
Response: { payment_id, status: "processing", estimated_completion:
... }

GET    /v1/payments/{payment_id}     → Payment status
POST   /v1/payments/{payment_id}/capture  → Capture pre-authorised
payment
POST   /v1/payments/{payment_id}/cancel   → Cancel before capture

# Refunds
POST   /v1/refunds                   → Initiate refund
Body: { payment_id, amount, reason }
GET    /v1/refunds/{refund_id}       → Refund status
```

```
# Webhooks
POST   /v1/webhooks                    → Register webhook URL for
payment events
```

HLD

Flow Explanation

Payment initiation: Client sends `POST /v1/payments` with `Idempotency-Key: uuid-123` → Payment Service checks Redis for this key (prevent duplicate). If new: validate amount and currency → call Fraud Service (synchronous, under 200ms) → if fraud score < threshold: proceed → call Payment Gateway Adapter → Adapter routes to appropriate network (Visa for card, NPCI for UPI).

Ledger write (double-entry): Every payment creates two ledger entries atomically: debit payer's account, credit payee's account. This is the double-entry bookkeeping model — the sum of all debits always equals the sum of all credits. Implemented as a single PostgreSQL transaction with serializable isolation.

Card network flow: Payment Gateway Adapter sends an authorisation request to Visa/Mastercard with: card token, amount, merchant ID, CVV hash. Network responds with authorisation code (200–800ms). Adapter writes `AUTHORISED` status to DB. A separate capture step moves money from cardholder's bank to merchant (T+1 or T+2 days for settlement).

Saga for distributed failure: If the ledger write succeeds but the card network call fails: the Saga Orchestrator runs a compensating transaction — reverses the ledger debit entry. If the card network charge succeeds but the DB write fails: the Payment Service retries the DB write (idempotent). If all retries fail: the charge is logged to an anomaly queue for manual reconciliation.

Refund flow: `POST /v1/refunds` → Refund Service creates a new refund record → calls Payment Gateway Adapter with the original `authorisation_code` → network reverses the charge → Ledger Service creates reverse entries (credit payer, debit payee) → event published to Kafka → Notification Service sends confirmation.

Detailed Component Design

Idempotency Engine

Every payment API call requires a client-supplied `Idempotency-Key` header (UUID). The Payment Service computes `SHA256(idempotency_key + user_id)` and stores it in Redis with the payment result (TTL 24 hours). If the same key arrives again (retry), the stored

result is returned immediately — the payment is not processed again. This is safe for network retries, duplicate requests, and client crashes.

Tradeoff: Redis is not durable by default. If Redis crashes before AOF sync, an idempotency key is lost — a retry could double-charge. Mitigation: also write the idempotency key to PostgreSQL (slower but durable) and use Redis as a fast cache over PostgreSQL.

Ledger Service (Double-Entry Accounting)

Every monetary movement creates exactly two entries: a debit and a credit. Schema:

```
ledger_entry: entry_id, account_id, amount, type(DEBIT/CREDIT),
              currency, payment_id, created_at
```

Account balance = SUM of credits - SUM of debits for that account. Never update a balance field directly — compute it from ledger entries. This makes the ledger an immutable audit trail — any balance at any point in time can be reconstructed.

Scaling: Ledger is append-only (inserts only, no updates). PostgreSQL partitioned by `created_at` (monthly partitions) keeps hot data in fast storage. Old partitions moved to cold storage. Read queries for balance use a materialized view updated every 60 seconds.

Fraud Detection Service

Runs synchronously on the payment hot path (must return in under 200ms). Uses a feature store (Redis) with pre-computed user features: average transaction amount, transaction frequency in last 1 hour, device fingerprint, geolocation anomaly (payment from a new country). Scores each transaction using a gradient boosting model (XGBoost, served via TensorFlow Serving). Score > 0.85 → block. Score 0.5–0.85 → trigger 3D Secure (OTP). Score < 0.5 → approve.

Tradeoff: Synchronous fraud check adds latency but catches fraud before charge. Asynchronous check is faster but allows fraudulent charges that must be reversed (chargebacks are expensive — $15–50 per chargeback).

Reconciliation

End-of-day job compares Ledger DB entries against Card Network settlement reports and Bank statements. Mismatches (failed network call after successful DB write, or vice versa)

are flagged as exceptions and routed to a reconciliation queue for manual review or automated retry. This is the financial safety net — ensures no money is created or destroyed.

---

6. BOOKMYSHOW — TICKET BOOKING SYSTEM

Functional Requirements

- Browse movies, events, sports by city and date
- Select cinema/venue, date, show time
- View seat map — available, booked, selected seats
- Seat selection and temporary hold (5-minute lock)
- Booking with payment
- E-ticket generation (QR code)
- Cancellation and refund
- Offers and promo codes
- Notifications — booking confirmation, reminders
- Partner APIs — cinemas update show schedules and seat availability

Non-Functional Requirements

Scalability: 20 million daily active users. 5 million bookings per day. Peak traffic: new Marvel movie release — 100,000 concurrent users all booking the same show at the same time. The system must not oversell a single seat.

Availability: 99.99%. A booking failure during a popular release is extremely high visibility.

Low Latency & Performance: Seat map load under 200ms. Booking confirmation under 3 seconds. Seat hold must be instant (Redis atomic operation) — if it takes 5 seconds, 100 users trying the same seat will all think they got it.

Reliability: A seat can only be booked once — enforced at the DB level (unique constraint) and application level (distributed lock). No double booking under any race condition.

Consistency: Strong consistency for seat booking — pessimistic locking. No two users can book the same seat. Eventual consistency acceptable for movie listings, reviews, recommendations.

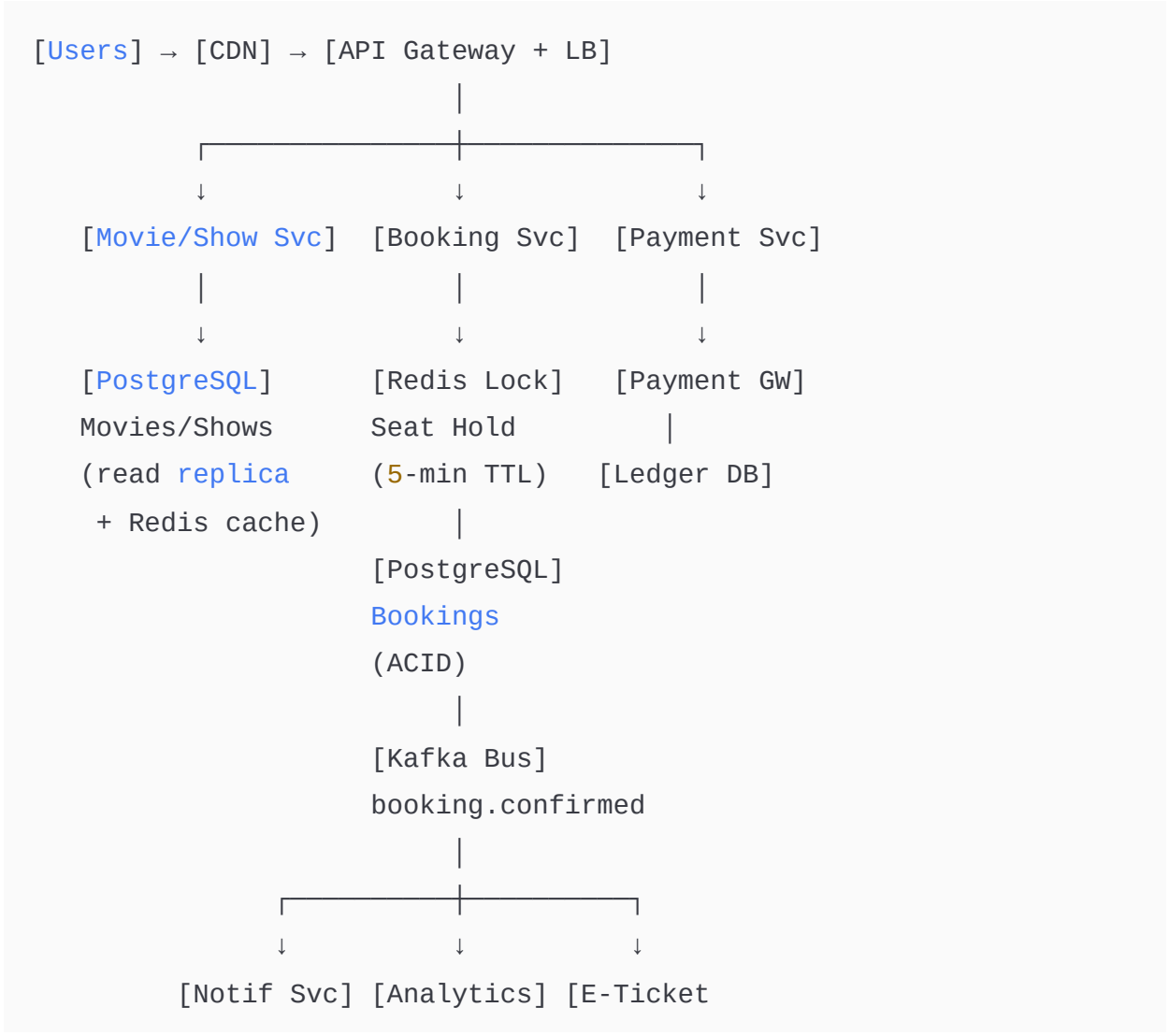
Concurrency: 100,000 users simultaneously hitting the same show's booking page. The architecture must handle this without the DB collapsing.

API Design

```
# Discovery
GET  /v1/movies?city={}&date={}          → Movie listings
GET  /v1/movies/{movie_id}               → Movie details + available
shows
GET  /v1/shows?movie_id={}&date={}       → Shows for a movie
GET  /v1/shows/{show_id}/seats           → Seat map with
availability

# Booking Flow
POST /v1/bookings/hold                   → Temporarily hold selected
seats (5 min)
Body: { show_id, seat_ids: ["A1","A2"], user_id }
Response: { hold_id, expires_at, seats_held }

POST /v1/bookings                        → Confirm booking + payment
Body: { hold_id, payment_method_id, promo_code }
Response: { booking_id, e_ticket_url, qr_code }

GET  /v1/bookings/{booking_id}           → Booking details
```

```
POST /v1/bookings/{booking_id}/cancel    → Cancel booking

# Venue Partner APIs
PUT  /v1/shows/{show_id}/seats/bulk      → Bulk update seat
availability (cinema API)
POST /v1/shows                           → Add new show

# Notifications
POST /v1/notifications/reminders         → Schedule pre-show
reminder
```

HLD



```
          Email/SMS/    ClickHouse  Generator]
          Push                      S3 + CDN
```

Flow Explanation

Seat map load: User selects a show → `GET /v1/shows/{show_id}/seats` → Show Service reads seat availability from Redis (cached, refreshed every 2 seconds from PostgreSQL). Seats are colour-coded: available (green), held (yellow — someone is in the process of booking), booked (red). Under massive concurrent load, Redis serves the seat map — PostgreSQL is not hit directly.

Seat hold (the critical path): User selects seats A1, A2 → `POST /v1/bookings/hold` → Booking Service runs a Lua script on Redis atomically:

```
for each seat_id:
  if EXISTS seat:{show_id}:{seat_id} then RETURN "already_held"
  else SET seat:{show_id}:{seat_id} {user_id} EX 300
```

This is atomic (single Redis operation) — no two users can hold the same seat simultaneously. Returns `hold_id`. If user abandons, TTL expires automatically and seat returns to available.

Booking confirmation: User submits payment → Booking Service acquires the hold → calls Payment Service → on payment success: writes booking record to PostgreSQL with SERIALIZABLE isolation (prevents race conditions) → inserts seat records as BOOKED → releases Redis hold (now DB is source of truth) → publishes `booking.confirmed` to Kafka → E-ticket Generator creates QR code → uploads to S3 → sends URL to Notification Service → user receives email/SMS confirmation.

Flash sale scenario (100K concurrent users, same show): All users hit Redis for the seat map simultaneously — Redis handles 500K reads/second, no problem. When they all try to hold the same seats — only the first user per seat wins the Redis atomic lock. Remaining users immediately get "seat unavailable" without ever hitting the DB. The DB is only written for successful bookings — typically a small fraction of total traffic.

Detailed Component Design

Seat Inventory — The core problem

The hardest challenge: preventing overselling under concurrent load. Three-layer defence:

Layer 1 — Redis atomic SETNX (hold phase): First line of defence. Atomic set-if-not-exists prevents concurrent holds. Under 1ms.

Layer 2 — PostgreSQL optimistic locking (booking phase): When writing the booking, check `WHERE status = 'AVAILABLE'` in the UPDATE statement. If another transaction already booked the seat, the row count returned is 0 → transaction rolled back → user informed.

Layer 3 — Unique constraint (database guarantee): `UNIQUE (show_id, seat_id)` on the booking table. Even if both application layers fail, the DB constraint prevents a duplicate insert. This is the final safety net.

Tradeoff: Pessimistic locking (SELECT FOR UPDATE) is simpler but creates a lock queue — 100K concurrent users all waiting on the same row lock would cause a thundering herd. Optimistic locking with Redis pre-filtering handles 99.9% of conflicts at the Redis layer, leaving only a tiny fraction to reach the DB.

Show Service — Read scalability

Movie listings and show schedules are read millions of times per day but change infrequently. Architecture: PostgreSQL as source of truth, Redis cache with 5-minute TTL for show details, Elasticsearch for search (movie by name, genre, language, city). CDN caches the movie poster images and trailers. Cache invalidation is event-driven via Kafka — when a cinema partner updates a show, `show.updated` event triggers cache eviction.

Queue for Peak Load

During a blockbuster release, the booking endpoint receives 100K concurrent requests. Instead of letting all of them hammer the system, a virtual queue is implemented. Users who arrive simultaneously are given a queue position (Redis INCR on a counter). Workers process them in order from the queue. Users see a "You are #4523 in queue — estimated wait: 2 minutes" message. This smooths out the spike, protects the DB, and prevents timeouts.

E-Ticket Generation

After successful booking, `booking.confirmed` event triggers the E-ticket Service. Generates a unique QR code containing `booking_id + show_id + seat_ids` digitally signed with HMAC-SHA256 (prevents forgery). Rendered as a PNG using a QR library, uploaded to S3, CDN URL returned to user. QR code is scanned at the venue — the venue's scanning system calls `GET /v1/bookings/{booking_id}/validate` to confirm validity.

Complete Technology Summary Across All 6 Systems

System	Message Bus	Primary DB	Cache	Search	Special
Kafka	N/A (is the bus)	Segment files on disk	Page cache (OS)	N/A	Zero-copy, ISR replication
YouTube/Netflix	Kafka	PostgreSQL + Cassandra	Redis + CDN	Elasticsearch	HLS/DASH, ABR, transcoding pipeline
Uber	Kafka	PostgreSQL	Redis Geo	N/A	TimescaleDB, WS location, surge pricing
Distributed Cache	N/A	RAM (Redis)	IS the cache	N/A	Consistent hashing, AOF, LRU eviction
Payment	Kafka	PostgreSQL (ACID)	Redis (dedup)	N/A	Double-entry ledger, Saga, PCI-DSS
BookMyShow	Kafka	PostgreSQL	Redis (seat lock)	Elasticsearch	SETNX hold, virtual queue, QR e-ticket

5/7/2026, 11:29:17 AM

Design E-Commerce System Design Notification System Design Banking System Design Library Management System Design Online Shopping Cart Design Hospital Management System Design Healthcare Claims Processing mention below these things in all system design functional requirements non - functional requirements (scalability, availibility, low latency & peformance, Reliablility etc..etc..) API design HLD Flow explaination (how the request flows will go) Detailed Component Design (Explain how they work, how they scale, discuss tradeoffs) add some followup expected Q&A

5/7/2026, 11:30:01 AM

1. E-COMMERCE SYSTEM

Functional Requirements

- User registration, login, profile management
- Product catalogue — browse, search, filter by category/price/brand/rating
- Product detail pages with images, reviews, Q&A
- Shopping cart — add, update, remove items
- Wishlist / save for later
- Checkout — address, payment method, order summary
- Multiple payment methods: card, UPI, wallet, COD, EMI
- Order management — place, track, cancel, return
- Inventory management — real-time stock updates
- Seller portal — list products, manage inventory, view orders
- Promotions — coupons, flash sales, referral codes
- Reviews and ratings
- Notifications — order updates, offers, shipping alerts
- Recommendation engine — personalised product suggestions

Non-Functional Requirements

Scalability: 100 million DAU, 500 million product catalogue. Each component scales independently. Read path (product browse, search) scales horizontally via read replicas and CDN. Write path (orders, inventory) uses DB sharding by seller_id and order_id. Flash sales generate 100x normal traffic in seconds — auto-scaling groups spin up within 90 seconds.

Availability: 99.99% uptime. Multi-AZ deployment for all services. No single point of failure. During a flash sale, the inventory service is the most critical — it runs in Active-Active mode across 2 regions.

Low Latency & Performance: Product page load under 200ms. Search results under 100ms. Order placement under 2 seconds. Cart operations under 50ms. CDN serves all product images and static assets globally under 30ms.

Reliability: Orders are never lost — written to Kafka before DB commit. Idempotency keys on all payment and order APIs. Saga pattern for distributed transactions across Order, Inventory, and Payment services.

Consistency: Strong consistency for inventory (never oversell), order state, and payment. Eventual consistency for product reviews, view counts, and recommendation scores.

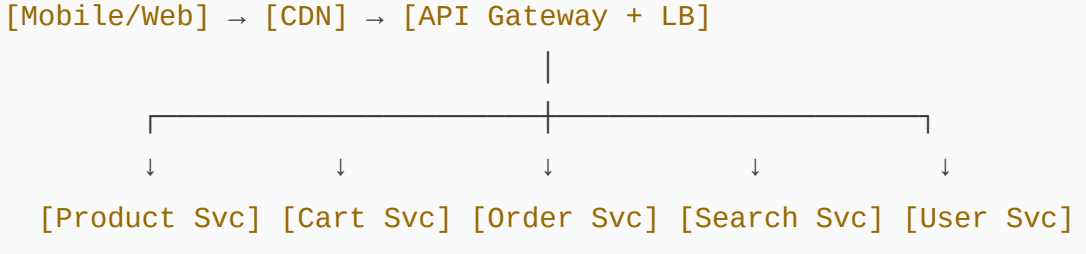
Security: PCI-DSS for card data. All card numbers tokenised. mTLS between internal services. Rate limiting on all public APIs. CAPTCHA on checkout during flash sales.

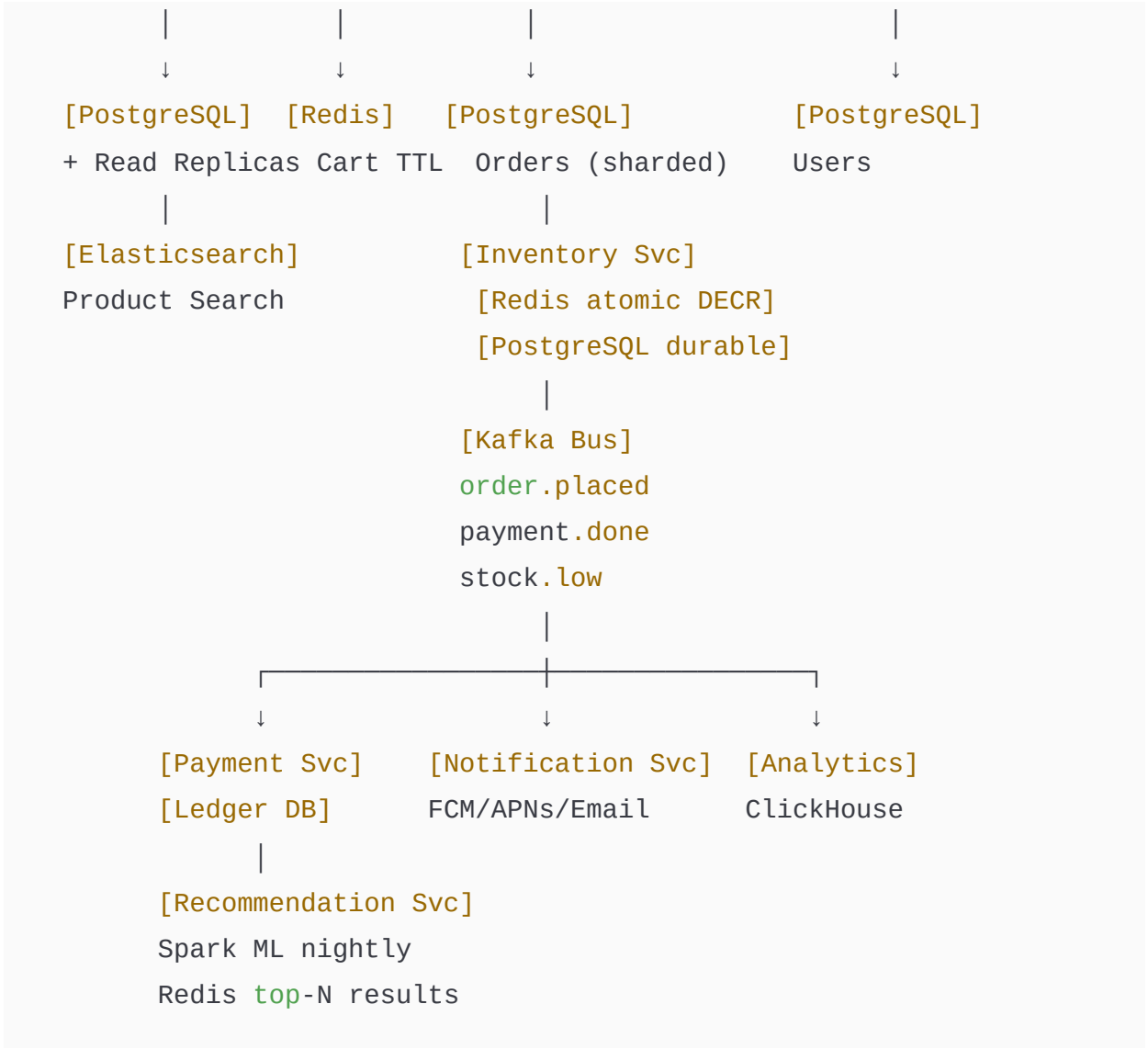
API Design

```
# Catalogue
GET  /v1/products?category={}&q={}&price_min={}&price_max={}&sort={}&page={}
GET  /v1/products/{product_id}
GET  /v1/products/{product_id}/reviews
GET  /v1/categories

# Cart
POST   /v1/cart/items                    → Add item
```

```
PUT    /v1/cart/items/{item_id}          → Update quantity
DELETE /v1/cart/items/{item_id}          → Remove item
GET    /v1/cart                          → View cart with live
prices
POST   /v1/cart/validate                 → Validate stock before
checkout

# Orders
POST  /v1/orders                         → Place order (idempotency-
key required)
GET   /v1/orders/{order_id}              → Order details + status
GET   /v1/orders                         → Order history
POST  /v1/orders/{order_id}/cancel       → Cancel order
POST  /v1/orders/{order_id}/return       → Initiate return

# Inventory (Seller/Internal)
PUT   /v1/inventory/{sku_id}             → Update stock
GET   /v1/inventory/{sku_id}             → Stock level

# Promotions
POST  /v1/coupons/validate               → Validate coupon code
GET   /v1/promotions/active              → Active flash sales

# Recommendations
GET   /v1/recommendations?user_id={}     → Personalised suggestions
GET   /v1/products/{id}/related          → Related products
```

HLD





Flow Explanation

Browse & Search flow: User searches "iPhone 15" → API Gateway → Search Service → Elasticsearch (BM25 relevance + filter + personalisation boost) → returns product IDs with scores → Product Service fetches metadata from Redis cache (5-min TTL over PostgreSQL read replica) → CDN serves product images → response assembled under 100ms.

Add to Cart flow: User clicks "Add to Cart" → Cart Service reads current cart from Redis (key: `cart:{user_id}`) → checks soft inventory availability (Redis counter) → adds item with quantity to Redis hash → returns updated cart total. Cart TTL = 30 days. Cart is Redis-first — PostgreSQL only written on checkout (performance optimisation).

Order placement flow: User hits "Place Order" → Cart Service calls Inventory Service to soft-reserve stock (Redis DECR) → calls Payment Service → on success: Order Service writes order to PostgreSQL → publishes `order.placed` to Kafka → Inventory Service converts soft reserve to hard deduct → Notification Service sends confirmation → Warehouse Management System receives pick-pack-ship event.

Flash sale flow: 100K concurrent users all buying the same item at 12:00 PM. A virtual queue (Redis INCR counter) serialises requests. Inventory reservation done atomically via Redis `DECR` — only the first N users (N = stock count) succeed. Remaining users immediately see "Sold Out" without touching the DB. No overselling possible.

Detailed Component Design

Inventory Service

The most critical component. Two-layer architecture: Redis atomic DECR for real-time stock (prevents overselling), PostgreSQL for durable ground truth. Soft reserve on cart add (holds stock for 15 minutes), hard deduct on order confirm. Background reconciliation job syncs Redis and PostgreSQL every minute to catch discrepancies. Low-stock events published to Kafka trigger seller notifications and reorder workflows.

Scaling: Redis Cluster with hash slot per SKU. At 10M SKUs with 100 updates/second each = 1B ops/second — distribute across 100 Redis nodes. Each node handles 10M ops/second.

Tradeoff: Redis failure means inventory data is temporarily unavailable. Mitigation: Redis AOF persistence + fallback to PostgreSQL read replica with slightly higher latency (20ms vs 1ms).

Product Catalogue & Search

Elasticsearch index with 500M products, each document containing: name, description, category path, brand, price, rating, images. Custom scoring function: text relevance × popularity score × personalisation signal × recency. Elasticsearch updated via Kafka CDC consumer (under 5 seconds from DB write to search index). Category tree stored in Redis (rarely changes, needs fast hierarchical reads).

Tradeoff: Elasticsearch eventual consistency means a newly listed product may take 5 seconds to appear in search. Acceptable — sellers don't expect instant indexing.

Recommendation Engine

Collaborative filtering (ALS) trained weekly on user-item interaction matrix (views, purchases, cart adds, wishlist adds). Stored in S3. Top-100 recommendations per user written to Redis (TTL 24 hours). Real-time signals (current session behaviour) blended with batch recommendations by the serving layer. Cold start for new users: popular products in their city and category.

Follow-up Expected Q&A

Q: How do you handle flash sales without overselling? A: Three-layer defence: Redis atomic DECR (first line — catches 99.9% of conflicts at memory speed), PostgreSQL optimistic locking with `version` column (second line), unique constraint on (order_id, sku_id) as final safety net. Virtual queue smooths the spike.

Q: How do you handle cart abandonment? A: Cart stored in Redis with 30-day TTL. A background job detects carts inactive for 1 hour and publishes `cart.abandoned` event to Kafka. Email/push trigger after 1 hour, 24 hours, 72 hours. Soft inventory reservation released after 15 minutes to free stock for other users.

Q: How does order search work for millions of orders? A: Orders table sharded by `user_id % N` in PostgreSQL. User always queries their own orders — single shard hit. Cross-user queries (admin, fraud analysis) go to a read replica or ClickHouse analytics DB. Order history paginated using cursor-based pagination (not offset) for consistent performance regardless of page depth.

Q: What happens if payment succeeds but the order write fails? A: The Saga pattern handles this. Order Service is the Saga orchestrator. If DB write fails after payment: a compensating transaction issues a refund via Payment Service. The `payment_id` and `idempotency_key` ensure the refund is processed exactly once. The failure is logged to a dead letter queue for manual review.

Q: How do you scale the review system? A: Reviews written to Cassandra (append-heavy, time-series, product_id as partition key). Average rating maintained as a Redis counter (INCR numerator and denominator, compute ratio). Full-text review search indexed in Elasticsearch. Spam detection runs asynchronously via Kafka consumer.

---

2. NOTIFICATION SYSTEM

Functional Requirements

- Send notifications via push (FCM/APNs), email (SMTP), SMS (Twilio), in-app, WhatsApp
- Support notification types: transactional (OTP, payment), marketing (offers), alerts (fraud, system)
- User preference management — opt-out per channel per category
- Scheduled notifications (send at 9AM user's local time)
- Batch / bulk notifications (send to 10M users for a sale)
- Template management — reusable notification templates with variable substitution
- Delivery status tracking — sent, delivered, failed, read
- Retry failed notifications with exponential backoff
- Rate limiting — prevent notification spam per user
- Priority queues — OTP/fraud alerts bypass all other queues
- Analytics — open rates, click rates, delivery rates

Non-Functional Requirements

Scalability: Send 1 billion notifications per day (11,500/second average, 100K/second peak for flash sale announcements). Each channel scales independently. Email worker pool scales separately from SMS pool. Kafka partitions per channel allow linear throughput scaling.

Availability: 99.99% for transactional (OTP, payment alerts). 99.9% acceptable for marketing. If the SMS provider is down, fall back to email. Multi-provider strategy — primary and backup provider per channel.

Low Latency & Performance: OTP delivery under 3 seconds end-to-end. Push notifications under 1 second for online users. Bulk marketing campaign to 10M users initiated within 60 seconds of trigger.

Reliability: At-least-once delivery — Kafka guarantees no message loss. Idempotency via `notification_id` deduplication in Redis — processing the same notification twice sends it only once. Dead Letter Queue for permanently failed notifications.

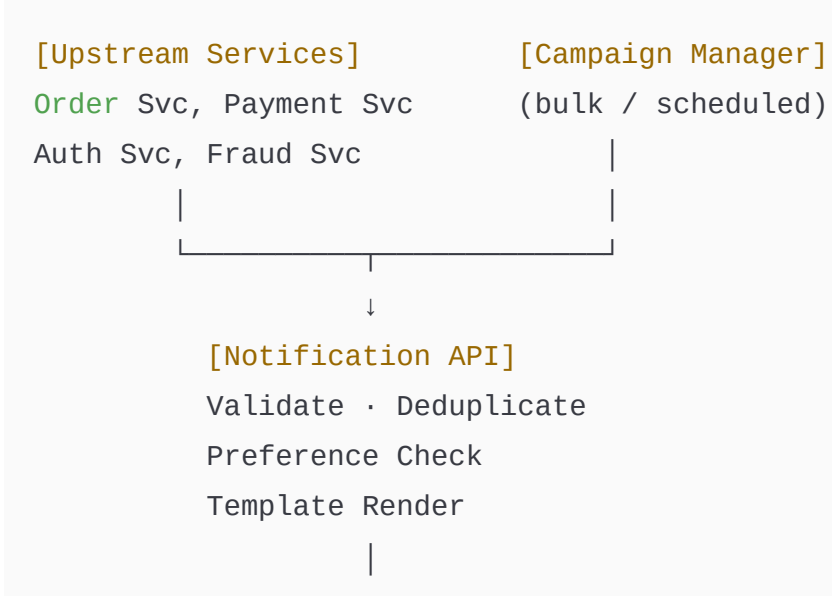
Consistency: Delivery status eventually consistent — status updated asynchronously from provider webhooks. The exact timestamp of "delivered" may lag by 1–5 seconds.

API Design

```
# Send Notification
POST /v1/notifications/send
Body: {
  user_id: "u123",
  type: "TRANSACTIONAL",
  channels: ["PUSH", "SMS"],      ← preferred channels (system
checks preferences)
  template_id: "order_confirmed",
  variables: { order_id: "ORD001", amount: "₹499" },
  priority: "HIGH",
  idempotency_key: "uuid"
}

# Bulk / Campaign
```

```
POST /v1/notifications/bulk
Body: {
  segment: { city: "Mumbai", age_min: 18 },
  template_id: "diwali_sale",
  schedule_at: "2024-10-30T09:00:00+05:30"
}

# Templates
POST /v1/templates                    → Create template
GET  /v1/templates/{template_id}      → Get template
PUT  /v1/templates/{template_id}      → Update template

# Preferences
GET  /v1/users/{user_id}/preferences      → Get notification
preferences
PUT  /v1/users/{user_id}/preferences      → Update preferences

# Status
GET  /v1/notifications/{notification_id}/status
GET  /v1/notifications?user_id={}&page={}  → Notification history
```

HLD



```
            [Priority Kafka Topics]
            HIGH   → order.notif.high
            MED    → order.notif.medium
            LOW    → order.notif.low (marketing)
                       |
      ┌────────────────┼────────────────┐
      ↓                ↓                ↓
[Push Worker] [SMS Worker] [Email Worker]
 FCM/APNs       Twilio/AWS     SendGrid
      |           SNS             SES
      ↓              ↓                ↓
[Status DB]  ← Provider Webhooks → [Analytics]
PostgreSQL                        ClickHouse
      |
[Retry Queue]
DLQ → retry up to 3x
```

Flow Explanation

Transactional notification flow (OTP): Auth Service calls `POST /v1/notifications/send` with `priority: HIGH` and `type: OTP` → Notification API validates the request → checks Redis for `notification_id` deduplication (prevents duplicate OTP sends on retry) → checks user preferences (did user opt out of SMS? If yes, fall back to email) → renders the template (`Your OTP is {{otp}}`) with variable substitution → publishes to HIGH priority Kafka topic → SMS Worker consumes immediately (no batching on high-priority queue) → calls Twilio API → Twilio returns delivery status → Worker writes status to PostgreSQL → webhook from Twilio confirms delivery → status updated.

Marketing bulk campaign flow: Campaign Manager receives trigger "Send Diwali Sale to all users in Mumbai" → queries user segment from PostgreSQL (or ClickHouse for large segments) → splits into batches of 10,000 users → publishes each batch to LOW priority Kafka topic → Email Workers consume batches → render personalised templates (user's name, past purchase category) → call SendGrid batch API → track delivery via webhooks. Rate limiter enforces max 3 marketing notifications per user per day (Redis counter per user per day).

Retry flow: Worker calls Twilio → Twilio returns 500 → Worker does not commit Kafka offset → message retried with exponential backoff (1s, 2s, 4s). After 3 retries → message moved to Dead Letter Queue → monitoring alert fired → team investigates. If Twilio is completely down → circuit breaker opens → SMS Worker automatically switches to backup provider (AWS SNS).

Detailed Component Design

Priority Queue Architecture

Three Kafka topics per channel based on priority. HIGH (OTP, fraud alerts, payment failures) — consumed immediately, no batching, dedicated worker pool. MEDIUM (order confirmations, shipping updates) — small batches of 100, 500ms batch window. LOW (marketing, newsletters) — large batches of 10,000, rate-limited to 10K/second globally. This ensures an OTP is never delayed behind a marketing email campaign.

Tradeoff: More topics = more Kafka partitions = more consumer coordination overhead. Balance: 3 priority levels is sufficient for most use cases. Adding more priorities reduces the benefit of each level.

Preference Service

User preferences stored in PostgreSQL (source of truth) and cached in Redis (TTL 1 hour). Schema: `{user_id, channel, category, opted_in, updated_at}`. Checked on every notification send. Preference cache updated immediately on opt-out (critical — sending a marketing email after opt-out is a GDPR violation). Global opt-out (DND: Do Not Disturb 10PM–8AM) enforced by the scheduling layer — notifications scheduled for delivery within the user's active hours.

Template Engine

Templates stored in PostgreSQL with Handlebars/Mustache syntax. Compiled and cached in Redis (template rarely changes). Variable substitution done in-memory at the worker — no DB call during rendering. Multi-language support: template has variants per locale (`en`, `hi`, `ta`). User's preferred language fetched from Profile Service (Redis cached).

Delivery Status Tracking

Each notification has a state machine: `QUEUED → SENT → DELIVERED → READ / FAILED`. Provider webhooks update the state. Status stored in PostgreSQL with an index on `notification_id`. Analytics events published to Kafka → ClickHouse for open rate, click rate, delivery rate dashboards. Redis used for real-time dashboard metrics (INCR counters per campaign per status).

Follow-up Expected Q&A

Q: How do you prevent notification spam? A: Redis rate limiter per user per channel per category: `INCR notif:{user_id}:{channel}:{category}:{date}` with daily TTL. If counter exceeds limit (e.g. 3 marketing push/day), notification is dropped and logged. Global campaign frequency capping enforced at Campaign Manager before publishing to Kafka.

Q: How do you handle 10M users for a bulk campaign without overwhelming the DB? A: User segment query runs against ClickHouse (OLAP, handles large scans efficiently). Results streamed in batches of 10,000 users to Kafka. Workers process batches in parallel. The DB is never hit during sending — only during segment computation. Sending rate is throttled to respect provider API limits (SendGrid: 1M emails/hour).

Q: What if FCM/APNs rejects a push token (user uninstalled the app)? A: Provider returns a `404 Not Registered` error. Worker catches this and publishes a `device.token.invalid` event to Kafka → Profile Service removes the token → future notifications for this user skip push and fall back to email/SMS. Token validation runs before sending to avoid wasted API calls.

Q: How do you guarantee exactly-once delivery for OTP? A: True exactly-once delivery is impossible with external providers (network issues can cause unknown states). Best practice: at-least-once with idempotency. `notification_id` dedup in Redis prevents double-send from our side. Twilio's own deduplication handles provider-side retries. The OTP itself has a 5-minute expiry — even if delivered twice, only one use is possible.

Q: How do you handle timezone scheduling? A: User's timezone stored in Profile Service. Scheduled notifications store target local time (`09:00 AM`). Scheduler Service runs every minute and queries notifications due in the next 5 minutes for each timezone. Converts to

UTC, publishes to Kafka with `deliver_after` timestamp. Workers check `deliver_after` before sending — if not yet due, the message is re-queued with a delay.

---

3. BANKING SYSTEM

Functional Requirements

- User KYC registration and account creation
- Account types: savings, current, fixed deposit, recurring deposit
- Core transactions: deposit, withdrawal, fund transfer (NEFT, IMPS, RTGS, UPI)
- Account balance inquiry (real-time)
- Transaction history with filters and download
- Debit/credit cards — issue, block, set PIN, set limits
- Loans — apply, disburse, EMI schedule, repayment
- Fixed deposit creation and premature closure
- Standing orders and scheduled payments
- Multi-factor authentication (OTP + biometric)
- Fraud detection on every transaction
- Regulatory reporting (RBI, SEBI compliance)
- Notifications for every debit/credit

Non-Functional Requirements

Scalability: Process 10,000 transactions per second at peak. Core banking scales horizontally for read-heavy workloads (balance inquiry, statement download). Write path (actual money movement) is intentionally limited to ensure consistency over throughput.

Availability: 99.999% (five nines) for core transaction processing. Less than 5 minutes downtime per year. Active-Active across two data centres with synchronous replication (RPO = 0).

Low Latency & Performance: IMPS transfers complete within 30 seconds (RBI mandate). UPI within 10 seconds. Balance inquiry under 100ms. Card authorisation under 500ms (card network SLA).

Reliability: Zero data loss. Every transaction written to WAL before acknowledging. Exactly-once transaction processing — the same transaction processed multiple times produces exactly one debit/credit. Full audit trail — every state change immutably logged.

Consistency: Strong consistency for all money movement. ACID transactions. No eventual consistency — a bank cannot say "your transfer might have gone through." Serializable isolation for account debits.

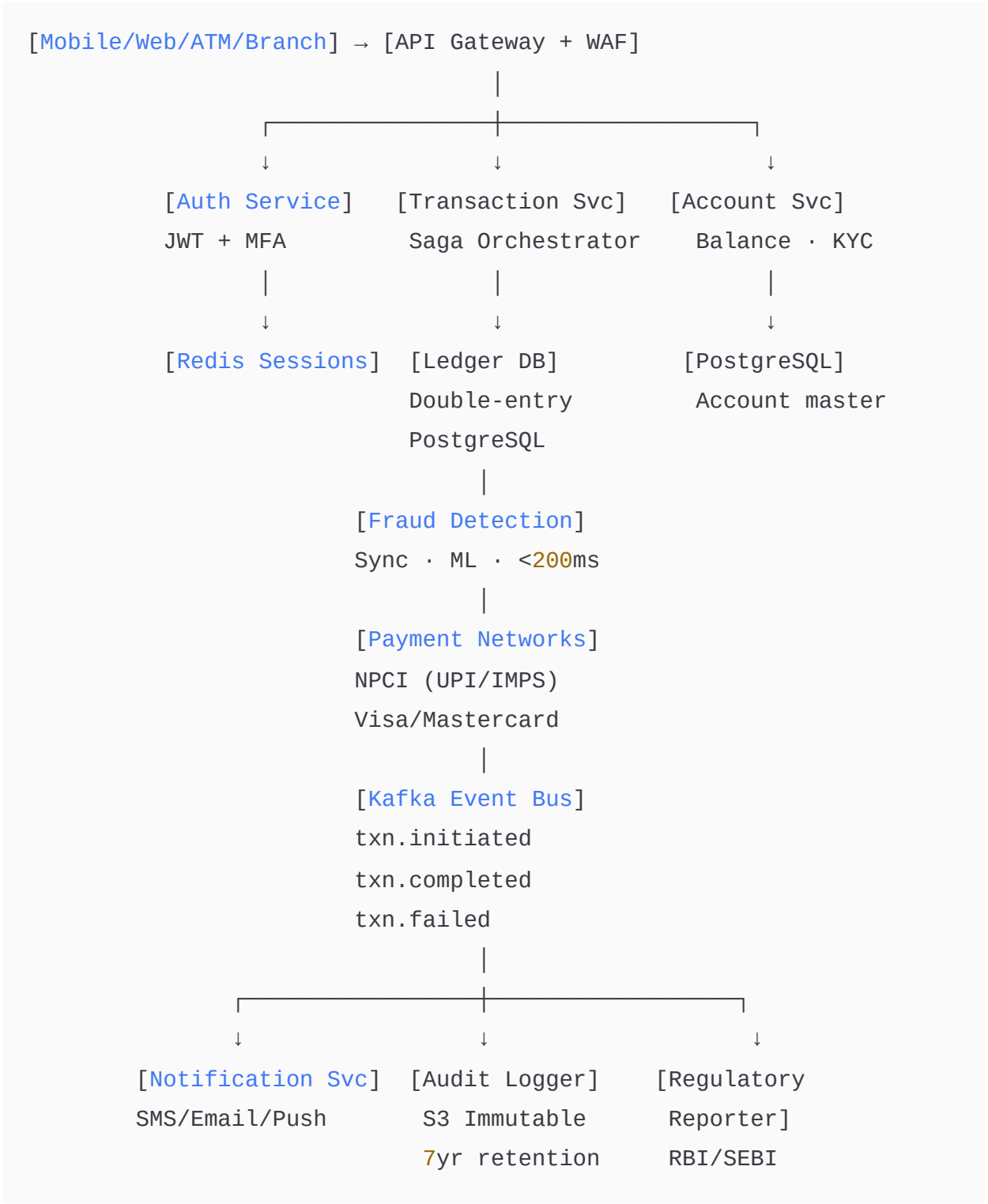
Security: RBI-mandated encryption. PCI-DSS Level 1 for card data. 4-eyes principle for large transactions (two officers must approve transfers over ₹10 crore). All logs retained for 7 years. Immutable audit log in S3.

API Design

```
# Accounts
POST   /v1/accounts                          → Create account (post KYC)
GET    /v1/accounts/{account_id}             → Account details + balance
GET    /v1/accounts/{account_id}/statement → Transaction history

# Transactions
POST   /v1/transactions/transfer             → Fund transfer
Body: {
  from_account: "ACC001",
  to_account: "ACC002",
```

```
  amount: 5000,
  currency: "INR",
  mode: "IMPS",
  idempotency_key: "uuid",
  remarks: "Rent payment"
}
Response: { transaction_id, status, reference_number }

POST  /v1/transactions/deposit           → Cash deposit
POST  /v1/transactions/withdrawal        → Cash withdrawal

# UPI
POST  /v1/upi/pay                        → UPI payment
POST  /v1/upi/collect                    → Collect request

# Cards
POST  /v1/cards                          → Issue card
PUT   /v1/cards/{card_id}/block          → Block card
PUT   /v1/cards/{card_id}/pin            → Set/change PIN
PUT   /v1/cards/{card_id}/limits         → Set transaction limits

# Loans
POST  /v1/loans/apply                    → Loan application
GET   /v1/loans/{loan_id}/schedule       → EMI schedule
POST  /v1/loans/{loan_id}/repay          → Repay EMI

# Auth
POST  /v1/auth/login                     → Login (returns JWT)
POST  /v1/auth/otp/send                  → Send OTP
POST  /v1/auth/otp/verify                → Verify OTP
```

HLD



Flow Explanation

Fund transfer (IMPS) flow: User initiates ₹5,000 IMPS transfer → API Gateway validates JWT + OTP → Transaction Service receives request with idempotency key → checks Redis for

duplicate (prevent double-transfer on retry) → calls Fraud Detection Service synchronously (under 200ms) → if score < threshold: begin Saga. Saga Step 1: debit sender account (PostgreSQL `UPDATE accounts SET balance = balance - 5000 WHERE account_id = 'ACC001' AND balance >= 5000` — atomic deduct with balance check). Saga Step 2: send message to NPCI via IMPS gateway. Saga Step 3: credit receiver account. Each step has a compensating transaction: if Step 3 fails after Step 1 and 2 succeed → reverse debit + notify NPCI to reverse credit.

Double-entry ledger write: Every transaction creates exactly two immutable ledger rows: `DEBIT sender 5000` and `CREDIT receiver 5000`. Balance = SUM of all credits - SUM of all debits for that account. No balance field is ever mutated directly — only ledger entries are appended. This makes the database an immutable financial record — any historical balance can be computed at any point in time.

Card authorisation flow: Customer swipes card at POS → merchant terminal sends authorisation request to Visa → Visa calls our card authorisation API → API Gateway → Card Service → checks card status (not blocked), available balance, daily limit, merchant category restrictions → calls Fraud Detection → approves/declines → responds to Visa within 500ms → Visa responds to merchant terminal.

Detailed Component Design

Transaction Service and Saga Pattern

The Saga pattern manages distributed transactions without a distributed lock. Each Saga step is a local ACID transaction. If a step fails, compensating transactions are run in reverse order. The Saga state is persisted to PostgreSQL — if the orchestrator crashes mid-saga, it resumes from the last completed step on restart. This gives exactly-once semantics: each step checks if it has already been applied (idempotent check) before executing.

Tradeoff: Two-Phase Commit (2PC) would give stronger ACID guarantees but cannot tolerate coordinator failure — a blocked coordinator locks all participants indefinitely. Saga allows failure and compensation at the cost of potential temporary inconsistency (money debited but not yet credited during the saga). Mitigated by: very fast saga completion (under 5 seconds) and clear compensating transactions.

Ledger Database Design

```
accounts:     account_id, user_id, account_type, currency,
created_at
ledger_entries: entry_id, account_id, transaction_id, amount,
                type(DEBIT/CREDIT), balance_after, created_at
transactions: transaction_id, from_account, to_account, amount,
              mode, status, idempotency_key, created_at
```

`balance_after` is denormalised into each ledger entry for fast balance lookup (read last entry for account). But the true balance is always computable from scratch as SUM of entries — `balance_after` is a performance optimisation only.

Fraud Detection

Feature vector computed at transaction time from Redis (pre-computed user features, updated by background jobs): transaction velocity (how many transactions in last 1 hour), average transaction amount, device fingerprint match, geolocation (is this location consistent with recent transactions?), merchant category risk score. XGBoost model served by TensorFlow Serving (GPU-accelerated). Latency target: p99 under 150ms. High-risk transactions trigger 3D Secure / additional OTP. Blocked transactions published to Kafka for analyst review.

Audit Logging

Every action that modifies financial data — transaction creation, status change, balance update, account modification — is written to an append-only audit log table in PostgreSQL and streamed to S3 (immutable, write-once buckets enabled via S3 Object Lock). S3 Glacier archive after 1 year. Query interface via Athena. Retained for 7 years as per RBI requirement.

Follow-up Expected Q&A

Q: How do you ensure zero data loss in a bank? A: PostgreSQL WAL (Write-Ahead Log) — every transaction is written to disk before acknowledging. Synchronous replication to

standby — primary waits for standby acknowledgment before returning success. RPO = 0 (zero data loss). RTO = 30 seconds (automated failover). Cross-region replication via PostgreSQL logical replication for disaster recovery.

Q: How do you handle the case where money is debited but the transfer fails? A: The Saga compensating transaction reverses the debit. The debit entry is immutable in the ledger — a new credit entry is created to reverse it (audit trail shows both). If the compensating transaction itself fails, the failure is logged to a reconciliation queue. A reconciliation job runs every 30 minutes comparing ledger balances with expected state and flags discrepancies for manual review.

Q: How do you handle concurrent withdrawals from the same account (e.g. ATM + mobile simultaneously)? A: PostgreSQL row-level locking with `SELECT FOR UPDATE SKIP LOCKED`. The first transaction locks the account row, applies the debit, and releases. The second transaction either queues behind the lock or is rejected if balance is insufficient. Serializable isolation prevents phantom reads. Combined with an application-level Redis lock (`SETNX account:{id}:lock`) for fast rejection before hitting the DB.

Q: How does UPI work in your system? A: User initiates UPI payment → UPI Service generates a payment request → calls NPCI's UPI API → NPCI routes to beneficiary's bank → beneficiary bank confirms → NPCI sends callback → Transaction Service updates status. NPCI timeout is 30 seconds. If timeout: Transaction Service marks status as `PENDING` and polls NPCI for final status. Never double-debit — idempotency key sent to NPCI prevents this.

---

4. LIBRARY MANAGEMENT SYSTEM

Functional Requirements

- Book catalogue management — add, update, search books by title, author, ISBN, genre
- Member registration and membership management

- Book borrowing — issue, return, renew
- Reservation / hold — reserve a book currently borrowed by someone else
- Fine calculation for overdue returns
- Multiple copies of the same book
- Digital books (ebooks, audiobooks) — online reading/download
- Librarian portal — manage catalogue, members, transactions
- Automated reminders — due date approaching, overdue
- Reports — most borrowed books, member activity, inventory status
- Inter-library loans — borrow from partner libraries
- Search across multiple libraries (consortium search)

Non-Functional Requirements

Scalability: 500,000 members, 1 million book catalogue, 10,000 concurrent borrowing transactions per day. Catalogue search scales via Elasticsearch. Transaction DB scales via read replicas.

Availability: 99.9% uptime. A library system is not life-critical — brief downtime is acceptable. Multi-AZ deployment for core services.

Low Latency & Performance: Book search under 100ms. Borrowing transaction under 1 second. Member lookup under 50ms.

Reliability: No book can be double-issued — distributed lock on `copy_id` during issue transaction. Fine calculations are deterministic and replayable from transaction history.

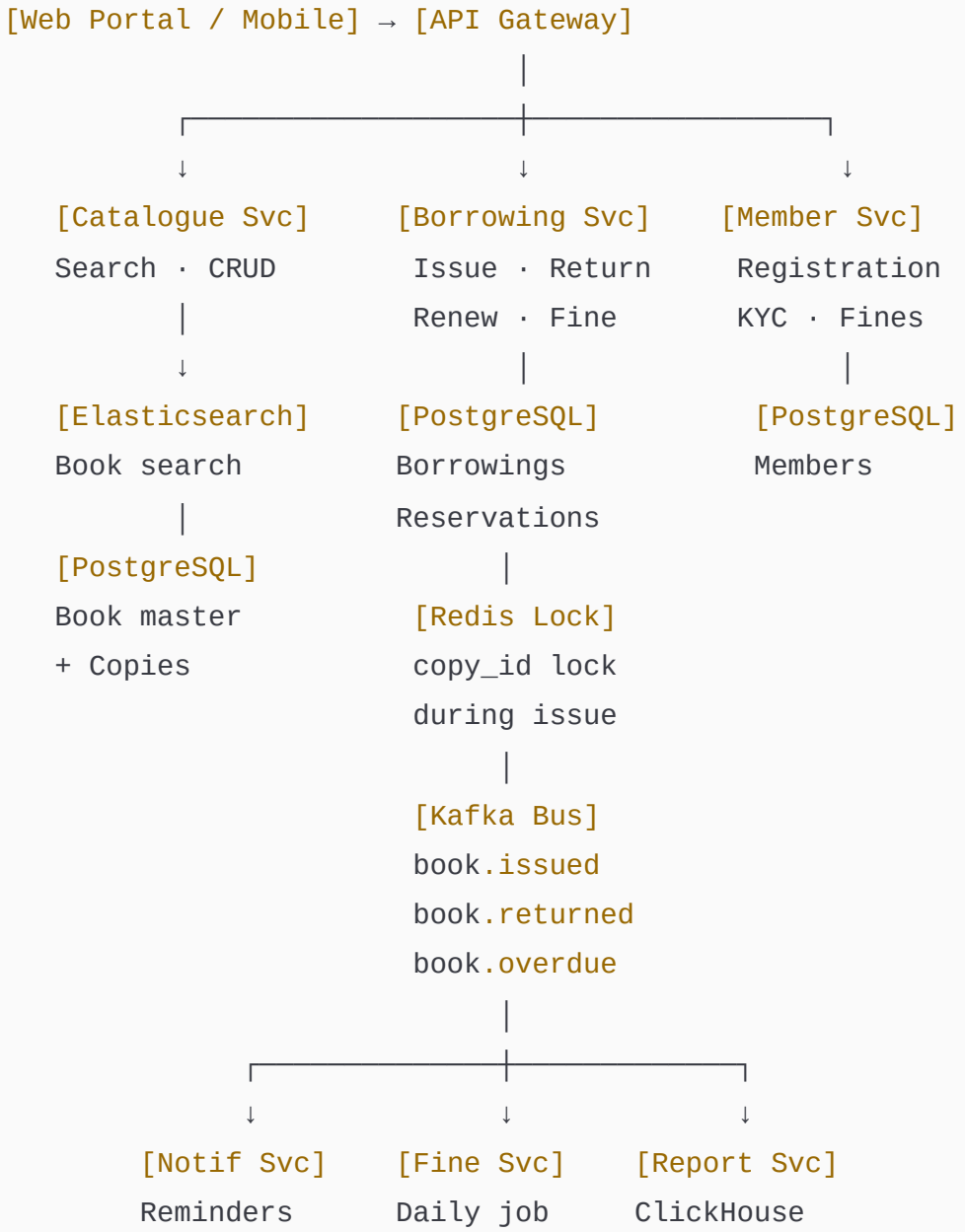
Consistency: Strong consistency for borrowing (a copy cannot be issued to two people). Eventual consistency for reservation queue order (slight delay acceptable).

API Design

```
# Catalogue
POST   /v1/books                          → Add book (librarian)
GET    /v1/books?q={}&author={}&genre={}  → Search catalogue
GET    /v1/books/{isbn}                   → Book detail with
availability
GET    /v1/books/{isbn}/copies            → All copies and their
status

# Borrowing
POST   /v1/borrowing/issue                → Issue book to member
Body: { member_id, copy_id, due_date }

POST   /v1/borrowing/return               → Return book
Body: { copy_id }
Response: { fine_amount, days_overdue }

POST   /v1/borrowing/renew                → Renew borrowed book
Body: { borrowing_id }
Response: { new_due_date }

GET    /v1/borrowing/history?member_id={} → Borrowing history

# Reservations
POST   /v1/reservations                   → Reserve a book (if all
copies borrowed)
Body: { member_id, isbn }
Response: { reservation_id, position_in_queue,
estimated_availability }

DELETE /v1/reservations/{id}              → Cancel reservation

# Members
POST   /v1/members                        → Register member
GET    /v1/members/{member_id}            → Member profile + active
loans
GET    /v1/members/{member_id}/fines      → Outstanding fines
POST   /v1/members/{member_id}/fines/pay  → Pay fine
```

```
# Reports (librarian)
GET   /v1/reports/popular-books          → Most borrowed
GET   /v1/reports/overdue                → All overdue borrowings
GET   /v1/reports/inventory              → Stock status
```

HLD



```
        Overdue         computes
        alerts          overdue fines
```

Flow Explanation

Book issue flow: Librarian scans member card and book barcode → `POST /v1/borrowing/issue` → Member Service validates member is active and has no outstanding fines over limit → Borrowing Service acquires Redis lock on `copy_id` (prevents concurrent issue of same copy) → checks copy status in PostgreSQL (`WHERE status = 'AVAILABLE'`) → writes borrowing record with due date → updates copy status to `BORROWED` → releases lock → publishes `book.issued` event to Kafka → Notification Service schedules a reminder 2 days before due date.

Book return and fine calculation: Librarian scans book → `POST /v1/borrowing/return` → Borrowing Service fetches borrowing record → calculates days overdue (if any) → computes fine (`overdue_days × fine_rate_per_day`) → updates borrowing status to `RETURNED` → updates copy status to `AVAILABLE` → checks reservation queue for this ISBN — if someone has a reservation: copy status set to `RESERVED` and reserved member notified → publishes `book.returned` event.

Reservation queue flow: All copies of "Clean Code" are borrowed → User reserves → Reservation Service adds entry to reservation queue (`ISBN: queue → [user1, user2, user3]` stored as Redis sorted set with timestamp as score + PostgreSQL for durability) → when a copy is returned → Reservation Service picks the next user from the queue → sends notification → holds the copy for 48 hours → if not picked up: notifies the next user in queue.

Detailed Component Design

Copy Management

A book (identified by ISBN) can have multiple physical copies (each with a unique `copy_id` / barcode). Schema: `books (isbn, title, author, publisher)`, `copies (copy_id, isbn, condition, location, status)`. Status state machine: `AVAILABLE → BORROWED → RETURNED / RESERVED → BORROWED`. The Redis lock on `copy_id` prevents two librarians at different desks from issuing the same copy simultaneously (race condition in a busy library).

Fine Calculation Engine

A scheduled job runs every midnight. Queries all borrowings where `due_date < today AND status = 'BORROWED'`. Computes fine as `(current_date - due_date) × per_day_rate`. Updates member's outstanding fine balance. Publishes `book.overdue` event for each overdue borrowing → Notification Service sends reminder. Fine waivers logged with librarian approval for audit trail.

Elasticsearch for Catalogue Search

Books indexed with fields: title, author, ISBN, genre, publisher, keywords, description. Full-text search with typo tolerance (fuzzy matching for misspelled author names). Faceted search: filter by genre, language, availability (available copies > 0). Availability filter is a challenge — Elasticsearch is eventually consistent. Solution: index available_copies count, updated via Kafka CDC. Stale count of ±1 is acceptable — actual availability confirmed at borrowing transaction time.

Follow-up Expected Q&A

Q: How do you handle the case where two librarians try to issue the same book simultaneously? A: Redis SETNX lock on `copy_id` before the DB transaction. First librarian acquires lock, issues book, releases lock. Second librarian gets "lock acquisition failed" and retries — at which point the copy is already BORROWED and the transaction fails cleanly. Lock TTL = 10 seconds to prevent orphaned locks if a librarian's terminal crashes mid-transaction.

Q: How do you compute the position in the reservation queue? A: Redis sorted set with `ISBN` as key, `timestamp_of_reservation` as score, `member_id` as value. `ZRANK` returns 0-indexed position. `ZRANGEBYSCORE` returns the next member to notify when a copy is

returned. Position is an estimate — someone ahead in the queue may cancel, so actual availability is communicated as a range.

Q: How would you extend this for a consortium of 50 libraries? A: Each library maintains its own instance. A federated search layer queries all libraries via an aggregation service. Inter-library loan (ILL) requests go through an ILL Service that creates a borrowing record at the lending library and tracks the transit. Consortium-wide Elasticsearch index aggregates all catalogues. Member cards can be virtual (QR code) and validated against a shared member registry.

---

5. ONLINE SHOPPING CART

Functional Requirements

- Add items to cart (from product catalogue)
- Update item quantity
- Remove items from cart
- View cart with real-time pricing and stock status
- Apply promo codes and coupons
- Calculate taxes and delivery charges by address
- Save cart across devices (logged-in users)
- Guest cart (session-based, convertible to account on login)
- Merge guest cart with account cart on login
- Save items for later / wishlist
- Cart expiry — items held for 30 days
- Price change notification — alert user if price changed since adding
- Checkout from cart

Non-Functional Requirements

Scalability: 100 million users, each with an average cart size of 3 items. 500 million cart operations per day. Cart reads vastly outnumber writes (users browse their cart often). Redis handles all hot cart operations.

Availability: 99.99%. If the cart service is down, users cannot buy — direct revenue impact. Redis Cluster ensures no single node failure causes a cart outage.

Low Latency & Performance: Cart read under 20ms. Cart add/update under 50ms. Cart validation (stock check + price refresh) under 200ms.

Reliability: Cart data durable in PostgreSQL as backup to Redis. If Redis fails, cart is rebuilt from PostgreSQL (slower but correct).

Consistency: Cart item prices refreshed from Product Service on every cart view (not cached in cart). Stock status checked at cart validation (before checkout) — not on every cart view (too expensive).

API Design

```
# Cart Operations
GET    /v1/cart                          → View cart (authenticated
or guest)
POST   /v1/cart/items                    → Add item
Body: { product_id, sku_id, quantity }

PUT    /v1/cart/items/{item_id}          → Update quantity
DELETE /v1/cart/items/{item_id}          → Remove item
DELETE /v1/cart                          → Clear cart

# Coupon
POST   /v1/cart/coupon                   → Apply coupon code
DELETE /v1/cart/coupon                   → Remove coupon
```

```
# Cart Validation
POST   /v1/cart/validate                    → Pre-checkout validation
Response: {
  valid: true/false,
  items: [{
    item_id, available: true,
    price_changed: false,
    current_price: 999
  }],
  total, tax, delivery_charge
}

# Save for Later
POST   /v1/cart/items/{item_id}/save-later  → Move to wishlist
POST   /v1/wishlist/items/{item_id}/add-to-cart → Move back to cart

# Guest → Account Merge
POST   /v1/cart/merge                       → Merge guest cart into
account cart
Body: { guest_session_id }
```

HLD

```
[Web/Mobile] → [API Gateway]
                      |
               [Cart Service]
               ┌──────┴──────┐
               ↓             ↓
          [Redis]       [PostgreSQL]
          Hot cart       Durable cart
          TTL 30 days    backup
               |
          [Product Svc]    [Inventory Svc]
```

```
          Price refresh    Stock check
          (on cart view)   (on validate)
               |
          [Promo/Coupon]   [Tax Svc]
          Validate code    Calculate GST
          discount calc    delivery fee
               |
          [Kafka Bus]
          cart.abandoned
          cart.checkout
               |
          [Notification Svc]
          Abandoned cart
          price drop alert
```

Flow Explanation

Add to cart (authenticated): `POST /v1/cart/items` with `{product_id: "P1", quantity: 2}` → Cart Service verifies product exists (Product Service cache lookup) → checks if item already in cart (`HGET cart:{user_id} {sku_id}`) → if yes: increment quantity (`HINCRBY`). If no: `HSET cart:{user_id} {sku_id} {quantity:2, added_at: ts}` → also writes to PostgreSQL asynchronously (Kafka event consumer) → returns updated cart.

Cart view flow: `GET /v1/cart` → Cart Service reads all items from Redis (`HGETALL cart:{user_id}`) → for each item: fetches current price from Product Service (Redis cache over DB, 5-min TTL) → calculates line total → checks if price changed since `added_at` (flags changed items) → applies coupon discount → calls Tax Service for GST calculation → calls Delivery Service for shipping estimate → returns fully enriched cart response under 50ms.

Guest cart merge on login: User has a guest cart (stored in Redis with key `cart:guest:{session_id}`) → logs in → `POST /v1/cart/merge` with `guest_session_id` → Cart Service reads guest cart items → for each item: if already in account cart: take the higher quantity. If not: add to account cart → deletes guest cart → publishes `cart.merged` event.

Pre-checkout validation: `POST /v1/cart/validate` → Cart Service calls Inventory Service for each SKU (checks Redis stock counter) → flags out-of-stock items → refreshes prices → calculates final total with tax and delivery → returns validation report. User sees: "2 items are available, 1 item (Nike Shoes L) is out of stock." User decides to remove out-of-stock item and proceed.

Detailed Component Design

Redis Cart Data Structure

Cart stored as a Redis Hash: `HSET cart:{user_id} {sku_id} {json_blob}`. JSON blob per item: `{quantity, added_at, saved_price}`. `saved_price` is stored at add time — used to detect price changes at view time (flag but do not block). `HGETALL` retrieves entire cart in one O(N) operation. Cart TTL managed with `EXPIRE cart:{user_id} 2592000` (30 days), reset on every access.

Guest cart: Same structure, key = `cart:guest:{session_id}`. Session ID is a UUID in a cookie (HttpOnly, Secure). TTL = 7 days.

Tradeoff: Redis Hash stores quantity and metadata but not current price — price is always fresh-fetched. This means the cart never shows stale prices, but requires N Product Service calls per cart view (N = number of items). Mitigated by: Product Service cache (Redis, 5-min TTL) returns prices in under 1ms, and a batch price-fetch API reduces N calls to 1 bulk call.

Coupon / Promo Service

Coupon rules stored in PostgreSQL: `{coupon_code, type(PERCENT/FIXED/BXGY), discount_value, min_order_value, applicable_categories, usage_limit, per_user_limit, expiry}`. Validation logic: check expiry → check min order value → check category restrictions → check usage count (Redis INCR for atomic count tracking) → check per-user usage (Redis key `coupon:{code}:user:{user_id}` with TTL = coupon expiry). Applied discount stored in cart Redis hash, re-validated on every checkout to prevent stale coupon application.

Price Change Detection

When an item is added to cart, `saved_price` is recorded. On every cart view, current price is fetched from Product Service. If `current_price ≠ saved_price`: the item is flagged with `price_changed: true, current_price: X, saved_price: Y`. A Kafka consumer also monitors `product.price_changed` events — for all users who have that SKU in their cart (queried from PostgreSQL cart backup), a push notification is sent: "Price of [product] in your cart dropped from ₹999 to ₹799!"

Follow-up Expected Q&A

Q: What happens to the cart if Redis goes down? A: Cart Service has a fallback to PostgreSQL. Redis writes are mirrored to PostgreSQL asynchronously via a Kafka consumer (cart.item.added events). On Redis failure, Cart Service detects the connection error (circuit breaker opens), reads from PostgreSQL (higher latency — 20ms vs 1ms, but functional). On Redis recovery, the cart is lazily repopulated from PostgreSQL on the next cart access.

Q: How do you handle the case where an item in the cart becomes out of stock? A: Stock status is NOT checked on every cart view (too expensive for 100M users). It is checked only at cart validation (pre-checkout). This is the industry standard — Amazon's cart does the same. If out of stock at checkout: user is informed and cannot proceed until they remove the item or wait for restocking. For popular items, a "Notify me" option subscribes the user to `stock.available` events.

Q: How do you handle a user with 500 items in their cart? A: `HGETALL` is O(N) — for N=500, this is still fast (Redis processes millions of operations per second). The bottleneck is price fetching — 500 Product Service calls. Solved by: bulk price fetch API `POST /v1/products/prices` with array of `sku_ids`. A single call returns all prices. Cart size is also soft-capped at 100 items in practice — above this a warning is shown.

---

6. HOSPITAL MANAGEMENT SYSTEM

Functional Requirements

- Patient registration and profile (demographics, medical history, insurance)
- Appointment booking — OPD, specialist, telemedicine
- Electronic Health Records (EHR) — diagnoses, prescriptions, lab results, vitals
- Doctor schedule management — availability, leaves, slot management
- In-patient management — bed allocation, ward management, discharge
- Lab and radiology — order tests, receive results, attach to patient record
- Pharmacy — prescription fulfilment, drug inventory management
- Billing — itemised bill generation, insurance claim integration
- Operation Theatre (OT) scheduling
- Emergency/Casualty management
- Staff management — nurses, technicians, admin shifts
- Reports — patient outcomes, bed occupancy, revenue per department

---

Non-Functional Requirements

Scalability: 1000-bed hospital with 5,000 daily patient interactions. Multi-hospital chain: 100 hospitals, 500,000 patients. Scale per department independently.

Availability: 99.99% for clinical systems (EHR, pharmacy). A doctor being unable to access a patient's allergy history is life-threatening. Emergency systems are critical — run on separate infrastructure with dedicated power backup.

Low Latency & Performance: EHR load under 500ms. Drug interaction check under 200ms. Appointment slot display under 100ms. Lab result fetch under 300ms.

Reliability: Medical records must never be lost. Write-ahead logging on all clinical data. Backups every 6 hours. Cross-site replication for DR. HIPAA-compliant data handling (India: DISHA compliance).

Consistency: Strong consistency for bed allocation (two departments cannot allocate the same bed), drug dispensing (dispensed once per prescription), appointment booking (no

double-booking of a doctor's slot).

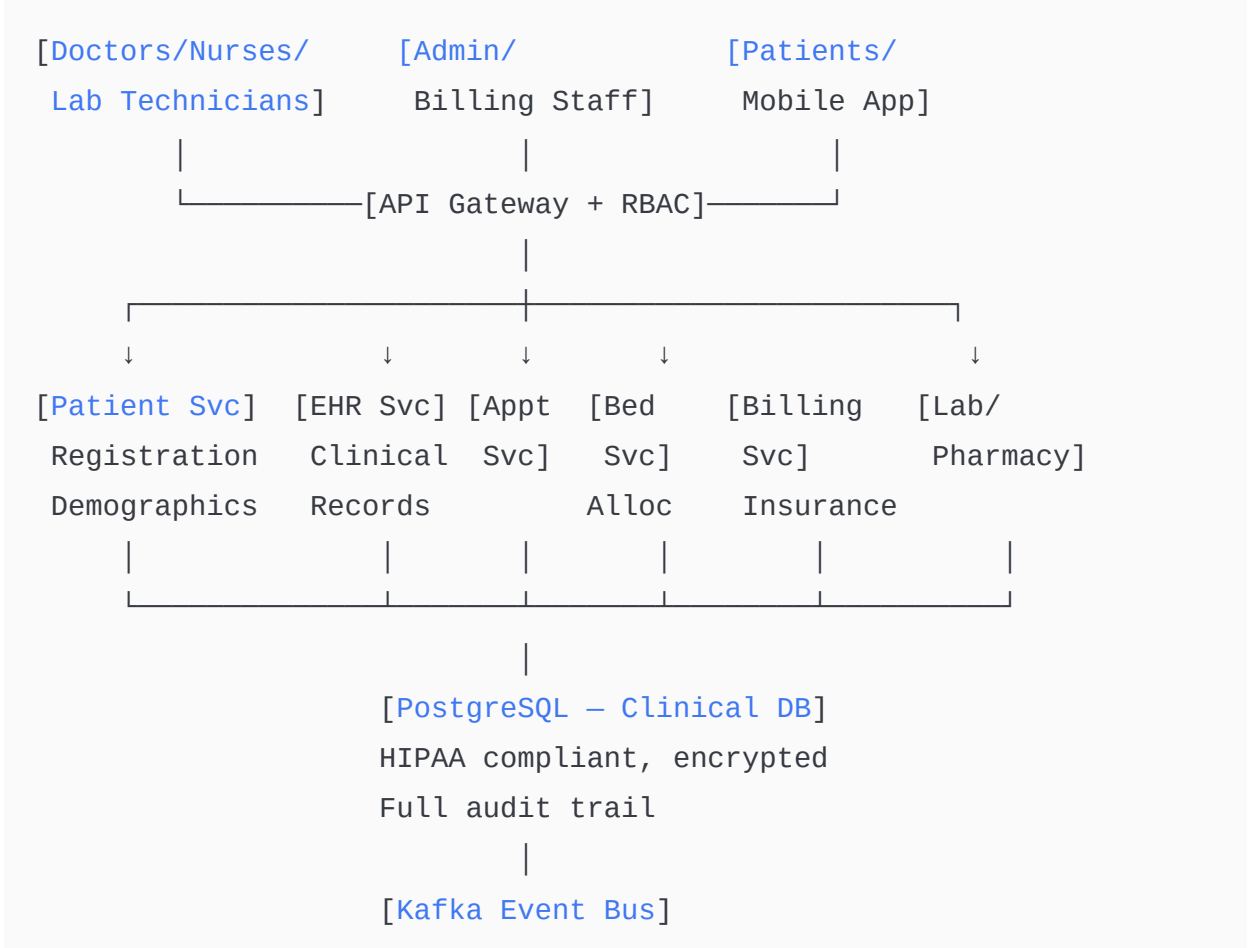
Security: HIPAA/DISHA compliance. Role-based access: doctors see their patients, nurses see assigned ward, admin sees billing. All access logged for audit. PHI (Protected Health Information) encrypted at rest and in transit.

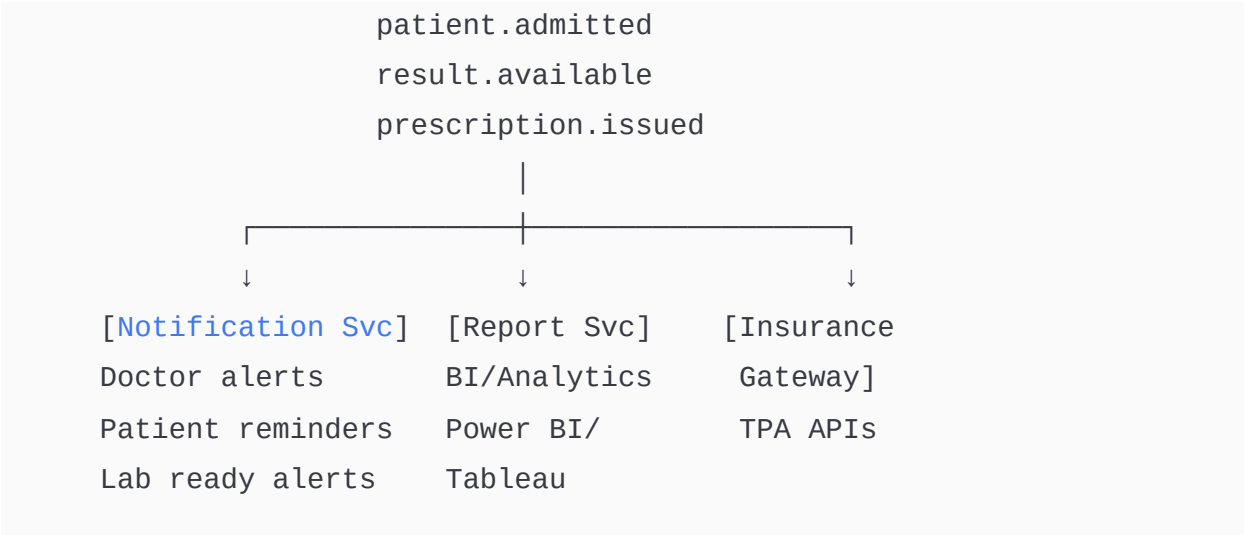
API Design

```
# Patient
POST   /v1/patients                              → Register patient
GET    /v1/patients/{patient_id}                 → Patient profile + summary
GET    /v1/patients/{patient_id}/history         → Full medical history (EHR)

# Appointments
GET    /v1/doctors/{doctor_id}/slots?date={}     → Available slots
POST   /v1/appointments                          → Book appointment
PUT    /v1/appointments/{id}/status              → Confirm/cancel/reschedule
GET    /v1/patients/{id}/appointments            → Patient's appointments

# EHR (Clinical)
POST   /v1/encounters                            → Create clinical encounter
POST   /v1/encounters/{id}/diagnosis             → Add diagnosis (ICD-10 code)
POST   /v1/encounters/{id}/prescriptions         → Issue prescription
POST   /v1/encounters/{id}/orders                → Order lab/radiology test

# Lab & Radiology
GET    /v1/orders/{order_id}                     → Order status + results
```

```
POST  /v1/orders/{order_id}/results          → Upload lab results
GET   /v1/patients/{id}/lab-results          → All lab results

# Billing
POST  /v1/bills                              → Generate bill
GET   /v1/bills/{bill_id}                    → Itemised bill
POST  /v1/bills/{bill_id}/payment            → Process payment
POST  /v1/bills/{bill_id}/insurance-claim    → Submit to insurer

# Beds
GET   /v1/beds?ward={}&status=AVAILABLE       → Available beds
POST  /v1/admissions                          → Admit patient to bed
PUT   /v1/admissions/{id}/discharge           → Discharge patient
```

HLD





Flow Explanation

Patient registration and OPD flow: Patient registers at reception → Patient Service creates record (demographics, insurance, emergency contact) → Patient gets UHID (Unique Health ID) → Books appointment (`POST /v1/appointments`) → Appointment Service checks doctor's slot availability (PostgreSQL row-level lock on slot) → confirms booking → sends reminder via Notification Service. At appointment time: doctor opens EHR → `GET /v1/patients/{id}/history` → reviews past diagnoses, allergies, medications → creates new encounter → adds diagnosis (ICD-10 coded) → issues prescription → prescription validated against Drug Interaction Service (checks for contraindications with current medications).

Inpatient (IPD) admission flow: ER doctor requests bed → Bed Service queries available beds by ward/type → acquires Redis lock on `bed_id` → creates admission record → updates bed status to `OCCUPIED` → nursing staff alerted → vitals monitoring linked to patient's EHR in real time. Discharge: doctor issues discharge summary → Billing Service generates itemised bill (collects charges from all departments: consultation, lab, pharmacy, OT, bed rent) → insurance claim submitted to TPA API → patient pays balance → bed status back to `AVAILABLE`.

Lab result flow: Doctor orders blood test → Lab Order created in DB with status `ORDERED` → Lab technician receives order on lab terminal → collects sample → runs test → uploads results to Lab System → `POST /v1/orders/{id}/results` → result stored and linked to patient EHR → `result.available` event published to Kafka → Doctor and patient notified.

Detailed Component Design

EHR Service — The clinical core

EHR is the most sensitive data in the system. Stored in PostgreSQL (ACID, strong consistency) with full encryption at rest (AES-256, column-level for PHI). Schema follows HL7 FHIR standard (international healthcare interoperability standard) — enables data exchange between hospitals and insurance systems. Every access to an EHR record is logged: `{user_id, patient_id, action, timestamp}` — HIPAA audit trail requirement. Soft delete only — a medical record can never be hard-deleted (legal requirement).

Drug Interaction Service

When a new prescription is issued, the Drug Interaction Service checks the new drug against all current active medications for that patient. Uses a drug database (RxNorm / CIMS India) stored in PostgreSQL and cached in Redis. Checks: contraindication (should not be taken together), dose duplication, allergy cross-reaction. Returns a severity-coded warning (INFO, WARN, CRITICAL). CRITICAL interactions block the prescription — doctor must acknowledge override. This is a synchronous call on the prescription API — patient safety cannot wait.

Bed Management

Bed is a finite, physical resource. A 1000-bed hospital has exactly 1000 beds. Double allocation is catastrophic. Architecture: Redis lock on `bed_id` during allocation (prevents simultaneous allocation by two departments). PostgreSQL as source of truth for bed status. Bed board (real-time availability dashboard) reads from Redis (refreshed every 30 seconds from PostgreSQL). Discharge workflow runs bed-cleaning protocol — bed status transitions through `OCCUPIED → CLEANING → AVAILABLE` (not instant).

Follow-up Expected Q&A

Q: How do you handle emergency cases that bypass appointment booking? A: Emergency module bypasses the appointment flow entirely. ER staff create a patient record

(or link to existing) and directly create an encounter. Bed allocation goes through a priority queue — emergency patients are given highest priority. The triage system assigns a severity code (P1–P4) and the system ensures P1 patients get a bed within 5 minutes (SLA alert if exceeded).

Q: How do you ensure a prescription is dispensed exactly once? A: Each prescription has a unique `prescription_id` and a status (`ISSUED → DISPENSED → COMPLETED`). The Pharmacy Service checks status before dispensing — if already `DISPENSED`, rejects. A unique constraint on `(prescription_id, dispensing_event)` prevents DB-level duplicates. Controlled substances additionally require two-pharmacist sign-off.

Q: How do you handle integration with health insurance companies? A: A dedicated Insurance Gateway Adapter handles TPA (Third Party Administrator) APIs — different insurers have different APIs (REST, SOAP, HL7). Pre-authorisation request sent at admission for IPD cases. Final claim submitted at discharge with itemised bill in FHIR/HL7 format. Claim status tracked via polling or webhook. Rejected claims routed to billing staff for manual review.

---

7. HEALTHCARE CLAIMS PROCESSING

Functional Requirements

- Patient submits claim (or hospital submits on behalf)
- Claim types: reimbursement (patient paid first), cashless (hospital directly paid)
- Document upload — discharge summary, bills, prescriptions, diagnostic reports
- Claim intake and registration — assign claim ID
- Eligibility verification — is the patient covered, is the treatment covered?
- Medical coding — ICD-10 diagnosis codes, CPT procedure codes
- Claim adjudication — approve, deny, or partially approve with reasons

- Payment processing — pay hospital or reimburse patient
- Claim status tracking — real-time status updates
- Appeals — patient/hospital can appeal a denied claim
- Fraud detection — identify suspicious patterns
- Regulatory reporting — IRDAI compliance (India), HIPAA (US)
- Bulk claims processing for corporate health policies

Non-Functional Requirements

Scalability: Process 1 million claims per day for a national insurer. Each step in the claims pipeline is independently scalable. Document processing (OCR, data extraction) is compute-heavy — auto-scale GPU workers.

Availability: 99.99% for claims intake and status checking. A claim submission failure means a patient may not get reimbursed — high stakes.

Low Latency & Performance: Cashless claim pre-authorisation under 30 minutes (for standard procedures). Reimbursement claim decision under 7 working days (regulatory SLA). Real-time eligibility check under 2 seconds. Claim status API under 200ms.

Reliability: A submitted claim must never be lost. Kafka guarantees at-least-once delivery through the processing pipeline. Each processing step is idempotent — if a worker crashes after processing but before committing, re-processing produces the same output.

Consistency: Strong consistency for payment — a claim is paid exactly once. Idempotency key on every payment disbursement. Eventual consistency for claim status updates — status may lag the actual processing step by a few seconds.

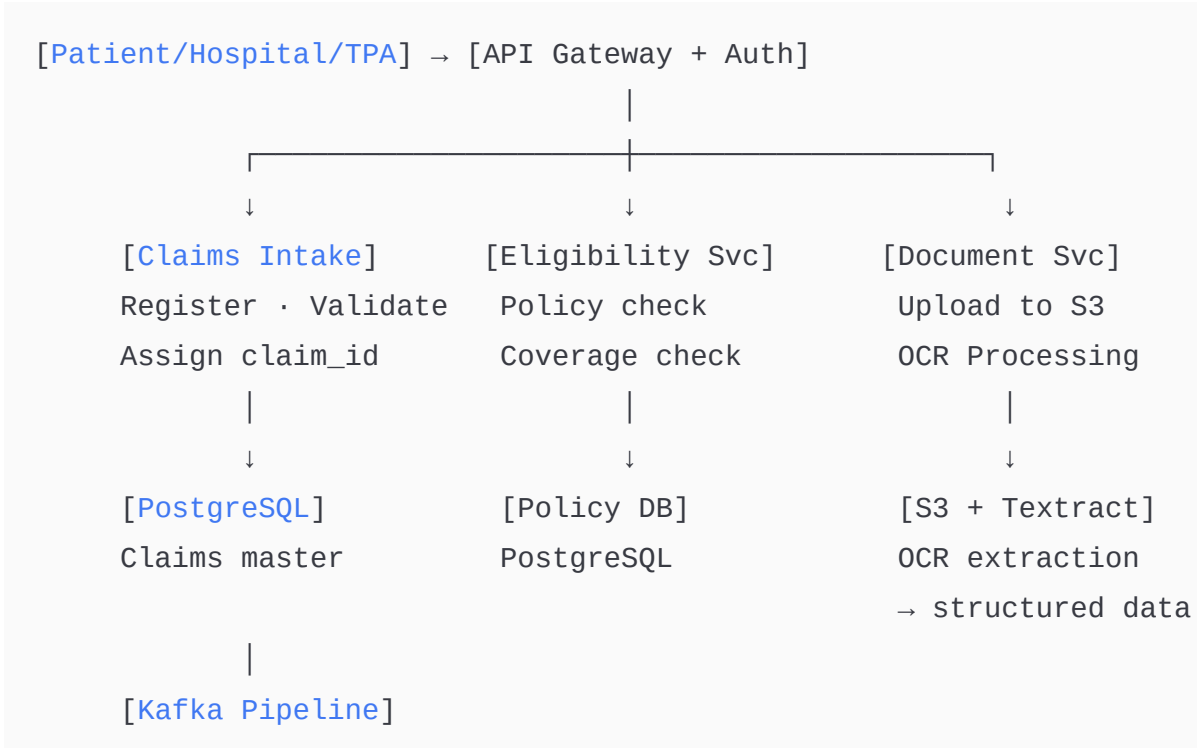
Audit & Compliance: Every action on every claim — who viewed it, who approved it, what was changed — is immutably logged. IRDAI requires 10-year retention of all claim records.

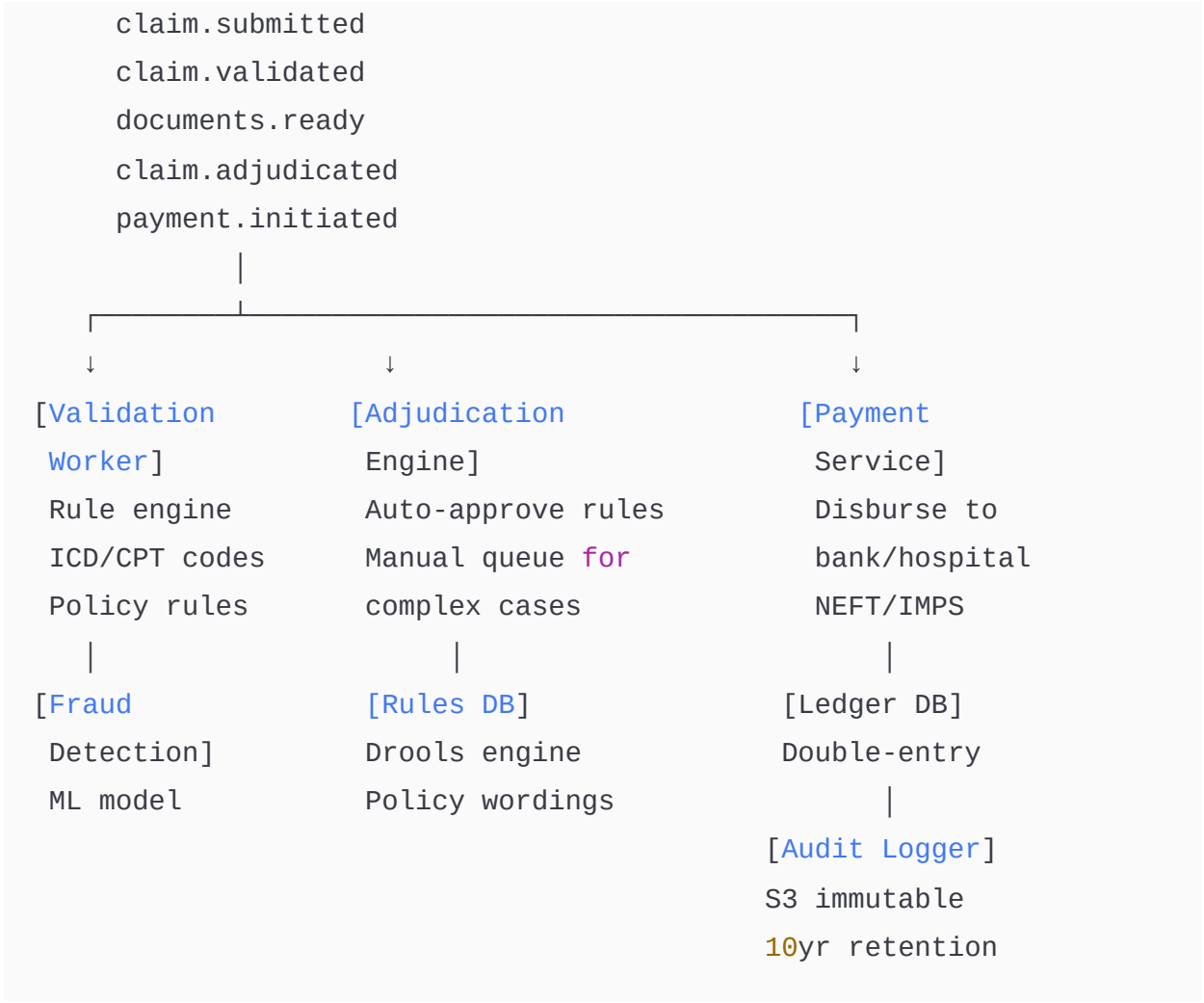
API Design

```
# Claim Submission
POST  /v1/claims                          → Submit new claim
(idempotency-key required)
Body: {
  policy_id: "POL001",
  patient_id: "PAT001",
  claim_type: "REIMBURSEMENT",
  treatment_date: "2024-01-15",
  hospital_id: "HOSP001",
  diagnosis_codes: ["J18.9"],
  procedure_codes: ["99213"],
  claimed_amount: 50000
}
Response: { claim_id, status: "SUBMITTED", reference_number }

# Document Upload
POST  /v1/claims/{claim_id}/documents    → Upload supporting
documents
Body: multipart/form-data (PDF, JPEG)
Response: { document_id, ocr_status: "PROCESSING" }

# Claim Status
GET   /v1/claims/{claim_id}              → Full claim detail +
current status
GET   /v1/claims/{claim_id}/timeline     → Status history with
timestamps
GET   /v1/claims?policy_id={}&status={}  → Claims list

# Adjudication (Internal/TPA)
PUT   /v1/claims/{claim_id}/adjudicate   → Record adjudication
decision
Body: {
  decision: "APPROVED/DENIED/PARTIAL",
  approved_amount: 45000,
  denial_reasons: ["CODE_NOT_COVERED"],
  adjudicator_id: "ADJ001"
}
```

```
# Payment
POST  /v1/claims/{claim_id}/payment      → Initiate payment
disbursement
GET   /v1/claims/{claim_id}/payment      → Payment status + UTR
number

# Appeals
POST  /v1/claims/{claim_id}/appeal       → Submit appeal
POST  /v1/claims/{claim_id}/appeal/documents → Upload appeal
documents
GET   /v1/claims/{claim_id}/appeal       → Appeal status

# Eligibility
POST  /v1/eligibility/check              → Real-time policy
eligibility check
Body: { policy_id, treatment_code, hospital_id, treatment_date }
```

HLD





Flow Explanation

Reimbursement claim flow: Patient uploads documents via portal → Claims Intake Service creates claim record in PostgreSQL → assigns claim_id → publishes `claim.submitted` to Kafka. Document Service downloads and stores PDFs in S3 → triggers AWS Textract (OCR) to extract structured data (hospital name, diagnosis, bill amounts, dates) → extracted data written back to claim record → publishes `documents.ready` to Kafka.

Validation pipeline: Validation Worker consumes `documents.ready` → runs rule engine (Drools): Is policy active on treatment date? Is hospital empanelled? Is diagnosis code covered under this plan? Is claim amount within policy limit? Is there a waiting period for this condition? Each rule pass/fail recorded in claim audit log. If all rules pass → publishes `claim.validated`. If any rule fails → claim status set to `QUERY` → claimant notified to provide additional documents or clarification.

Adjudication flow: Adjudication Engine consumes `claim.validated` → runs auto-adjudication rules: if claim amount < ₹50,000 AND standard procedure AND no fraud flag → auto-approve. If claim > ₹50,000 OR complex diagnosis OR fraud flag → route to manual adjudication queue. Manual adjudicator reviews on a dashboard, makes decision, adds denial codes or deductions. Decision published to Kafka → Payment Service triggered for approved claims.

Cashless pre-authorisation flow: Hospital requests pre-auth before procedure → Eligibility Service checks in real time: is patient covered? Is procedure covered? What is the sum insured remaining? → Returns pre-auth approval with approved amount and validity period (typically 7 days) → Hospital proceeds with treatment → Post-discharge: hospital submits final bill → compared against pre-auth → payment disbursed to hospital directly via NEFT.

Fraud detection flow: Every claim is scored by the Fraud Detection Service (runs asynchronously, does not block the submission). Features: claim frequency per policy (too many claims in short period), treatment date vs admission date consistency, diagnosis code vs procedure code plausibility, hospital billing pattern (is this hospital billing significantly above average for this procedure?), network analysis (is this claimant connected to known fraudulent claimants?). High-score claims flagged for investigator review — not auto-denied (false positives are costly to patient relationships).

Detailed Component Design

Claims Pipeline (Kafka-based)

The claims processing pipeline is a series of Kafka topics, each representing a processing stage. This gives: independent scalability per stage, fault isolation (a downstream failure doesn't affect intake), replay capability (re-process all claims from a date), and full observability (Kafka consumer lag shows where bottlenecks are). Each worker is stateless and idempotent — if it crashes after processing a claim but before committing the Kafka offset, reprocessing produces the same result (because the DB write checks for existing records with `INSERT ... ON CONFLICT DO NOTHING`).

Tradeoff: Kafka-based pipeline introduces latency (each stage has processing time + Kafka publish-consume cycle of 50–100ms). For cashless pre-auth (SLA: 30 minutes), this is fine.

For real-time eligibility (SLA: 2 seconds), the eligibility check is synchronous (direct DB call) — it does not go through Kafka.

Adjudication Engine (Drools Rules Engine)

Business rules for claims adjudication are encoded in a Drools rules engine (Java-based) rather than hardcoded. This allows business analysts to modify coverage rules without code deployments. Rules cover: policy coverage conditions (what is and isn't covered), deductibles and co-payments, waiting periods by condition, sub-limits by category (e.g. room rent capped at ₹5,000/day). Rules loaded from DB at startup, hot-reloaded without restart when updated.

Tradeoff: Rules engine adds operational complexity compared to simple if-else logic. Justified because insurance policy rules change frequently (new product launches, regulatory changes) and require non-developer involvement. Unit tests for rules are essential — a bug in a coverage rule can cause systematic incorrect denial or approval of thousands of claims.

Document Processing (OCR Pipeline)

Documents uploaded to S3 trigger a Lambda which calls AWS Textract for medical bill OCR. Textract returns structured JSON with extracted tables (itemised bills), key-value pairs (patient name, dates), and raw text. A post-processing transformer maps extracted fields to the claim schema (handling variation in bill formats across thousands of hospitals). Confidence scores < 0.8 for critical fields (amount, dates) route the document to a human reviewer. ML model fine-tuned on medical bill dataset improves extraction accuracy over time.

Payment Disbursement

Claims approved for payment enter the Payment Service. Beneficiary bank account verified via penny-drop verification (small ₹1 test transfer to confirm account exists) before first payment. Each payment generates a unique UTR (Unique Transaction Reference) for tracking. Payments batched for NEFT (4 settlement windows per day) or IMPS (immediate, 24/7) based on urgency. Every disbursement creates double-entry ledger records: debit from insurer's escrow account, credit to hospital/patient account. End-of-day reconciliation against bank statement.

Follow-up Expected Q&A

Q: How do you detect fraudulent claims? A: Multi-layer approach. Rule-based: claims with identical diagnoses and amounts from the same hospital within 30 days are flagged. ML-based: isolation forest model trained on historical claims detects statistical outliers. Graph-based: Neo4j graph analysis finds networks of patients/doctors/hospitals with suspicious co-occurrence patterns (staged accident rings). Flagged claims go to a Special Investigation Unit (SIU) queue — not auto-denied.

Q: What happens if a payment is initiated but the bank transfer fails? A: Payment Service uses idempotency key (claim_id + attempt_number). Bank transfer failure triggers a retry (3 attempts, 1-hour intervals). After 3 failures: payment marked `FAILED`, claim back to `ADJUDICATED` status, operations team alerted. Failed payment event published to Kafka → escalation workflow. NEFT failures return error codes — system retries in next NEFT batch window. No double-payment possible due to idempotency key.

Q: How do you handle a claim that is re-submitted after being denied? A: A denied claim generates a new claim_id (fresh record) with a reference to the original claim. Appeals are tracked separately from re-submissions. The original claim is never modified (immutable). The adjudicator sees the full history including previous denial and reason, and the additional documents provided. Appeals have a separate regulatory SLA (IRDAI: 30 days for appeal decision).

Q: How do you scale document processing for a peak of 100K claim submissions in a day? A: S3 event triggers Lambda (auto-scaling, up to 1000 concurrent Lambdas). Textract has its own managed scalability. Post-processing workers scale with Kafka consumer lag (Kubernetes HPA). The bottleneck is typically the rules engine (Drools) — run as a microservice behind a load balancer with 20 instances, each handling 50 claims/second = 1000 claims/second total throughput, well above the 1.15 claims/second average (100K/day).

Q: How do you ensure the audit log is tamper-proof? A: Audit events written to S3 with Object Lock (WORM — Write Once Read Many). S3 Object Lock prevents any deletion or modification for the retention period (10 years). Audit records also have a hash chain: each record includes the hash of the previous record — any tampering invalidates the chain. Periodic integrity checks verify the chain. Access to audit logs is restricted to compliance officers and requires MFA + approval workflow.

Complete Technology Summary Across All 7 Systems

System	Message Bus	Primary DB	Cache	Search	Special Tech
E-Commerce	Kafka	PostgreSQL (sharded)	Redis	Elasticsearch	Drools promo engine, Spark ML recommendations
Notification	Kafka (priority topics)	PostgreSQL	Redis dedup	N/A	FCM/APNs/Twilio/SendGrid, Handlebars templates
Banking	Kafka	PostgreSQL (ACID)	Redis sessions	N/A	Double-entry ledger, Saga, XGBoost fraud, WAL replication
Library Mgmt	Kafka	PostgreSQL	Redis (copy lock, reservations)	Elasticsearch	Redis sorted set reservation queue
Shopping Cart	Kafka	PostgreSQL (backup)	Redis (primary)	N/A	Redis Hash cart, guest-account merge
Hospital Mgmt	Kafka	PostgreSQL (encrypted)	Redis (bed lock)	N/A	HL7 FHIR, Drug interaction DB, RBAC
Claims Processing	Kafka pipeline	PostgreSQL	Redis eligibility	N/A	Drools rules engine, AWS Textract OCR, Neo4j fraud graph, WORM audit

5/7/2026, 11:35:45 AM